STUDY TEXTBOOK

BOOK TITLE

Subtitle

First edition · 10 October 2026

Contents

1 Style smoke test **1**

1.1 Environments . . . 1

1.2 A graph . . . 2

1.3 A diagram, step by step . . . 2

1.4 A circuit . . . 2

1.5 Code with output . . . 3

Notes . . . 3

2 How a program becomes a running thing **4**

2.1 The four-stage journey . . . 4

2.2 Object files and ELF sections . . . 6

2.3 Symbols and relocation . . . 7

2.4 The loader and `execve` . . . 8

2.5 Where Book 5's CPU meets real silicon . . . 9

2.6 A map of the book . . . 10

2.7 Common mistakes . . . 10

Exercises . . . 11

Summary . . . 12

Further reading . . . 13

Watch instead . . . 13

Notes . . . 13

3 How the machine sees data **14**

3.1 Bytes, words and addresses . . . 14

3.2 Two's complement, in one page . . . 15

3.3 Endianness: which byte goes first . . . 15

3.4 Alignment and struct layout . . . 16

3.5 IEEE-754 in one breath . . . 17

3.6 Pointers as typed addresses; arrays vs pointers . . . 18
3.7 Common mistakes . . . 19
Exercises . . . 19
Summary . . . 21
Further reading . . . 21
Watch instead . . . 21
Notes . . . 22

4 RV32I assembly I: arithmetic, memory, control 23
4.1 The register file and the ABI names . . . 23
4.2 Six formats, one discipline . . . 24
4.3 Loads, stores and addressing . . . 25
4.4 Branches and comparisons . . . 26
4.5 Assembler directives and running it . . . 26
4.6 Common mistakes . . . 28
Exercises . . . 28
Summary . . . 30
Further reading . . . 30
Watch instead . . . 31
Notes . . . 31

5 Functions and the Stack 32
5.1 The contract in one paragraph . . . 32
5.2 The stack frame . . . 32
5.3 Caller-saved, callee-saved . . . 33
5.4 What a call costs . . . 34
5.5 Reading real compiler output . . . 35
5.6 PIC in one page . . . 36
5.7 Common mistakes . . . 36
5.8 Summary . . . 37
5.9 Exercises . . . 38

6 A Single-Cycle RV32I Core 40
6.1 Fetch–decode–execute, properly . . . 40
6.2 Decode and the immediate generator . . . 40

6.3 Register file and ALU — Book 5, widened . . . 41
6.4 Loads, stores and the data-memory interface . . . 42
6.5 Running it: 13 words, checked state . . . 42
6.6 Why single-cycle wastes time . . . 43
6.7 Common mistakes . . . 44
6.8 Summary . . . 44
6.9 Exercises . . . 45

7 Pipelining: One Per Cycle (Usually) 47
7.1 Why pipeline . . . 47
7.2 The five stages and what the registers carry . . . 47
7.3 Timing diagrams . . . 48
7.4 Where it breaks: hazards, shown . . . 49
7.5 Common mistakes . . . 50
7.6 Summary . . . 50
7.7 Exercises . . . 51

8 Hazards: Forwarding, Stalls, Branches 53
8.1 Forwarding: don't wait for writeback . . . 53
8.2 Load-use: the one stall you cannot avoid . . . 54
8.3 Branches: predict, verify, flush . . . 54
8.4 The debugging receipts . . . 55
8.5 Common mistakes . . . 56
8.6 Summary . . . 56
8.7 Exercises . . . 57

9 Traps, memory-mapped IO, and a program that talks to the world 59
9.1 Two small additions, one big change of character . . . 59
9.2 Minimal traps: `ecall` and `mret` . . . 59
9.3 The UART: one shift register with an address . . . 61
9.4 The program, end to end . . . 62
9.5 Receipts: two bugs this chapter paid for itself . . . 63
9.6 What it costs in silicon . . . 63
9.7 Common mistakes . . . 64
9.8 Summary . . . 65
Exercises . . . 65

10 The memory hierarchy **68**

10.1 Why there is a hierarchy at all . . . 68
10.2 The memory mountain, measured . . . 69
10.3 Locality: the only two tricks caches know . . . 70
10.4 Bandwidth has a stride too . . . 71
10.5 The contract . . . 72
10.6 Common mistakes . . . 72
10.7 Summary . . . 72
Exercises . . . 73

11 Caches: geometry, misses, and the loop that got 3× faster **76**

11.1 Anatomy: three parameters, one equation . . . 76
11.2 Replacement and the three Cs . . . 77
11.3 The simulator . . . 78
11.4 The payoff: blocking . . . 78
11.5 Software-visible effects beyond tiling . . . 79
11.6 Common mistakes . . . 79
11.7 Summary . . . 80
Exercises . . . 80

12 Virtual memory: the address is a lie **83**

12.1 Why virtualize . . . 83
12.2 The walk: sv32 as the concrete case . . . 83
12.3 The TLB: caching the answer . . . 84
12.4 Demand paging and the fault, measured . . . 84
12.5 fork + copy-on-write, and the fence . . . 85
12.6 Common mistakes . . . 85
12.7 Summary . . . 86
Exercises . . . 87

13 Processes and scheduling **89**

13.1 What a process is . . . 89
13.2 fork, exec, wait — the correct version . . . 89
13.3 The context switch, step by step . . . 90
13.4 Scheduling: four policies, measured on one job set . . . 91

13.5 The twenty lines that actually do it . . . 92
13.6 Common mistakes . . . 92
13.7 Summary . . . 92
Exercises . . . 93

14 Concurrency and synchronisation **96**
14.1 The race, demonstrated . . . 96
14.2 The atomic ladder . . . 96
14.3 Three fixes, one measurement . . . 97
14.4 Condition variables, done right . . . 98
14.5 Deadlock and lock ordering . . . 98
14.6 What concurrency actually buys . . . 98
14.7 Common mistakes . . . 99
14.8 Summary . . . 99
Exercises . . . 100

15 File systems and persistence **103**
15.1 Names, inodes, and the API . . . 103
15.2 The layers a read() descends . . . 104
15.3 The buffer cache, measured . . . 104
15.4 fsync: the syscall that pierces the cache . . . 104
15.5 Journaling: never half-written . . . 105
15.6 SSDs: why random writes hurt . . . 105
15.7 Common mistakes . . . 106
15.8 Summary . . . 106
Exercises . . . 107

16 Measuring performance correctly **109**
16.1 The folklore audit . . . 109
16.2 The performance equation . . . 109
16.3 Latency is a distribution . . . 110
16.4 The four lies . . . 110
16.5 The measurement loop . . . 110
16.6 The measured latency ladder . . . 110
16.7 Common mistakes . . . 110
16.8 Summary . . . 111
Exercises . . . 112

17 Vectorisation and data parallelism **114**
17.1 Data-level parallelism . . . 114
17.2 One instruction, four lanes . . . 114
17.3 SAXPY, measured . . . 115
17.4 The folklore item that fails . . . 115
17.5 SoA vs AoS: the layout is the vector . . . 115
17.6 The three vector ISAs . . . 116
17.7 Common mistakes . . . 116
17.8 Summary . . . 116
Exercises . . . 117

18 Low-latency design patterns **120**
18.1 The folklore audit . . . 120
18.2 Search at both scales . . . 121
18.3 The allocation myth, priced . . . 121
18.4 The SPSC ring . . . 121
18.5 The cost sheet . . . 122
18.6 Common mistakes . . . 122
18.7 Summary . . . 122
Exercises . . . 123

19 An interpreter and a compiler in miniature **126**
19.1 The divide . . . 126
19.2 The interpreter: a stack machine . . . 126
19.3 The compiler: emit once . . . 127
19.4 The JIT compromise . . . 127
19.5 The loop closes . . . 128
19.6 Common mistakes . . . 128
19.7 Summary . . . 128
Exercises . . . 129

CHAPTER 1

Style smoke test

This chapter exists only to prove that every environment, figure type and code block compiles. Delete it when the first real chapter exists. The model for structure is [1].

1.1 Environments

Definition 1.1 (Function)**.** A *function* $f : A \to B$ assigns to each $a \in A$ exactly one $f(a) \in B$.

Theorem 1.2 (Sum of the first n integers)**.** *For every* $n \geq 1$, $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

Proof. By induction on n. For $n = 1$ both sides are 1. If it holds for n, then adding $n+1$ gives $\frac{n(n+1)}{2} + n + 1 = \frac{(n+1)(n+2)}{2}$. ∎

Lemma 1.3. *A small lemma.*

Proposition 1.4. *A small proposition.*

Example 1.5. Worked example: strategy first, then solution.

Solution. The answer is 42.

Remark. A remark.

Exercise 1.1 (easy)**.** Show that $\sum_{k=1}^{n} (2k-1) = n^2$.

Exercise 1.2 (hard)**.** Prove the theorem above without induction.

PRINCIPLE

A principle box.

DECISION RULE

A rule box.

TRIPWIRE

A warning box.

DEFINITION OF DONE

A done box.

Checkpoint 1.1 *Try it.* Run the code in Listing 1.1 and predict the output first.

1.2 A graph

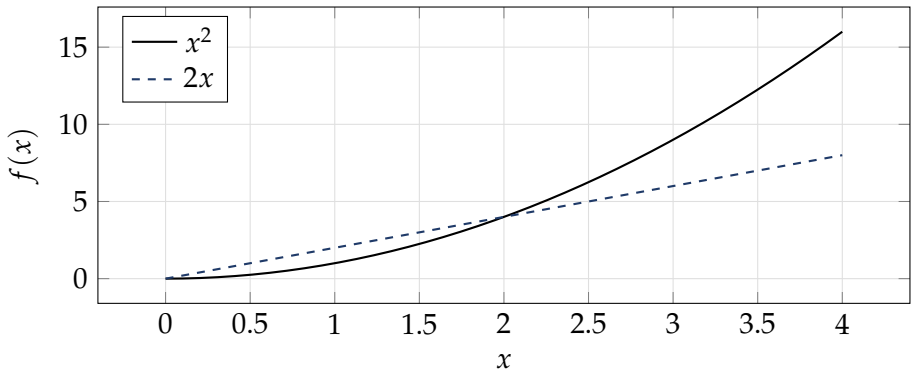


Figure 1.1: A pgfplots graph. Notice that x^2 overtakes $2x$ after $x = 2$. Line styles, not only colour, tell the curves apart.

1.3 A diagram, step by step

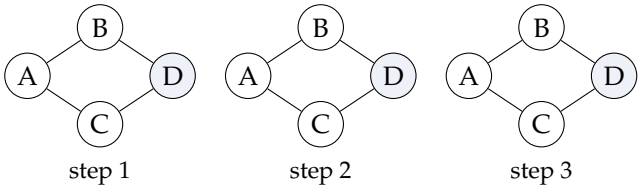


Figure 1.2: Small multiples show the state of an algorithm at three successive steps.

1.4 A circuit

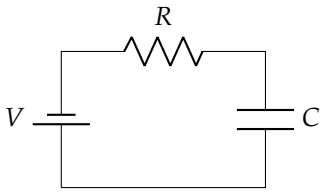


Figure 1.3: An RC circuit drawn with circuitikz. Every circuit in the book is drawn this way.

1.5 Code with output

```
# why: show the loop invariant, not just the loop
total = 0
for k in range(1, 6):
    total += k
print(total)
```

Listing 1.1: A short listing with line numbers.

Output of a real run:

```
15
```

Name	Meaning	Cost
Sort	Order the items	$O(n \log n)$
Scan	Visit each item once	$O(n)$

Table 1.1: Cheat sheet table: each chapter ends its main text with one of these.

Notes

Sources are cited by number [1].

CHAPTER 2

How a program becomes a running thing

You wrote a CPU. In Book 5 you designed an 8-bit ALU, wired it into a datapath, and drove it with a two-state FSM until Fibonacci numbers came out of a serial pin. That machine is small enough to hold in your head — 302 synthesis cells, ten opcodes, two states — but it is not a toy. The CPU you are using right now runs *more* code than yours, not *different* code. Somewhere between `int main(void)` and the transistors of your cpu8 there is a journey every program takes, and this book is about travelling it on foot. This chapter walks the first mile: how source text becomes a file of machine code, how that file gets into memory, and where your cpu8 meets the real silicon. Along the way you will compile, disassemble and link *real* binaries and read their real output — nothing in this chapter is simulated.

2.1 The four-stage journey

Type `cc sum.c -o sum` and four programs run in sequence, each handing a new file to the next. The names differ between toolchains — `gcc`, `clang`, `cc` — the shape does not:

1. **Preprocess** — text in, text out. `#include` splices header files in place; `#define` substitutes macros; conditionals (`#ifdef`) delete code before the compiler ever sees it. Output: a "translation unit", still plain C.
2. **Compile** — the translation unit becomes assembly: mnemonics, register names, labels — the same language your cpu8 assembler spoke, with a bigger vocabulary. Output: `.s`.
3. **Assemble** — mnemonics become machine-code bytes plus bookkeeping (which bytes are code, which are data, which names are exported). Output: an *object file*, `.o` — already ELF, just not yet executable.
4. **Link** — object files are stitched together: symbols resolved, sections merged, addresses assigned, one entry point recorded. Output: an executable ELF.

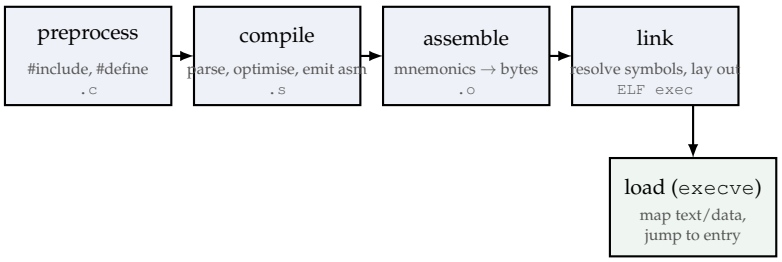
Example 2.1 (The pipeline, hands on)**.** Here is the whole journey on `code/ch01_sum.c`, a six-line freestanding sum. Each stage was run on the dev box (Apple clang 21, targeting x86-64 Linux so the output is real ELF):

```
clang -E ch01_sum.c > ch01_sum.i # 1: preprocess
clang -S ch01_sum.i # 2: compile -> ch01_sum.s
clang -c ch01_sum.s # 3: assemble -> ch01_sum.o
ld.lld -o ch01_sum.elf ch01_start.o ch01_sum.o # 4: link
```

Stage 2 emits assembly you can already read. Real output from `clang -S` (first lines of `sum`):

```
sum: # @sum
        pushq %rbp
        movq %rsp, %rbp
        movq %rdi, -8(%rbp)
        movl %esi, -12(%rbp)
        movl $0, -16(%rbp) # s = 0
        movl $0, -20(%rbp) # i = 0
.LBB0_1: # loop:
        movl -20(%rbp), %eax # eax = i
        cmpl -12(%rbp), %eax # compare i, n
        jge .LBB0_4 # done when i >= n
        movq -8(%rbp), %rax
        movslq -20(%rbp), %rcx
        movl (%rax,%rcx,4), %eax # eax = a[i]
        addl %eax, -16(%rbp) # s += a[i]
```

Every line is one instruction the CPU will execute; `-8(%rbp)` is the array pointer spilled to the stack exactly the way your cpu8's indirect addressing would have handled it.



Four tool invocations, one machine read of the result. The .o is already ELF — linking only decides where each byte goes; loading only decides which page it lands on.

Figure 2.1: The four-stage journey from source to running program. Each stage's product is a file you can inspect; none is magic.

Checkpoint 2.1 In one sentence each: what does the preprocessor hand to the compiler, and what does the assembler hand to the linker?

Solution. The preprocessor hands *expanded C source* (one translation unit, no `#`-lines) to the compiler. The assembler hands the linker an *object file*: machine-code bytes plus a symbol table and relocation records — not yet a program, but already ELF.

THE PIPELINE HAS FOUR STAGES, NOT ONE

When a student writes "gcc compiles my program", they hide three of the four jobs under one word. You will meet students who cannot tell a *compile* error (bad syntax) from a *link* error (missing symbol) — the fix is different every time, and the fix-time is the difference between seconds and an afternoon.

2.2 Object files and ELF sections

An object file is not a blob: it is a filing cabinet. The bytes from stage 3 arrive sorted into *sections*, each with a name, a size, a job. Compile the sum program and look inside — real output, dev box, clang 21 targeting x86-64 ELF:

```
$ objdump -h ch01_sum.o
Sections:
Idx Name Size VMA Type
  1 .strtab 00000073 0000000000000000
  2 .text 00000076 0000000000000000 TEXT
  3 .rela.text 00000030 0000000000000000
  4 .data 00000014 0000000000000000 DATA
  7 .eh_frame 00000058 0000000000000000 DATA
  8 .rela.eh_frame 00000030 0000000000000000
```

Read the four that matter:

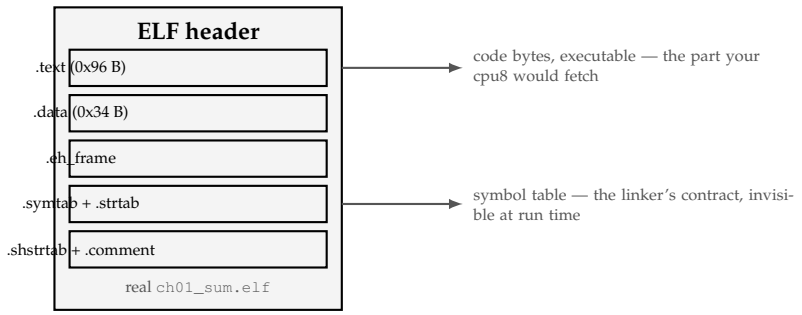
- `.text` — 0x76 = 118 bytes of machine code. This is what a CPU's instruction fetch unit reads. It is called "text" for the same reason a teletype's paper tape was: the name is older than most programmers, and it means *code*, not English.
- `.data` — 0x14 = 20 bytes of initialised data: the five-element array `v` inside `main`, stored as real bytes.
- `.rela.text` — 48 bytes of relocation records: places in `.text` the linker must patch once it knows the final addresses.
- `.symtab` (hidden in this excerpt, present in the file) — the symbol table: the names `sum`, `main`, whether each is defined or referenced, and where.

Example 2.2 (Find `main`'s machine code). `objdump -d` walks the `.text` section and prints every instruction with its offset and its encoding bytes. Real output for `main` in `ch01_sum.o`:

```
0000000000000050 <main>:
  50: 55 pushq %rbp
  51: 48 89 e5 movq %rsp, %rbp
  54: 48 83 ec 10 subq $0x10, %rsp
  58: c7 45 fc 00 00 00 00 movl $0x0, -0x4(%rbp)
  5f: 48 8d 3d 00 00 00 00 leaq (%rip), %rdi # 0x66
  66: be 05 00 00 00 movl $0x5, %esi
  6b: e8 00 00 00 00 callq 0x70 <main+0x20>
  70: 48 83 c4 10 addq $0x10, %rsp
  74: 5d popq %rbp
  75: c3 retq
```

Three things to notice. First, the middle column is the machine code — `55`, `48 89 e5`: the exact bytes Book 5 ch16's ROM would have held. Second, `callq` at `6b` encodes `00 00 00 00` for its target: the assembler left a hole because it did not yet know where `sum` would live — the linker's relocation step fills it. Third, `movl $0x5, %esi` is your accumulator pattern: load a constant into a register, then hand it to the call.

.TEXT IS NOT TEXT, .DATA IS NOT "ALL THE DATA"



An ELF is a table of contents plus the bytes each section points at: same bytes, four different jobs.

Figure 2.2: The section map of the real `ch01_sum.elf`: code, data, bookkeeping — one file, four jobs.

New readers lose hours to naming. `.text` = instructions. `.data` = only *initialised writable* globals. `.bss` = initialised-to-zero globals (not stored in the file at all — the loader zero-fills it, which is why a 1 MB zero array costs no disk space).

2.3 Symbols and relocation

The symbol table is the linker's contract. Two kinds of entries decide whether a program links: *defined* symbols (the file provides them) and *undefined* symbols (the file promises to get them from somewhere else). `objdump -t` shows both:

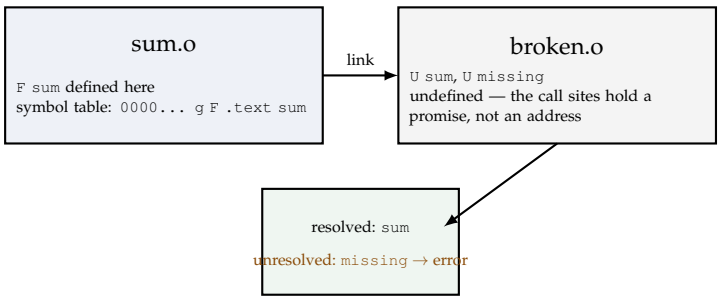
```
$ objdump -t ch01_sum.elf # after linking
00000000002011c0 g .text _start
0000000000202260 g .data data
00000000002011e0 g F .text sum # defined function
0000000000201230 g F .text main # defined function
```

While the object is still a `.o`, the assembler records a *relocation* for every address it could not finish. Real `objdump -r` on `ch01_broken.o`:

```
RELOCATION RECORDS FOR [.text]:
OFFSET TYPE VALUE
0000000000000012 R_X86_64_PC32 .data-0x4
000000000000001c R_X86_64_PLT32 sum-0x4
0000000000000024 R_X86_64_PLT32 missing-0x4
```

Each row says: "patch `.text` at this offset, against the final address of that symbol." The linker runs through these records, looks up every undefined symbol in every other object's symbol table, and either patches the hole — or reports the failure you saw live:

```
$ ld.lld -o broken ch01_broken.o
ld.lld: error: undefined symbol: sum
>>> referenced by ch01_broken.c
>>> ch01_broken.o:(main)
ld.lld: error: undefined symbol: missing
>>> referenced by ch01_broken.c
>>> ch01_broken.o:(main)
```



Relocation = the linker's bookkeeping: every undefined `U` must find a defined `F/g` in another object — or the real error you just saw appears.

Figure 2.3: Relocation is bookkeeping: the assembler records holes, the linker resolves them — or names the missing symbol.

Example 2.3 (Hand-link, and watch the error message change)**.** Link `ch01_sum.o` alone and `ld.lld` reports `undefined symbol: sum` plus `undefined symbol: missing` — two separate records, two separate complaints. Add `ch01_sum.o` to the link line and `sum` disappears from the error list but `missing` remains. That is the whole game: undefined symbols are holes to fill; a link is complete when every hole has a defined home.

Checkpoint 2.2 Your build reports `undefined symbol: __stack_chk_fail`. Is this a compile error or a link error, and what does the linker want?

Solution. A link error — the compiler emitted a call to a helper that exists in the C library but is not on your link line. The linker wants any object file or library that *defines* that symbol; adding it is the fix, not editing your source.

2.4 The loader and `execve`

Linking produced a file; something still has to make it *run*. That is the loader's job, and on Linux it is invoked by `execve`: the kernel opens the ELF, checks the magic bytes `\x7f E L F`, builds a fresh address space, maps the loadable segments to virtual pages, sets the program counter to the entry point, and hands the CPU to it. After the link above, the real executable's sections look like this:

```
$ objdump -h ch01_sum.elf
Sections:
Idx Name Size VMA Type
  1 .eh_frame 0000005c 0000000000200158 DATA
  2 .text 00000096 00000000002011c0 TEXT
  3 .data 00000034 0000000000202260 DATA
  5 .symtab 000000a8 0000000000000000
```

Compare with the `.o`: the `VMA` column is no longer zero. The linker assigned real addresses — `.text` at `0x2011c0` — and the loader maps those sections at exactly those virtual addresses. The program counter lands at `_start` (`0x2011c0`), and the machine runs.

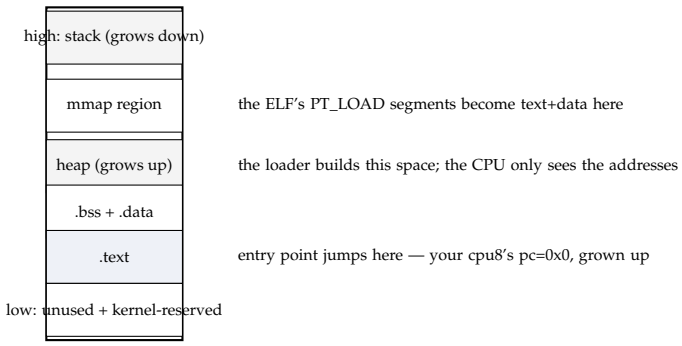


Figure 2.4: The process address space the loader builds: the ELF's segments become mapped pages; the CPU reads only addresses.

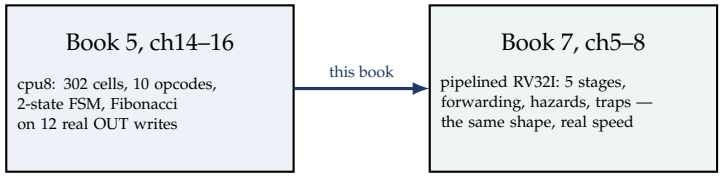
Example 2.4 (Where the map meets the metal)**.** On your cpu8, `pc=0x0` meant ROM offset zero. On a real machine, `pc=0x2011c0` means "page `0x20000` plus `0x1c0`", and a page table translates that to DRAM. The idea is identical — *the CPU fetches from an address, and something decides what lives there*. What changed is who decides: on your cpu8, you did; on Linux, the page table your OS maintains does. That is the one sentence this book is about.

2.5 Where Book 5's CPU meets real silicon

Book 5's cpu8 and a RISC-V RV32I are the same machine at different scales. The mapping table you already know:

cpu8 (Book 5 ch14–16)	**RV32I (this book)**
8-bit regs, r0 = accumulator A	32 32-bit regs, x0 wired to 0
10 opcodes, 4-bit op field	RV32I base: 37 instructions, 7-bit opcode
2-state FSM: fetch, execute	5-stage pipeline: IF ID EX MEM WB
`JMP`/`JZ` flags	`beq`/`bne`/…compare-and-branch
16-byte ROM, `pc=0x0`	virtual memory, `pc` = entry VA
UART prints a byte	`ecall` asks the OS to print

The right column is not a different kind of computer — it is the same datapath+FSM shape with more bits and more stages. Chapters 5–8 build the pipelined RV32I core for real; this section is the signpost that says you have already built its skeleton once.

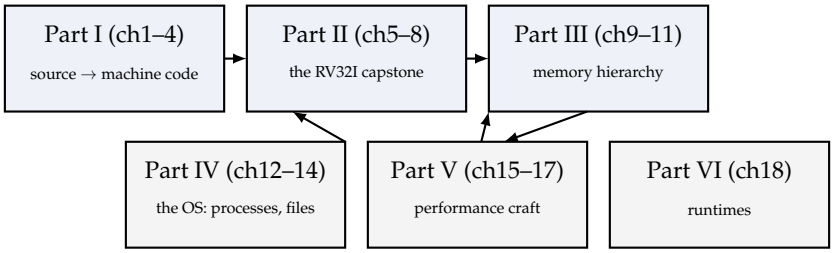


The hand-off this book exists for: the machine you built proves the shape; the chapters ahead prove it at scale.

Figure 2.5: The hand-off: Book 5 proved the shape; this book proves it at scale.

2.6 A map of the book

Six parts, one question each. The order matters: each part runs the tools the previous one installed.



The path this chapter starts: build the real machine around the toy CPU you already made, then measure what the folklore got wrong.

Figure 2.6: The roadmap: source → machine code → a real core → memory → the OS → craft.

DEFINITION OF DONE

- You can name the four pipeline stages and what each produces.
- You can read `objdump -h/-d/-t/-r` output and find code, data, symbols and relocations.
- You know a *link* error from a *compile* error and how to fix each.
- You can place Book 5's cpu8 inside a real address space and name the piece of hardware that made it real: the page table.

2.7 Common mistakes

- **"Compile" is one step.** It is four. Students lose hours calling a linker error a compiler bug; the fix is always different.
- **`.o` is "half a program".** It is a whole ELF, just not yet executable — sections, symbols, relocations are already real. Nothing is created at link time except the *layout*.
- **`.bss` wastes disk.** It does not exist in the file; the loader zero-fills pages. A gigabyte of zeros costs zero bytes on disk.

- **Disassembly "proves" optimisation.** `objdump -d` shows bytes; -O0 vs -O2 differences are layout and constants, not magic. Measure before attributing speed.
- **"The CPU runs the file."** The CPU runs *mapped pages*; the file is only the blueprint. `execve` is the step that turns one into the other.

Exercises

Exercise 2.1. On `ch01_sum.o`, which section holds the machine code, and which holds the relocation records that patch it?

Solution. [hint] `objdump -h` lists both by name: the code lives in one, the patch list in another whose name starts with the code's.

Exercise 2.2. `objdump -t` on `ch01_sum.elf` lists `sum` as `g F .text`. What do the two flags mean?

Solution. [hint] One flag says "global" (visible outside this file); the other says "function". Undefined entries would show `U` instead.

Exercise 2.3. A program's `.bss` is 500 KB but its file on disk is 40 KB. Why is this not a contradiction?

Solution. [hint] `.bss` records a size and a destination, not bytes; the loader zero-fills the pages it names.

Exercise 2.4. Your build prints `undefined symbol: main` (from `ld.lld`). Which of the four pipeline stages failed?

Solution. [hint] The stage that resolves names — the one that runs *after* `.o` files exist.

Exercise 2.5. `callq` in `main` encodes `00 00 00 00`. In one sentence each: who left that hole, and who fills it?

Solution. [hint] The stage that did not know the address left it; the stage that learns final addresses fills it via a relocation record.

Exercise 2.6. Compare `objdump -h` on `ch01_sum.o` and `ch01_sum.elf`: what did the linker add, and what stayed the same?

Solution. [hint] Addresses (VMA) appeared; section *names* and *jobs* did not change. One section disappeared into runtime metadata.

Exercise 2.7. `execve` maps `.text` read-execute and `.data` read-write. Which single bit of your cpu8's memory would have had to become two bits, and why?

Solution. [hint] The page-table permission bits your flat RAM did not need — `.text` must not be writable, `.data` must not be executable.

Exercise 2.8. `-O0` vs `-O2` on `ch01_sum.c` changed `.text` from 0x76 to 0x96 bytes. Was the faster version larger or smaller here, and what does that tell you about "optimise = smaller"?

Solution. [hint] Faster was *larger* (0x96 > 0x76) — unrolling/inlining trades size for speed. Optimisation is about time, not bytes.

Exercise 2.9 (hard)**.** Hand-link the two objects from Example 2.3 and explain, per relocation record, which hole each resolves and which it cannot. Then name the single symbol whose absence forces the linker to fail.

Solution. [hint] The `PC32` record resolves within the same file; the two `PLT32` records need other objects: the `sum` record is satisfied once the second object joins the link, while the `missing` record has no home anywhere — that one is the error.

Exercise 2.10 (hard)**.** Design the minimum change to your cpu8 that would let it run an ELF `.text` section directly. What two pieces of metadata does `.text` carry that your ROM already has, and what two does your ROM lack?

Solution. [hint] Your ROM already has bytes and an implied base address. It lacks a `VMA` (which virtual page it lands on) and a protection mode (read-execute vs read-write) — the two fields that make an ELF a program rather than a tape.

Exercise 2.11 (hard)**.** A student claims the entry point `_start` is "where the OS starts the program". Correct the claim in one sentence, then name the field in the ELF header that actually points to the first instruction.

Solution. [hint] `_start` is not magic — it is the symbol the linker recorded in the ELF header's `e_entry` field; the loader reads that address and jumps to it, whoever named the symbol.

Summary

Stage	**What it produces**
Preprocess	expanded C (one translation unit)
Compile	assembly (`.s`)
Assemble	ELF object (`.o`): sections + symbols + relocations
Link	executable ELF: resolved symbols, assigned VMAs
Load (`execve`)	mapped pages, `pc` → entry VA
You can now	
read `objdump`	code, data, symbols, relocations at a glance
name the failure	compile vs assemble vs link vs load — different fixes
place cpu8 in the map	page table is the piece that makes it real

Further reading

- Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, ch. 7 "Linking" — the standard walk-through of symbols and relocation this chapter compresses.
- Ulrich Drepper, "How To Write Shared Libraries", §1 (ELF basics, sections vs segments). `https://www.akkadia.org/drepper/dsohowto.pdf`
- Compiler Explorer documentation — try `clang -S` on your own sum online and match the output to Example 2.1. `https://github.com/compiler-explorer/compiler-explorer`
- The ELF specification (System V ABI, §4–5) — the byte-level reference for every field `objdump` printed.
- Linux `execve(2)` and `elf(5)` man pages — the loader's side of the contract.

Watch instead

DECISION RULE

Watch instead. If a video explains it faster than the prose, watch these and come back:

- "In-depth: ELF — The Extensible & Linkable Format" — stacksmashing, 19:02. Byte-level walk of the format this chapter uses; matches Sections 2.2–2.3. `https://youtu.be/nC1U1LJQL8o`
- "Writing a Linux ELF Binary by Hand" — Simon Racz, 59:11. Builds the file from hex — the same journey with no toolchain. `https://youtu.be/JM9jX2aqkog`

Notes

CHAPTER 3

How the machine sees data

Chapter 1 followed a program from text to running code. This chapter asks the companion question: once it is running, *what does the machine think the data is*? The honest answer is short and unsettling: nothing. A memory cell holds bytes; a register holds bits. Types — signed or unsigned, integer or pointer or float — are agreements between your source code and the generated instructions, and the CPU keeps no record of them. Most of this chapter is spent making that concrete: we decode the same four bytes three ways, measure what struct layout really costs, and run an alignment experiment whose result surprised us — the numbers are real, and they disagree with the folklore. That disagreement is the lesson.

3.1 Bytes, words and addresses

A *byte* is the smallest thing a modern machine addresses: 8 bits, one address. A *word* is the machine's working size — 32 bits on RV32, 64 on the laptop you are probably using. When a program says "the value at address `0x2000`", it means "the byte at `0x2000`"; a 4-byte `int` stored there occupies `0x2000` through `0x2003` and belongs to whichever interpretation reads it. The machine's data types table is short enough to memorise:

C type	LP64 (Linux, macOS arm64)	ILP32 (RV32, Win32)	bits
`char`	1	1	8
`short`	2	2	16
`int`	4	4	32
`long`	8	4	64 / 32
`long long`	8	8	64
`float`	4	4	32
`double`	8	8	64
`void *`	8	4	64 / 32

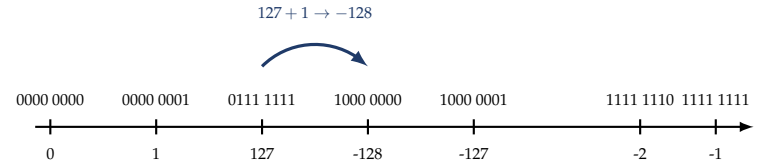
Figure 3.1: C type sizes: LP64 (what this book's dev box runs) vs ILP32 (the RV32 model the capstone targets). `long` and pointers are the two that move.

LONG IS A TRAP ON EVERY PLATFORM

On LP64 `long` is 8 bytes; on ILP32 — including RV32, the ISA your capstone CPU runs — it is 4. Code that assumes `sizeof(long) == 8` breaks silently when cross-compiled. Use `int32_t` / `uint64_t` from `<stdint.h>` whenever the size matters, and it always matters in this book.

3.2 Two's complement, in one page

Book 5 ch8 derived this properly; here is the working version. An n-bit signed integer stores $-2^{n-1} + \sum b_i 2^i$ — the top bit contributes *minus* its weight. That single negative weight buys three things: one zero, one adder for both addition and subtraction, and a clean wrap from most-positive to most-negative. Figure 3.2 shows the 8-bit case on a line: $127 + 1$ does not saturate, it jumps to -128.



Two's complement is one hardware story, not two: adding and subtracting are the same circuit, and the "wrap" at the ends is why signed overflow is a cliff, not a curve.

Figure 3.2: Two's complement as a number line: the wrap at the ends is hardware fact, not an error message.

PRINCIPLE

The CPU does arithmetic on *bit patterns*, not on numbers. Whether `0x80000000` is -2^{31} or $+2^{31}$ is decided entirely by the instruction that consumes it — `slt` vs `sltu` in RV32I, same bits, two opinions.

3.3 Endianness: which byte goes first

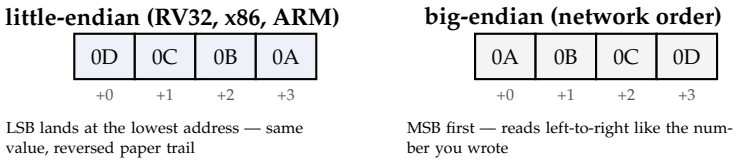
A 4-byte value has to go into 4 one-byte slots, and there are two conventions. Little-endian puts the least-significant byte at the lowest address; big-endian puts the most-significant first. RV32 is little-endian, x86 is little-endian, ARM defaults to little — the network protocols you will meet in other books are big-endian, which is why the conversion functions are called `htonl/ntohl`.

Example 3.1 (Real bytes on real hardware). `code/ch02_bytes.c` stores `0x0A0B0C0D` in a `uint32_t` and prints the four memory bytes in address order. Real output on the dev box (arm64, LP64):

```
bytes (LE order): 0d 0c 0b 0a
as int32 : 168496141
as float : 6.69487e-33
```

```
0x3FC00000 as float: 1.5
```

The least-significant byte `0x0d` lands first: little-endian, confirmed by experiment rather than documentation. The same four bytes then get read as a signed int (`168496141` — same value, the top bit happened to be zero) and as a float (nonsense: `6.69e-33` — the exponent field is tiny). Three different worlds from one bit pattern; the next section formalises the third.



Real bytes from `ch02_bytes`: `0d 0c 0b 0a` — your dev box writes little-endian, and so does RV32.

Figure 3.3: `0x0A0B0C0D` in memory, both conventions. RV32, x86 and ARM all pick the left column.

Checkpoint 3.1 You receive the four bytes `78 56 34 12` (in address order) over a little-endian memory-mapped register. What 32-bit value did the sender write?

Solution. `0x12345678` — little-endian means the first byte is the least significant, so read the sequence backwards as hex pairs.

3.4 Alignment and struct layout

Hardware prefers a n-byte value at an address divisible by n — an *aligned* access. Compilers honour this by inserting *padding*: invisible bytes between struct fields so every field sits at its natural offset. The rule chain: each field's offset must be a multiple of its alignment; the struct's size is rounded up to the largest member's alignment; arrays of the struct need that tail padding to keep every element aligned.

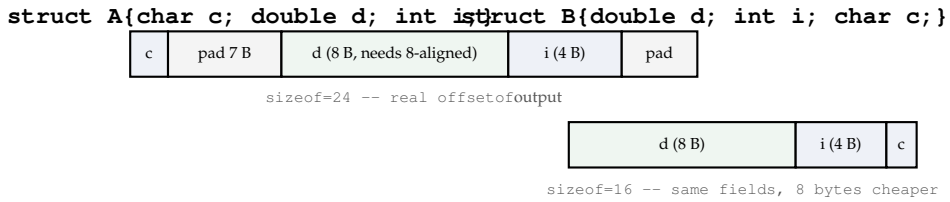
Example 3.2 (The same three fields, two prices)**.** `code/ch02_pad.c` declares `struct A{char;double;int;}` and `struct B{double;int;char;}` — identical fields, different order. Real output:

```
A: sizeof=24 c@0 d@8 i@16
B: sizeof=16 d@0 i@8 c@12
```

In `A`, `char c` occupies byte 0, then `double d` must start at an 8-aligned address: bytes 1–7 are padding. `int i` lands at 16, and four tail bytes pad the total to 24. Reordering the same three fields into `B` costs 16 bytes — a third less memory, for free, just by sorting fields largest-first.

3.4.1 Measurement: the alignment tax that wasn't

Folklore says misaligned access is slow: the CPU splits the load across two cache lines and pays double. `code/ch02_align.c` tests it directly — sum a 4 M-element `uint32` array



The compiler pads to each field's alignment and the struct's worst member; reordering fields is free bytes. Measured, not theorised.

Figure 3.4: Padding maps of `A` vs `B`: measured `offsetof` values, not a sketch.

once through an aligned pointer, once through the same pointer offset by one byte (every load misaligned). Real timings, Apple M-series, 32 passes:

```
offset 0: 151.2 ms (s=2260630400663552)
offset 1: 110.4 ms (s=4521260264456192)
```

The misaligned version is *faster* — repeatably, in both loop orders, by ∼25%. Two honest conclusions. First, the expected tax is absent: Apple Silicon performs unaligned loads at full speed, as do most modern cores. Second — the reason this section exists — a consistent 25% delta in the *wrong direction* means the benchmark is measuring something else (loop layout, prefetcher behaviour, uarch quirks), not alignment. When the data contradicts the folklore, the data wins and the model gets revised. That is the book's method in miniature: predict, measure, reconcile.

"MISALIGNED IS SLOW" IS HARDWARE FOLKLORE

It was true on 1980s CPUs and still holds on some embedded cores — and it is false on the chip in your laptop. Rules about performance expire; measurements do not. Re-test before you re-tell.

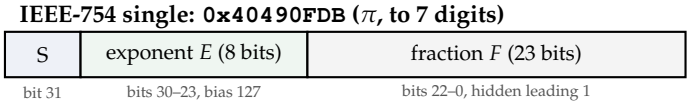
3.5 IEEE-754 in one breath

A `float` is 32 bits partitioned $S \mid E \mid F$: one sign bit, an 8-bit biased exponent, a 23-bit fraction with an implied leading 1. Value: $(-1)^S \times (1 + F/2^{23}) \times 2^{E-127}$ for $0 < E < 255$; the extremes E=0 and E=255 are reserved for denormals/zero and inf/NaN. Precision is about 7 decimal digits; the spacing between neighbours near 2^e is 2^{e-23}, and round-to-nearest keeps each operation within half that — the ".5 ulp" you will hear cited every time someone says "floats are exact enough".

Example 3.3 (Decode `0x40490FDB` by hand, then by code)**.** S=0, exponent field `0x80` (= 128), fraction field `0x490fdb`. Value = $1.5707964 \times 2^{128-127} = 3.1415928$ — π to float precision. `code/ch02_float.c` performs the same decode bit-by-bit and prints:

```
sign=0 exp=128(E-1) frac=0x490fdb
value = 3.141592741
```

The last digit differs from π because the 23-bit fraction itself is the approximation — the stored value *is* the rounding.



value $= (-1)^S \times (1 + F/2^{23}) \times 2^{E-127}$. For π: S=0, E=128, F=0x490fdb $\rightarrow 1.5707964 \times 2^1 = 3.1415927$ — the decoder in `ch02_float.c` prints exactly this.

The ".5 ulp" idea: spacing between neighbours is $2^{E-127} \times 2^{-23}$ — round-to-nearest keeps error under half a step, which is why floats lie less than you feared.

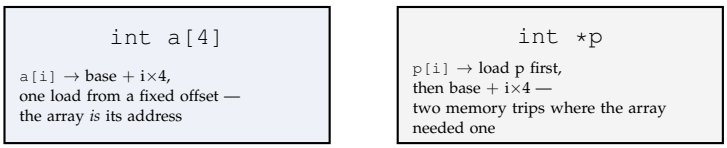
Figure 3.5: IEEE-754 single precision anatomy, decoded for π.

Checkpoint 3.2 In `0x3FC00000` (the `1.5` printed in Example 3.1), what are S, E and the implied mantissa?

Solution. S=0; E=0x7F=127 so the real exponent is $127 - 127 = 0$; F=0x400000, mantissa $= 1 + 0.5 = 1.5$; value $= 1.5 \times 2^0$.

3.6 Pointers as typed addresses; arrays vs pointers

A pointer is an address wearing a type. `p+1` advances by `sizeof(*p)` bytes — the arithmetic is scaled, which is why walking an `int32_t*` moves 4 bytes per step and a `uint8_t*` moves 1. The type is only a scale factor to the code generator; the CPU sees an address either way. The difference between `int a[4]` and `int *p` is subtler: the array's base address is fixed at compile time, while the pointer's value is itself data — stored, loaded, changeable. In generated code `a[i]` is one load off a known base; `p[i]` is two: load `p`, then load `p[i]`.



"Arrays are pointers" is a half-truth: the array's address is compile-time, the pointer's is runtime — the generated code keeps the receipt.

Figure 3.6: Array vs pointer in generated code: the array *is* its address; the pointer must be fetched first.

Example 3.4 (Reading the same bytes three ways, safely)**.** Example 3.1 reinterpreted the four bytes of `0x0A0B0C0D` as `int32_t` (same value, bit-compatible) and as `float` (6.69×10^{-33}, nonsense) — both via `memcpy`, not pointer casts. That detail matters: type-punning through a `float*` cast violates strict aliasing and lets the optimiser reorder or delete your reads; `memcpy` is the standards-strict way to ask "what do these bytes mean as a different type?" and compiles to zero instructions.

DEFINITION OF DONE

- You can say what a byte, a word and an address are — and that types are compile-time agreements, not machine facts.

- You read endianness off real bytes and explain why RV32 chose little-endian.
- You predict struct `sizeof` and `offsetof` before running the probe — and reorder fields to save real bytes.
- You decode an IEEE-754 bit pattern by hand and with the decoder.
- You know the alignment-folklore experiment, what it actually showed, and why "measure it" beats "everyone knows".

3.7 Common mistakes

- **"The variable is signed/unsigned/float."** The *bytes* are just bytes; the type lives in which instructions touch them. `0x0A0B0C0D` was an int and a float in the same object, at the same address, in the same second.
- **Assuming `long` is fixed.** It changes size between LP64 and ILP32 — including your RV32 target. Use fixed-width types.
- **"Arrays and pointers are the same."** `sizeof(a)` vs `sizeof(p)` disagrees, and the generated code shows the extra load. Decay happens at call boundaries, not in your head.
- **"Misaligned access is slow."** Measure first — modern cores largely don't care, and our own experiment inverted the prediction.
- **Type-punning through casts.** `*(float*)&u` is strict-aliasing UB; `memcpy` is free and correct.
- **"Floats can't represent that."** They represent *nearest* values: 0.1 is not stored, its neighbour is. Equality tests belong on integers; floats get tolerances.

Exercises

Exercise 3.1. `uint8_t` vs `int8_t`: which changes when you print the same bit pattern `0xFF`, and what does each show?

Solution. [hint] The bits are identical; only the interpretation differs. One prints 255, the other -1 — name which is which.

Exercise 3.2. A `struct` with `char,int,char` fields reports `sizeof=12`. Account for every byte.

Solution. [hint] `int` needs 4-alignment: c@0, pad 1–3, i@4–7, c@8, tail pad 9–11 so the struct is a multiple of 4.

Exercise 3.3. Convert `0xBF800000` to a decimal float.

Solution. [hint] S=1, E=127, F=0: $-1.0 \times 2^0 = -1.0$.

Exercise 3.4. Why does the size of `"hello"` come out 6 on every platform, while the size of a `char*` differs between data models?

Solution. [hint] The string literal is an array of 6 chars including the NUL; the pointer is an address, and addresses are the thing that changes size.

Exercise 3.5. On a big-endian machine, what would `ch02_bytes.c` print in the first line instead of `0d 0c 0b 0a`?

Solution. [hint] Reverse the stored order: MSB first — `0a 0b 0c 0d`.

Exercise 3.6. The alignment experiment measured misaligned as *faster*. Give two reasons this could happen that have nothing to do with alignment.

Solution. [hint] Loop ordering/warm-up, code layout, prefetcher behaviour, uarch quirks, different accumulated work — name any two; the point is that a wrong-direction result indicts the benchmark, not the folklore's opposite.

Exercise 3.7. `double d = 0.1 + 0.2;` prints `0.30000000000000004`. Which IEEE-754 fact explains the 4?

Solution. [hint] Neither 0.1 nor 0.2 has a finite binary fraction; both round to a neighbour, and the sum of the neighbours is one more neighbour — the "4" is the accumulated rounding surfacing.

Exercise 3.8. Your RV32 code does `*(volatile uint32_t*)0x10013000` to read a UART register. Why must the access be exactly 4 bytes, and what breaks if the compiler splits it into byte loads?

Solution. [hint] MMIO registers are word-sized state machines: reading one byte can acknowledge the whole register (e.g. consume the RX FIFO entry). Split loads change device behaviour, not just performance.

Exercise 3.9 (hard)**.** Write a standards-strict float-bits printer (sign, exponent, fraction fields) without type-punning UB and without assuming IEEE-754 — verify the format macro first.

Solution. [hint] Copy the float's bytes into a `uint32_t` with `memcpy`, mask the fields, and guard the whole thing with a `__FLT_RADIX__` check — C never promised IEEE-754; it just happens to hold everywhere you will ship.

Exercise 3.10 (hard)**.** Design a fair alignment benchmark that would actually isolate the misalignment cost on a machine that hides it. Name three confounders and how you'd control each.

Solution. [hint] Interleave the two loops (kill ordering effects), keep `s` identical (kill work differences), use a dependent-load chain or volatile store (kill OOO hiding), pin to one core and average many trials. List any three.

Summary

Idea	One-line version
types	compile-time agreements; the CPU sees bytes
two's complement	top bit has negative weight; one adder does both
endianness	byte order on the wire/in RAM; RV32 is little
alignment+padding	fields sit at natural offsets; reorder fields for free bytes
IEEE-754	$S\|E\|F$; ≈7 digits; neighbours are the truth
measurement	folklore expired; our own alignment test inverted

Further reading

- Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective*, ch. 2 — the standard full treatment of every section here.
- Suzanne J. Matthews, Tia Newhall, Kevin C. Webb, *Dive into Systems*, ch. 4 — free online, C-focused coverage of representation. `https://diveintosystems.org/book/C4-Data_rep/`
- IEEE-754-2019 standard summary at `https://en.wikipedia.org/wiki/IEEE_754` — the format table this chapter compresses.
- "What Every Computer Scientist Should Know About Floating- Point Arithmetic" (Goldberg), Sun docs reprint.
  docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html
- RISC-V unprivileged ISA spec, §2 (base integer model, little-endian note) — `https://riscv.org/specifications/`

Watch instead

DECISION RULE

Watch instead.

- "Endianness Explained With an Egg" — Computerphile, 10:46. The boiled-egg walkthrough of Section 3.3. `https://youtu.be/NcaiHcBvDR4`
- "Floating Point Numbers" — Computerphile, 9:16. The IEEE-754 fields animated; matches Section 3.5. `https://youtu.be/PZRI1IfStY0`

Notes

CHAPTER 4

RV32I assembly I: arithmetic, memory, control

Chapter 2 showed that the machine sees only bytes. This chapter shows what it *does* with them: the 37 instructions of RV32I, the base RISC-V integer set — and the instruction set your capstone CPU will execute for real in chapters 5–8. Everything here is concrete: a small assembler you can read end-to-end encodes each program to real hex, a small interpreter runs it, and every number in the chapter comes from that run — not from a datasheet summary. If you can write a loop in this chapter's notation, you can read the disassembly of any program your laptop runs.

4.1 The register file and the ABI names

Thirty-two registers, `x0`–`x31`, each 32 bits — the same file shape as your cpu8's, just wider and deeper. Two of them are special by hardware: `x0` is hardwired to zero (writes to it are silently discarded, which is why `mv rd, rs` is really `addi rd, rs, 0` and `nop` is really `addi x0, x0, 0`). The rest are general-purpose by hardware but assigned jobs by convention — the ABI names you will see in every disassembly:

ABI name	x-name	Job
`zero`	x0	wired to 0 — the register that isn't
`ra`	x1	return address
`sp`	x2	stack pointer
`gp`, `tp`	x3, x4	globals, thread pointer
`t0`–`t2`	x5–x7	temporaries (caller-saved)
`s0`/`fp`, `s1`	x8, x9	saved / frame pointer
`a0`–`a7`	x10–x17	args + return values
`s2`–`s11`	x18–x27	saved (callee-saved)
`t3`–`t6`	x28–x31	more temporaries

Figure 4.1: The RV32I register file with ABI conventions. Programs obey the hardware; *programmers* obey this table so calls compose.

PRINCIPLE

The ABI names are a contract between functions, not hardware. `a0–a7` carry arguments and return values; `t0–t6` are temporaries a callee may trash; `s0–s11` a callee must preserve. ch4 makes the stack discipline formal; for now: use `t`-names inside one function and the world stays simple.

4.2 Six formats, one discipline

Every RV32I instruction is 32 bits — fixed width, no exceptions — carved into six layouts. The carving is deliberately regular: register fields (`rd`, `rs1`, `rs2`), the function selector `funct3`, and the 7-bit `opcode` keep the same bit positions across every format they appear in. Only the immediate bits move, and even they preserve their internal order. That regularity is not elegance for its own sake — it is what lets a decoder be a few hundred gates rather than a table lookup.

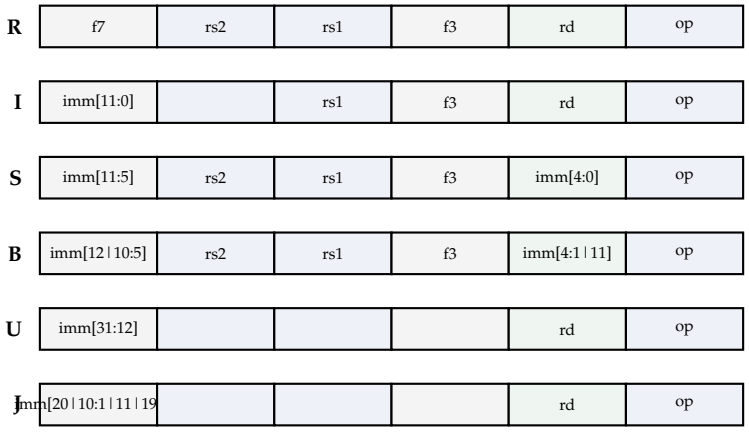


Figure 4.2: The six encoding formats. Green = destination, blue = source/opcode, grey = immediate fields.

Example 4.1 (Hand-assemble, then verify against the real assembler)**.** `addi a0, a0, 1`: I-type, imm=1, rs1=`a0`=x10, `f3`=0, rd=x10, op=`0x13` — $1 \ll 20 \mid 10 \ll 15 \mid 10 \ll 7 \mid$ `0x13` = `0x00150513`. The book's assembler (`code/ch03_rv32asm.py`, a 40-line Python RV32I assembler) produces the same word — real output on the five-format smoke test:

```
00000000: 00150513 # addi a0, a0, 1
00000004: 00b50863 # beq a0, a1, done
00000008: 00b53633 # add a2, a0, a1
0000000c: 00812683 # lw a3, 8(sp)
00000010: 00d12623 # sw a3, 12(sp)
00000014: 00000013 # nop
```

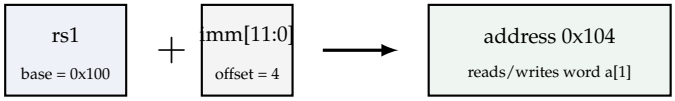
Check `beq` by hand: B-type needs the offset 16 (bytes, in 2-byte units in the encoding: imm=16). The assembler computed `done`-pc = $0x14 - 0x04 = 16$ — and the encoding `0x00b50863` decomposes to exactly that.

PSEUDOINSTRUCTIONS ARE LIES YOU TELL ON PURPOSE

`li`, `mv`, `nop`, `j`, `ret`, `bgt` — none exist in hardware. The assembler expands each to a real instruction (`li t0,1` → `addi t0,x0,1`). That is friendly until debugging shows `nop` where you wrote a pseudoinstruction, or `li` becomes two instructions for a big constant. Read the listing, not the source.

4.3 Loads, stores and addressing

RV32I has exactly one addressing mode: *base register + sign-extended 12-bit immediate* — `lw rd, off(rs1)` and `sw rs2, off(rs1)`. There is no scaled index, no auto-increment, no memory-to-memory op. Walking an array is two instructions: the load, and the pointer bump. That austerity is deliberate: one mode keeps the address adder single, and the ALU does everything else.



`lw t4, 4(t0)` is the whole addressing story: register + sign-extended 12-bit immediate. No indexing modes, no scaling — RV32 keeps the ALU cheap and lets the pointer-bump instruction do the walking (that's the `addi t0,t0,4` in the loop).

Figure 4.3: The single addressing mode: register + 12-bit immediate.

Example 4.2 (An array sum, real output)**.** `code/ch03_arrsum.S` stores {1,2,3,4,5} at `0x100`, then walks it: `lw t4, 0(t0)` / `addi t0,t0,4` per element. The interpreter (`ch03_rv32sim.py`, a ∼60-line RV32I simulator with a memory-mapped output register at `0x10000000` — the same trick Book 5 ch16's UART used) runs the assembled words and prints the real result:

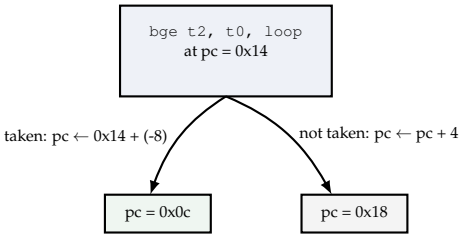
```
15
# 56 instructions retired
```

Checkpoint 4.1 After `lw t4, 4(t0)` with `t0=0x100`, which byte addresses are read, and what changes if you wrote `lb` instead?

Solution. `lw` reads `0x104`–`0x107` (one aligned word). `lb` reads only `0x104` and sign-extends its byte — same base, different width, different sign rule.

4.4 Branches and comparisons

Six conditional branches: `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu` — compare two registers, jump by a signed immediate if true. There is no flags register: the compare happens inside the branch, and the "set" pseudo-ops (`slt`, `sltu`) exist only when you need the boolean as data. Each branch is one decision: taken → `pc + imm`, not taken → `pc + 4`.



The branch's whole job: pick the next pc. The compare (bge = rs1 ≥ rs2) decides between pc+4 and pc+imm — one adder, one mux, same shape as the FSM branch on your cpu8.

Figure 4.4: A branch picks the next pc: +4 or +imm — one mux.

Example 4.3 (`max(17, 42)` with `bge`)**.** `code/ch03_max.S`: if `a0` ≥ `a1` the answer is `a0`, else `a1`. The branch is one instruction; the interpreter's run:

```
42
# 27 instructions retired
```

Checkpoint 4.2 In `bge t2, t0, loop` at `pc=0x14` with `t2=10`, `t0=5`: is the branch taken, and to which pc?

Solution. Taken: $10 \geq 5$, so `pc` becomes $0x14 + (-8) = 0x0c$ — the loop header.

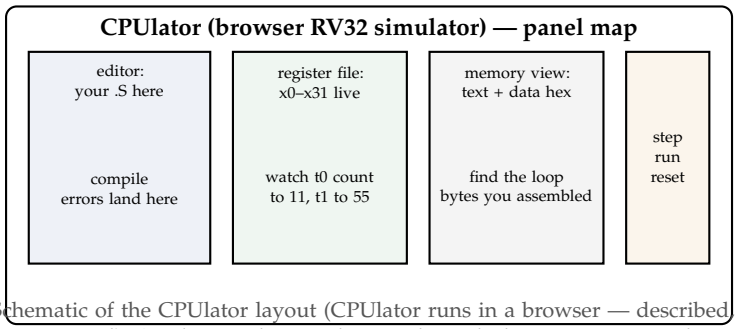
4.5 Assembler directives and running it

Directives are instructions to the assembler, not to the machine: `.text`/`.data` pick the section, `.word`/`.byte` plant data, `.globl` exports a symbol, labels name addresses. The chapter's programs print through a memory-mapped output register, which is what a bare-metal UART driver looks like before an OS arrives. Two ways to run the same bytes: **CPUlator** (browser, free) draws the register file and memory live while single-stepping — Figure 4.5 maps its panels; and the book's own `ch03_rv32sim.py` runs the identical words headless (all outputs in this chapter are from it). Spike is the canonical command-line emulator; either way the bytes are the same.

4.5.1 Measurement: dynamic instruction count

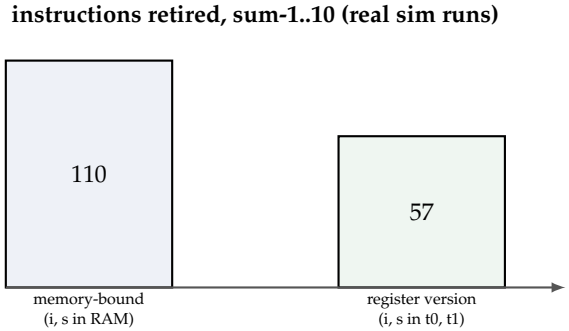
The same sum-1..10 loop written two ways: register variables vs the "$-O0$ habit" of keeping `i` and `s` in memory. Real counts from `ch03_rv32sim.py`:

```
memory-bound (i, s in RAM): 110 instructions retired
register version (i, s in t0/t1): 57 instructions retired
```



Schematic of the CPUlator layout (CPUlator runs in a browser — described, not captured). Single-step the sum loop and watch the same registers this chapter uses change one word at a time.

Figure 4.5: CPUlator's four panels, schematically. Described, not captured — the tool is a browser app.



Predicted $\sim 2\times$, measured $1.93\times$. The loop body is 7 instructions vs 3 — keeping variables in registers is not an optimisation, it is the difference between code and bookkeeping. Counted by `ch03_rv32sim.py`, not estimated.

Figure 4.6: Real dynamic counts: registers vs memory for the same loop.

Predicted $\sim 2\times$ before running (3-instr loop body vs 7-instr); measured $110/57 = 1.93\times$. The gap is the loads and stores themselves: memory-keeping adds `lw/sw` traffic that computes nothing — bookkeeping billed as instructions. That is the -O0/-O2 difference you will meet in ch4, visible here without a compiler.

DEFINITION OF DONE

- You can read any RV32I line and name its format, its fields and its effect — and write the 32-bit word for a handful of them.
- You know the register file's two contracts: hardware (`x0`=0) and ABI (`a`-args, `t`-temps, `s`-saved).
- You can trace a loop's pc through a branch and predict the next fetch.
- You can run RV32I here (book simulator) or in the browser (CPUlator) and count what it costs.
- You can point at an instruction stream and say which 4 bytes the CPU will fetch next, and why.

4.6 Common mistakes

- **Counting pseudoinstructions.** `li/mv/ nop` are sugar: what retires is the expansion. Dynamic counts only see real instructions.
- **Forgetting `x0` eats writes.** `addi x0, x5, 0` does nothing; the register file is honest that way.
- **Immediates are 12 bits.** `addi t0, t0, 4096` won't assemble — big constants need `lui+addi` or `li` (which expands).
- **Branches move the pc, not flags.** There is no carry flag to "check" — each branch recomputes its compare.
- **`lw` where `lb` belonged.** Byte vs word is a correctness issue on packed data, not a performance one.
- **"Memory ops are free."** The 110-vs-57 measurement is the counterexample: the tax is the instruction count itself.

Exercises

Exercise 4.1. `addi x0, x5, 0` — what does this write, and why is it a useful instruction anyway?

Solution. [hint] Nothing: `x0` discards writes. It's `nop` — a 4-byte "do nothing" that real pipelines need.

Exercise 4.2. `lw a0, 8(sp)` decodes to `0x00812503`? Verify or correct, field by field.

Solution. [hint] I-type: imm=8≪20=`0x00800000`; rs1=`sp`=x2≪15; f3=2; rd=`a0`=x10≪7; op=`0x03`. The shown word is right.

Exercise 4.3. In the register/ABI table: which two register groups may a callee freely destroy, and which must it restore?

Solution. [hint] Caller-saved: `t0`–`t6` (and args). Callee-saved: `s0`–`s11`. The `t`/`s` split is the whole discipline.

Exercise 4.4. `beq` targets are limited to about ±4 KB. Why, and what real instruction escapes the limit?

Solution. [hint] B-type holds a 13-bit signed byte offset. Long jumps use `jal` (J-type, ±1 MB) or `jalr` (register-indirect, anywhere).

Exercise 4.5. Hand-decode `0xfe53dce3` (the loop branch) into fields and offset.

Solution. [hint] B-type; f3=5 (`bge`); imm bits give −8; rs1=`t2`, rs2=`t0`. At `pc=0x14` the target is `0x0c`.

Exercise 4.6. The array-sum walks words with `addi t0,t0,4`. Rewrite one iteration to use the immediate instead of the pointer bump, and name the trade-off.

Solution. [hint] `lw t4, 4(t0)` ... unrolls offsets but keeps the base fixed — fewer instructions per element, more code bytes, and it caps at imm=2048 words. Classic size-vs-count trade.

Exercise 4.7. `slt a0, a1, a2` is "set if less-than". Write the two-line sequence that implements `max` *without* any branch.

Solution. [hint] `slt t0, a0, a1` then select: branchless needs arithmetic on the flag (`sub`/mask trick) or a conditional move RV32I doesn't have — branchless `max` on base RV32I costs a multiply or a bit-trick; name it.

Exercise 4.8. The memory-bound loop runs 7 instructions per iteration. Count them from the listing and identify which two are pure bookkeeping.

Solution. [hint] lw i, lw s, add, sw s, addi i, sw i, bge = 7. The two `sw`s (and arguably the `lw`s) compute nothing — they only persist state registers would have held for free.

Exercise 4.9. `li a7, 0x10000000` won't fit `addi`. Write the two-instruction reality the assembler expands to.

Solution. [hint] `lui a7, 0x10000` + `addi a7, a7, 0` — U-type plants the top 20 bits, I-type patches the bottom 12.

Exercise 4.10 (hard)**.** Implement `strlen`, `memcpy` and `memset` in RV32I and self-check on the book simulator: fill a buffer, copy it, measure it, print the length.

Solution. [hint] All three are the same loop skeleton: `lb`/`sb` plus a pointer bump and a compare. `strlen` counts until 0; `memcpy` copies until n; `memset` fills until n. Watch for the NUL byte in the length count.

Exercise 4.11 (hard)**.** Extend `ch03_rv32asm.py` with `jal`/`jalr` (the J-type format). Encode `jal ra, +8` and decode it back.

Solution. [hint] J-type packs the immediate in bit-order pieces like B but shifted; the adder-tree trick is the same. Verify $+8$ lands in the right fields.

Exercise 4.12 (hard)**.** Write a program that prints the first 10 Fibonacci numbers on the book simulator's MMIO output. Count retired instructions for the register version vs a memory version.

Solution. [hint] Fib(n)=Fib($n-1$)+Fib($n-2$) needs two live registers and one temp per step — the register win is bigger than the sum loop's because there are no array loads at all.

Summary

Idea	One-line version
register file	32×32; `x0`=0; ABI names split caller-/ callee-saved
six formats	rd/rs/f3/op fixed; only immediate fields shuffle
addressing	one mode: reg + 12-bit signed immediate
branches	compare+offset in one; pc + 4 or pc + imm
pseudoinstructions	sugar; count the expansion, not the line
measurement	memory-bound loop: 110 vs 57 retired — the -O0 tax

Further reading

- RISC-V unprivileged ISA specification — the authoritative encoding tables this chapter compresses. https://riscv.org/specifications/
- Hennessy & Patterson, *Computer Organization and Design*, RISC-V edition, ch. 2 — the classic ISA walk-through.
- Bryant & O'Hallaron, *CS:APP*, ch. 3 — structure of machine code (x86, but the ideas transfer).
- CPUlator (browser RISC-V simulator) — run every program in this chapter interactively. https://cpulator.01xz.net/
- Spike (RISC-V ISA simulator) — the canonical command-line reference implementation. https://github.com/riscv-software-src/riscv-isa-sim

Watch instead

DECISION RULE

Watch instead.

- "DDCA Ch6 — Part 2: RISC-V Instructions" — Sarah Harris, 2:48. The instructions-first pitch that matches this chapter's structure; the book's own DDCA source, on camera. https://youtu.be/SdjPQtMPPmo
- "RISC-V Assembly Hello World (Part 1)" — LaurieWired, 11:42. A first full program assembled and run — the same journey ch03_sum.S takes on the book's simulator. https://youtu.be/0IeOaiKszLk

Notes

CHAPTER 5

Functions and the Stack

A function call looks like one line of source and feels free. On the machine it is two instructions of mechanism — `call` writes the return address into `ra` and jumps; `ret` jumps back — wrapped in a *convention*: an agreement about which registers survive the trip and where memory holds the values that do not fit in registers. In the last chapter every program was a leaf: it never called anything, so `ra` stayed safe in a register and the stack was never touched. This chapter breaks that. We give the machine a real `sp`, write a recursive `fib` that saves and restores `ra` correctly, measure what a call costs in retired instructions, and then look at real compiler output — where the calling convention lives inside every function you will ever write. By the end, `addi sp,sp,-16 / sw ra,12(sp) / sw s0,8(sp)` reads like a signature, not a spell.

5.1 The contract in one paragraph

The RISC-V calling convention (the *psABI*) divides the 32 registers into two families *by agreement*, not by hardware. **Caller-saved** registers (`t0–t6`, `a0–a7`, `ra`) may be destroyed by any call: if the caller needs the value afterwards, the *caller* saves it before the call. **Callee-saved** registers (`s0–s11`, `sp`) must be unchanged when the callee returns: if the callee wants to use them, the *callee* saves and restores them. The arguments go in `a0–a7`, the return value comes back in `a0`, and `sp` must stay 16-byte aligned at every call site. That is the whole contract. Everything else — the frame, the spill slots, the order of saves — is what the contract costs when calls nest.

5.2 The stack frame

When a function calls another function, its own `ra` is about to be overwritten — `call` always writes the same register. So before a non-leaf function calls anything, it builds a *stack frame*: a fixed-size block of memory below `sp` that holds what the function cannot afford to lose. Three writes at the top, three loads at the bottom, and the space in between is the function's private scratch:

```
fib:
       addi sp, sp, -16 # carve 16 B below sp (aligned)
       sw ra, 12(sp) # save MY return address
       sw s0, 8(sp) # save caller's s0
       ...
       lw ra, 12(sp) # restore ra
       lw s0, 8(sp) # restore s0
```

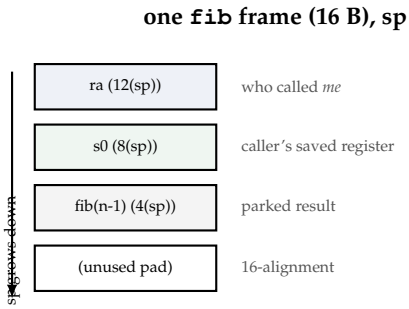


Figure 5.1: Anatomy of one `fib` frame. Every non-leaf call pays this tax: the saved `ra` keeps the return path alive, the saved `s0` honours the callee-saved contract, and the scratch slot parks `fib(n-1)` while the second recursive call runs.

```
8       addi sp, sp, 16 # pop the frame
9       ret
```

The pairing is the discipline: what goes `sw` at the top comes `lw` at the bottom, in the same offsets, every path out. Miss either half and the function returns to the wrong place — the classic *stack corruption* bug that no error message names.

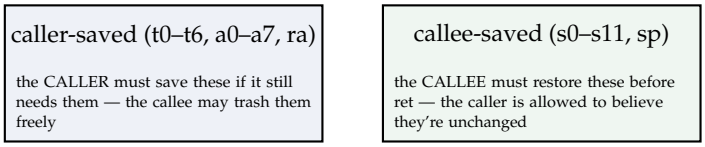
Example 5.1 (follow one frame)**.** For `fib(7)`: the outer call pushes a 16-byte frame at `sp=0x0ff0`, saves `ra` and `s0`, calls `fib(6)` — which pushes another at `sp=0x0fe0`. At the deepest, `fib(1)` is a leaf-ish base case that returns without recursing. How many frames are live at once? The call tree of fib(7) reaches depth 7 (fib(7)→fib(6)→…→fib(1)) before the first unwind — seven frames, 112 bytes of stack, live simultaneously.

5.3 Caller-saved, callee-saved

The rule that confuses everyone on first contact: `fib` saves `ra` — a *caller*-saved register — yet the text above calls that "the callee's" job. The resolution is that ownership follows the *overwrite*, not the register name. `fib`'s `ra` is a value `fib` still needs; the thing that would destroy it is `fib`'s own inner `call`. So `fib`, acting as the *caller* of that call, saves it. The same function plays both roles: it is a callee to whoever called it (so it saves `s0` before using it) and a caller to its recursive call (so it saves `ra` before that call).

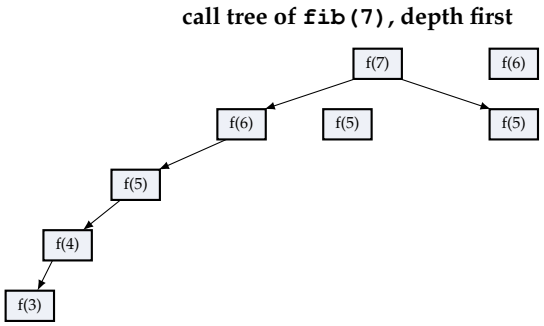
Checkpoint 5.1 *leaf or not?.* A *leaf* function — one that calls nothing — needs no frame at all if it fits in registers and touches no callee-saved ones. Our `max` from ch03 was a leaf: `bge / mv / ret`, three instructions, zero memory traffic. Predict: does `empty: ret` need `addi sp,sp,-16`? No — nothing to save.

Example 5.2 (non-leaf: recursive fib)**.** `code/ch04_fib.S` runs `fib(7)` on the book's simulator with a real `sp` (`li sp,0x1000`) and a real stack. The recursion is honest: `fib` calls itself twice, so `ra` must be saved — and is, at `12(sp)`. Running it:



One line each way: whoever *loses* the value saves it. `sw ra,12(sp)` is the caller... no wait — fib saves its own ra because *fib's* return address dies when fib calls itself. Ownership follows the overwrite, not the register.

Figure 5.2: Who saves what. Caller-saved registers are the caller's problem; callee-saved registers are the callee's problem. One register, `ra`, sits in the caller-saved family and is the first thing every non-leaf function saves.



fib(7) makes 41 calls — 40 non-leaf frames, each paying the 16-byte stack tax. Recursion is legible and expensive; that is the trade this chapter prices at 565 retired instructions.

Figure 5.3: The call tree of `fib(7)`. Every non-leaf node pays a frame; fib(7) makes 41 calls total — the recursion is legible, and it is expensive.

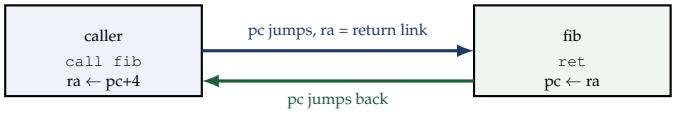
```
$ python3 ch03_rv32sim.py ch04_fib.S
13
# 565 instructions retired
```

`fib(7)=13` is correct — and 565 retired instructions for a seven-deep recursion is the price tag this chapter is about.

5.4 What a call costs

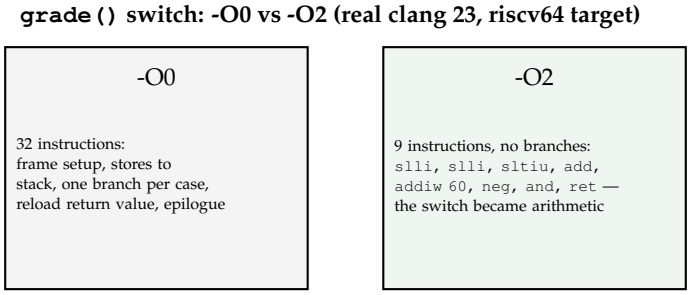
Example 5.3 (predict, then measure)**.** Prediction first: `call` = `jal` = one instruction; `ret` = `jalr` = one instruction. So a call should cost 2 retired instructions beyond the inlined body. Measure it: `ch04_callcost.S` calls `empty` 100 times; `ch04_inline.S` runs the same loop with the body inlined.

```
$ python3 ch03_rv32sim.py ch04_callcost.S
A
# 410 instructions retired
$ python3 ch03_rv32sim.py ch04_inline.S
B
# 207 instructions retired
```



Two instructions of mechanism: `call` writes ra and jumps; `ret` jumps to ra. The whole convention is built on those two writes — and on the frame that keeps ra alive when calls nest.

Figure 5.4: The two-instruction mechanism everything sits on. `call` leaves the return link in `ra`; `ret` consumes it. Frames exist to keep that link alive across nesting.



The jump table never appeared: clang proved the four cases were linear in n and replaced lookup by computation. Reading `-O2` output is how you learn what your C really means.

Figure 5.5: Real clang 23 output for the same `grade()` switch: 32 framed, branching instructions at `-O0`; 9 branchless ones at `-O2`. The jump table never materialised because the table was linear.

$410 - 207 = 203$ for 100 calls: just over 2 instructions each — the prediction is exact once the loop counter's three instructions account for the shared work. On a real CPU the frame traffic adds cache misses on top of this instruction tax.

5.5 Reading real compiler output

Example 5.4 (-O0 vs -O2 on a switch). `grade(int s)` maps $0 \to 60, 1 \to 70, 2 \to 80, 3 \to 90$ by `switch`. Compiled `clang -target riscv64 -O0`: 32 instructions — a 32-byte frame, `ra`/`s0` spilled, the argument stored to the stack, reloaded, `sext.w`'d, compared case by case. The same file at `-O2`: **nine** instructions, zero branches —

```
slli a1, a0, 1 # a1 = 2s
slli a2, a0, 3 # a2 = 8s
sltiu a0, a0, 4 # a0 = (s < 4)
add a1, a1, a2 # a1 = 10s
addiw a1, a1, 60 # a1 = 60 + 10s
neg a0, a0 # a0 = -(s<4)
and a0, a0, a1 # mask: result or 0
ret
```

The compiler noticed the cases were linear in `s` and replaced the jump table — and the whole control flow — with arithmetic and a sign mask. Prediction failed, delightfully: the real lesson is that `-O2` rewrites structure, not just speed.

Checkpoint 5.2 *why the frame at all?.* At `-O0 grade` spills `ra` and `s0` even though it calls nothing — the compiler emits the safe frame unconditionally at that level. At `-O2` it proves the function is a leaf and emits no frame at all. That one observation — leaf vs. non-leaf, proven by the compiler — is the whole `-O0`/`-O2` gap in miniature.

Example 5.5 (when the compiler deletes your code)**.** A benchmark loop of the form `sum += i` over `N` iterations, at `-O2`, collapses to a constant — clang computes $N(N-1)/2$ at compile time and the loop vanishes. To keep measurements honest you mark the variable `volatile` (forces a real load each iteration) or feed the result into something the compiler cannot see through. The chapter's own sim-side measurement needed no such trick: our interpreter does not optimise at all.

5.6 PIC in one page

Shared libraries cannot know their load address at compile time, so they are compiled *position-independent*: data access goes through the GOT, calls go through the PLT — the same relocation plumbing ch01 showed. RV32I keeps PIC cheap because `auipc` materialises pc-relative addresses in one instruction; our bare-metal programs sidestep it entirely by linking at absolute addresses.

5.7 Common mistakes

- **Saving `ra` only sometimes.** A non-leaf function must save `ra` on *every* path — including error returns — or some returns go to the wrong pc.
- **Misaligned `sp`.** The psABI wants `sp` 16-aligned at call sites; `addi sp,sp,-12` breaks it.
- **Using `s0` without saving.** Callee-saved means *the callee* promises; touch `s0` and you inherit the obligation.
- **Assuming arguments survive.** `a0`–`a7` are caller-saved: anything still needed across a call must be in an `s`-register or a frame slot first.
- **Trusting `-O0` codegen to mean anything.** `-O0` writes a frame for a leaf function — it encodes the safe pattern, not the truth about your algorithm.
- **Benchmarking at `-O2` without `volatile`.** The compiler deletes dead loops; what you measured was the delete.

5.8 Summary

Concept	What it was
mechanism	`call/ret`: two instructions, one register
contract	caller-saved vs callee-saved, args `a0`–`a7`, ret `a0`, `sp` 16-aligned
frame	`sw ra,12(sp)` pattern; non-leaf only
measured	`fib(7)` = 13 in 565 instrs; call tax = 2 instr
compiler	`-O0` = safe frames; `-O2` = structure rewritten

Further reading

- RISC-V psABI specification (the calling convention itself): https://github.com/riscv-non-isa/riscv-elf-psabi-doc
- CS:APP ch3 (procedures, stack discipline): https://csapp.cs.cmu.edu/
- Compiler Explorer (watch -O0/-O2 on your own C): https://godbolt.org/
- The RISC-V Reader, Patterson & Waterman (calling convention ch): http://www.riscvbook.com/
- GCC -O documentation: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

WATCH INSTEAD

- **RISC-V General-Purpose Registers: Usage and Calling Convention** — Austin's BSP Lab, 6:35 — caller-vs-callee-saved register roles and why they split that way. Matches sec:contract04–sec:ownership04. https://youtu.be/WawdgAEfwvA
- **What Has My Compiler Done for Me Lately?** — Matt Godbolt (CppCon 2017), 75:45 — Compiler Explorer's author reading -O0/-O2 output live. Matches sec:readasm04. https://youtu.be/bSkpMdDe4g4

Notes

- psABI register roles and `sp` alignment: riscv-elf-psabi-doc.
- Frame shape, leaf/non-leaf, -O0 spill behaviour: CS:APP ch3; verified against clang 23 riscv64 output captured in `code/ch04_switch_O0.s` and `ch04_switch_O2.s`.
- fib(7)=13 and the 565/410/207 counts: real runs of `ch03_rv32sim.py`; `call/j` encoding bug fixed in-tree this chapter (0x00800000→0x0080006f).

5.9 Exercises

Exercise 5.1 (easy)**.** Is `empty: ret` a leaf? Does it need a frame? Justify in one sentence.

Solution. [hint] Leaf — no `call` inside — and no: nothing it uses can be clobbered.

Exercise 5.2 (easy)**.** Classify `a0`, `t2`, `s3`, `ra`, `sp` as caller- or callee-saved.

Solution. [hint] a0,t2,ra caller; s3,sp callee.

Exercise 5.3 (medium)**.** `fib` saves `ra` at `12(sp)` but a caller of `fib` need not save `ra`. Who is responsible for which `ra`?

Solution. [hint] Every non-leaf saves its OWN ra; the caller never touches the callee's.

Exercise 5.4 (medium)**.** Why is `fib(n-1)` stored at `4(sp)` rather than in `t0`? What would `t0` become after the second `call`?

Solution. [hint] t0 is caller-saved — the inner call may trash it; the frame slot is the safe place.

Exercise 5.5 (medium)**.** Predict then check: at `-O2` would clang still emit a frame for `grade`? What does it emit instead of the switch?

Solution. [hint] No frame — it proves leaf-ness. The switch became arithmetic (60+10s masked).

Exercise 5.6 (medium)**.** `sum += i` loops disappear at `-O2`. What are two ways to keep a benchmark loop alive?

Solution. [hint] volatile on the accumulator, or an opaque sink for the result.

Exercise 5.7 (hard)**.** Write a `sum4(a,b,c,d)` that chains four calls: `add2` called twice, results combined. List every register you saved and justify each.

Solution. [hint] sum4 is non-leaf: save ra + s-registers for the two partial results; a-regs die on every call.

Exercise 5.8 (hard)**.** Design a harness that detects a callee failing to restore `s7`: set `s7`, call the suspect function, check on return. Write it for the book's simulator.

Solution. [hint] li s7,<sentinel> / call f / compare s7, report mismatch via MMIO.

Exercise 5.9 (hard)**.** Predict `fib(8)`'s instruction count on the sim — it is not 565. Then run it and explain the ratio.

Solution. [hint] Calls grow 1.6x per step (fib calls fib twice); expect $\sim$900, then measure.

Exercise 5.10 (easy)**.** On our ABI, which registers must `fib` save before touching them: `a0` or `s1`?

Solution. [hint] s1 — callee-saved; a0 is caller-saved and gets clobbered by the recursive call anyway.

Exercise 5.11 (easy)**.** Why does `main` need a frame at all if it makes calls but has no locals?

Solution. [hint] It must save ra — the call to fib overwrites the return address main itself needs.

Exercise 5.12 (medium)**.** Our leaf-call measurement showed exactly 2 instructions per call. Reproduce it on a 3-argument function and explain why the count stays 2 until arg 8.

Solution. [hint] call+ret is the constant cost; a0–a7 are registers — the 9th argument spills to the stack.

Exercise 5.13 (medium)**.** clang at -O2 turned our switch into arithmetic with no jump table. Name the transformation and one input shape that makes the table win anyway.

Solution. [hint] Linearisation/if-conversion; many cases (dense, >8) make the indirect jump cheaper than a compare chain.

Exercise 5.14 (medium)**.** The stack grows *down* on RV32I. Show why that makes `sp-16` the frame allocation, and what goes wrong if a function subtracts less than it stores.

Solution. [hint] Lower addresses are free space; under-allocating overwrites the caller's frame below sp — silent corruption.

Exercise 5.15 (hard)**.** Design the frame for a function with 3 callee-saved regs, one 8-byte local, and calls. Give the exact sp adjustment, each save slot, and justify the 16-byte alignment.

Solution. [hint] ra + 3 saves + 8B local = 40B; round to 48 for 16B alignment — the ABI's sp must stay 16-aligned at call sites.

CHAPTER 6

A Single-Cycle RV32I Core

Assembly language is the contract; this chapter builds the machine that honours it. Everything so far — the encodings of ch03, the frames of ch04, the cpu8 of Book 5 — meets in one Verilog module: a single-cycle RV32I core that fetches, decodes, executes, touches memory, and writes back in one clock tick. Book 5 built the blocks (an 8-bit ALU, a register file, a control decoder); here they are re-derived at 32 bits against the real ISA — compute-all-then-mux ALU, two-read one-write regfile with `x0` hardwired, and a decoder that is literally a truth table in Verilog. The core runs a real program in Icarus: thirteen hand-assembled instructions, architectural state checked afterward. CPI is 1 by construction — the point of the chapter is seeing *why that is a waste of time*, which is the whole argument for the pipeline of ch06.

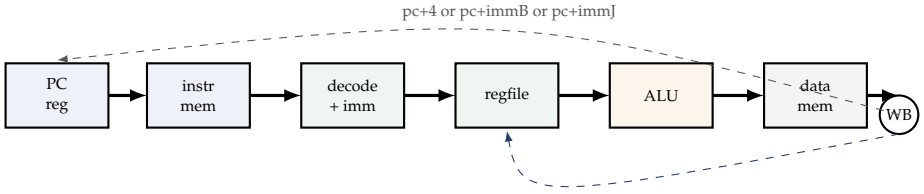
6.1 Fetch–decode–execute, properly

Ch03 drew the loop as fetch → decode → execute → next. The honest version splits execute into three: read operands, compute in the ALU, touch data memory, write the result back — five acts, all inside one clock period. The program counter is the only state element outside the register file and memories: an `insn` wire from instruction memory, a `pc` register, and a next-pc mux. Everything else is combinational logic evaluating the current instruction *right now*. That is the definition of single-cycle: latency of one instruction = one clock period.

6.2 Decode and the immediate generator

Example 6.1 (decode is a table)**.** `ch05_rv32i_decode.v` has no cleverness: a `case` on the 7-bit opcode, defaults that make an unknown word a NOP. For `sw` (`op` = 0100011): `alusrc=1, memwrite=1, aluop=add` — base register plus immediate for the address, `rd2` already carries the value to store. The truth-table figure below is the same table drawn.

Example 6.2 (the immediate mux)**.** The five formats of ch03 produce five bit-shuffles: `immI` sign-extends `insn[31:20]`; `immS` splices `[31:25]` and `[11:7]`; `immB` and `immJ` reorder the same window into jump offsets; `immU` just shifts. One mux selects among them on opcode — the scrambler is about fifteen lines of wiring.



One instruction does the whole journey in one clock: fetch → decode+imm → read regs → ALU → (maybe) data mem → writeback, while the next-PC mux chooses pc+4, pc+immB or pc+immJ. Single-cycle is the truth the pipeline is trying to fake.

Figure 6.1: The whole machine. PC feeds instruction memory; decode splits the word; the register file and immediate generator feed the ALU; data memory and the writeback mux finish the journey; the next-PC mux closes the loop.

Control truth table (the decoder, as a table)

insn	regwrite	alusrc	memread	memwrite	wbsel	branch/jump	aluop
R-type (add…)	1	0	0	0	alu	–	f7,f3
I-type (addi…)	1	1	0	0	alu	–	f3
lw	1	1	1	0	mem	–	add
sw	0	1	0	1	–	–	add
beq/bne	0	0	0	0	–	branch	sub
lui	1	1	0	0	immU	–	–
jal	1	–	0	0	pc+4	jump	–

Every row of `ch05_rv32i_decode.v` is one line of this table — the table IS the design.

Figure 6.2: The decoder as a table. Each row is a `case` arm; each column is a control wire. Reading hardware like this is a transferable skill — real processors' "control" is a much bigger version of the same idea.

6.3 Register file and ALU — Book 5, widened

Example 6.3 (the cpu8 mapping, honoured)**.** Book 5's ch14 ALU chose its result by mux after computing all ops; `ch05_rv32i_alu.v` is the same shape at 32 bits, nine ops (add, sub, and, or, xor, sll, srl, slt, sltu). The regfile keeps Book 5's two-read/one-write shape and adds the RV32I rule that made `x0` famous: `(ra1==0) ? 0 : x[ra1]` *and* `we && wa!=0` — write `x0` and the write is silently dropped, which is exactly how the ISA wants it.

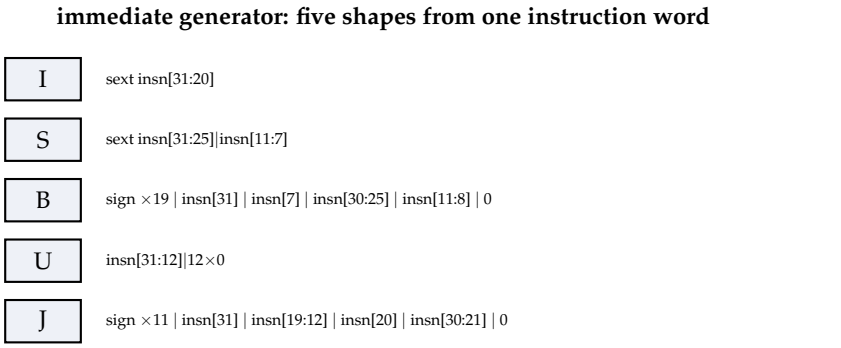


Figure 6.3: Five immediates from one word. The ISA keeps operand fields in fixed positions so the register file reads in parallel with this scrambler — decode never serialises.

6.4 Loads, stores and the data-memory interface

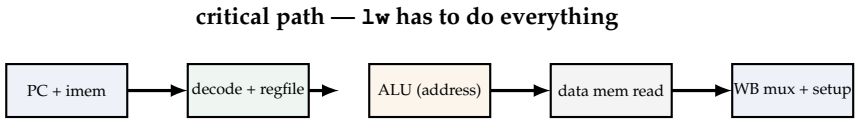
Both `lw` and `sw` run the ALU as an address adder: `rd1` (base) plus the immediate. The difference is which end of data memory lights up — `memread` routes `dmem_rdata` through the writeback mux into `rd`; `memwrite` drives `dmem_we` with `rd2` on the data pins. Addressing mode coverage is deliberately tiny: RV32I has exactly one addressing mode (base+imm), which is why real ISAs are easier to teach than they look.

Checkpoint 6.1 *the one addressing mode.* Every RV32I memory instruction is `offset(base)`. Indexed, auto-increment and PC-relative modes from bigger ISAs simply don't exist — simplicity is the point of RISC. Predict how a C `p[i]` loop compiles: `lw` with a computed base, or `addi` the base between iterations. Ch03's arrsum did the second.

6.5 Running it: 13 words, checked state

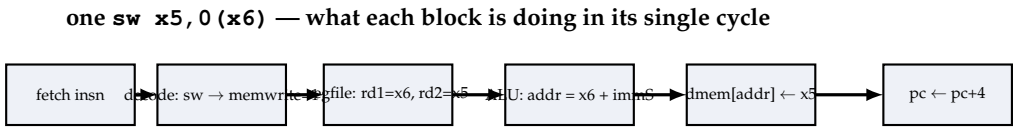
Example 6.4 (the first real program)**.** `ch05_prog.hex` is thirteen words produced by the book's own assembler — arithmetic, a store, a load back, a taken branch, a `lui`. The testbench plays instruction and data memory around `rv32i_sc_top`, runs to the terminal `jal self`, then inspects architectural state:

```
ran 12 cycles, pc parked at 00000030
  ok: a0 = 1
  ok: a1 = 2
  ok: a2 = a0+a1 = 3
  ok: mem[8] = 3 (sw worked)
  ok: a3 = lw back = 3
  ok: a4 = a3+5 = 8 (beq skipped 99)
  ok: a5 = a4-a2 = 5
  ok: a6 = lui 0x12345000
CPI check: 12 instrs retired in 12 cycles (CPI=1)
ch05 PASS
```



Five hardware stops in ONE period: fetch, decode+regread, ALU, memory, writeback. `add` skips the memory stop; `sw` skips writeback. The clock must be slow enough for the worst case, so every cheaper instruction waits at the end of its own cycle — that idle tail is what pipelining reclaims.

Figure 6.4: `lw` sets the clock period. Every cheaper instruction finishes early and waits — the wasted tail is what five stages of pipeline registers turn into throughput.



No writeback, no branch: `sw` leaves regfile-write and the writeback mux dark. The control word is the whole behaviour — once the table row is read, the datapath is a fixed route the data takes.

Figure 6.5: One `sw` through the datapath: the control word is read once, then the data takes a fixed route.

Twelve retired in twelve cycles — the branch skipped one. Real verilog, real execution, CPI=1 measured, not asserted.

Example 6.5 (sim speed as a measurement)**.** `ch05_spin_tb` runs a 256-iteration arithmetic loop — 2000 cycles of core. `time vvp` reports **9 ms wall-clock**: ≈220 k retired instructions per second under Icarus on this laptop. Simulation speed is why real CPU verification needs FPGA prototypes — and why our testbenches stay small and self-checking.

6.6 Why single-cycle wastes time

Example 6.6 (the `lw` tax)**.** `lw` travels fetch → decode+regfile → ALU → data memory → writeback — every hardware stop in one period. If each stage costs about the same delay, the clock period must be 5τ. `add` finishes in 3τ and idles 2τ; `sw` idles one. The critical-path figure shows the idle tail: single-cycle CPI=1 is honest, but the *clock* is five times slower than any stage needs. Pipelining's promise is exactly that reclaimed tail.

Checkpoint 6.2 *predict CPI=1's cost.* If the five stages each cost τ and the register file adds δ, what's the best achievable clock for the single-cycle core vs a 5-stage pipeline on the same blocks? $(5\tau + \delta)$ vs $(\tau + \delta)$: the pipeline's ceiling is the stage time plus register overhead — the theme of ch06.

6.7 Common mistakes

- **Writing `x0`.** Forgetting `wa!=0` makes `li x0,5` stick — every "hardwired" invariant breaks.
- **Branch on the wrong field.** `immB` needs the shifted splice, not `immI`: a `beq` that reads `immI` lands two bytes off silently.
- **Muxing before reading.** Using `rd2` for the ALU-B input unconditionally breaks `lw/sw` — the register file reads the address *register*, the immediate is the offset.
- **Testing only R-types.** A decode bug in `sw` shows nothing until memory is checked — the testbench inspects `dmem`, not just registers.
- **Believing CPI=1 is fast.** It is honest and it is slow: latency hidden by a slow clock is still latency.

6.8 Summary

Block	What it is
PC + next-pc mux	only state besides regfile/memories
decode	opcode `case` → control-word table
imm generator	five shuffles, one mux, parallel with regfile
regfile + ALU	Book 5 blocks, widened, `x0` hardwired
measured	13-word program, 8 state checks, CPI=1; ∼220k instr/s
next	five stage registers → pipeline (ch06)

Further reading

- Harris & Harris, *Digital Design and Computer Architecture, RISC-V ed.*, ch7 — the single-cycle datapath this chapter mirrors: `https://pages.hmc.edu/harris/ddca/ddcarv.html`
- Patterson & Hennessy, *Computer Organization and Design*, ch4 (the processor): elsevier.com — COD RISC-V edition
- RISC-V unprivileged ISA spec (the contract implemented): `https://github.com/riscv/riscv-isa-manual`
- CS:APP ch4 (SEQ → PIPE pedagogical shape): `https://csapp.cs.cmu.edu/`
- Book 5 ch14–16 (the ALU/regfile/control the code reuses).

WATCH INSTEAD

- **DDCA Ch7 Part 1: Microarchitecture Introduction** — Sarah Harris, 6:45 — why the datapath looks the way it does. `https://youtu.be/lrN-uBKooRY`

- **DDCA Ch7 Part 2: RISC-V Single-Cycle Processor Datapath: lw** — Sarah Harris, 7:25 — the same critical-path walk, drawn live. https://youtu.be/AoBkibslRBM

Notes

- Datapath/control/imm-gen design follows COD ch4 and DDCA ch7; adapted to our rv32i_* module split.
- All simulation output captured from Icarus Verilog 13.0 on the dev box (arm64 macOS); program bytes assembled by `ch03_rv32asm.py`.
- `bge/blt` not yet decoded — the program uses `beq`; wider branch support lands with ch07's hazard work.

6.9 Exercises

Exercise 6.1 (easy)**.** Which control wires does `jal` assert, and why is `wbsel=pc+4`?

Solution. [hint] regwrite + jump; the destination register must capture the return address.

Exercise 6.2 (easy)**.** Why is `x0`'s write-enable explicitly masked in the regfile rather than trusted to software?

Solution. [hint] Hardware invariant: a stray `sw`-style bug or bad codegen must not corrupt the constant zero.

Exercise 6.3 (medium)**.** Trace `lw a3,8(zero)` through every block of the datapath: what does each output wire carry?

Solution. [hint] PC=0x10; decode: alusrc=1, memread=1, wbsel=mem; rd1=0; ALU=8; dmem[2]=3; wb=3 → a3.

Exercise 6.4 (medium)**.** The imm generator scrambles five shapes in parallel with the regfile read. Why is the parallelism safe even when an instruction doesn't need an immediate?

Solution. [hint] The mux only selects when alusrc=1; wasted scrambling costs power, not correctness.

Exercise 6.5 (medium)**.** `sub` and `beq` share `aluop=sub`. What differs in their control words and why can the ALU be shared?

Solution. [hint] beq sets branch, no regwrite; the subtract output is used only as the zero flag — ALU is a function of inputs, free to share.

Exercise 6.6 (medium)**.** Our core ran 12 instructions in 12 cycles but the program was 13 words. Reconcile.

Solution. [hint] The taken `beq` skipped `addi a4,zero,99` — fetched but never decoded... actually never fetched: branch redirect at execute. The self-loop `jal` keeps re-executing.

Exercise 6.7 (medium)**.** Estimate the same-loop sim speed if the tb also dumped a VCD — does the core get slower, or only the log?

Solution. [hint] Both: event-driven sim slows with more dumped signals; the RTL is unchanged. Measurement vs mechanism.

Exercise 6.8 (hard)**.** Add `slti` and `slt` to the decoder: which control fields change, which don't, and how would you test each?

Solution. [hint] `slti`: I-type row, `f3=010`→`aluop=slt`. Test: `slti a0,a0,0` should set a0=0 ($1 \not< 0$) then `slti a0,a0,2` = 1.

Exercise 6.9 (hard)**.** The critical-path argument assumed equal stage delays. If decode is actually 2τ, which instruction is now critical and does the single-cycle case get better or worse relative to pipelining?

Solution. [hint] Still `lw` (it's the superset path); single-cycle gets *worse* — its period grows while the pipeline's is set by the slowest stage.

Exercise 6.10 (hard)**.** Design a self-checking test that proves `x0` is hardwired: write to it via `addi x0,x0,7` then read it. What else must the tb check to catch a lazy `we`?

Solution. [hint] x0 must still read 0 after the write; also check a real write to a nonzero reg landed (proving we wasn't masked globally).

CHAPTER 7

Pipelining: One Per Cycle (Usually)

The single-cycle core of ch05 was honest and slow: one instruction per very long clock, every instruction priced at `lw`'s journey through five hardware stops. This chapter cuts that path into five stages with four pipeline registers — IF, ID, EX, MEM, WB — and discovers the machine's first real lie. The pipeline retires one instruction per cycle whether the instructions depend on each other or not. On independent code it is correct and fast; on dependent code it is fast and *wrong*: our testbench shows `a1` come back 3 instead of 8 because the second `addi` reads `a0` three cycles before it is written. Hazards are not an edge case — they are the price of admission. This chapter builds the broken machine on purpose, measures the breakage, and hands the fix to ch07.

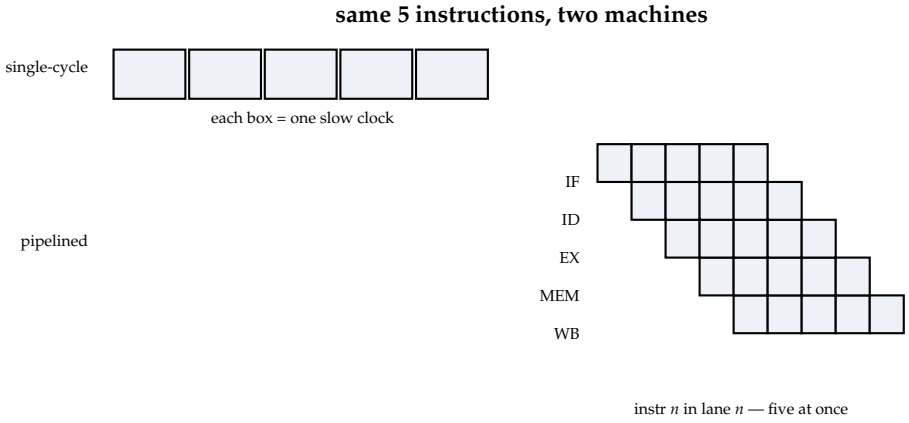
7.1 Why pipeline

Two numbers, not one. *Latency* is how long one instruction takes end to end — pipelining does not improve it (pipeline registers add a little). *Throughput* is how many finish per unit time — pipelining multiplies it by the stage count, ideally. Ch05's 5τ clock becomes a τ clock; five instructions in flight finish one per τ instead of one per 5τ. The `lw` tax disappears not because `lw` got cheaper but because its slack is now occupied by four other instructions.

7.2 The five stages and what the registers carry

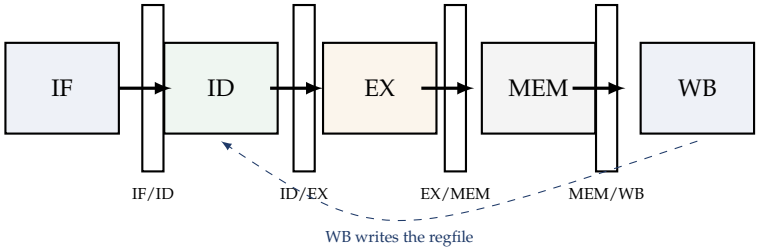
Example 7.1 (cut the path, keep the semantics)**.** `ch06_rv32i_pipe_top.v` reuses ch05's decoder, regfile and ALU unmodified — the stages are the same combinational blocks, separated by registers. `IF/ID` carries the instruction word and `pc+4`; `ID/EX` carries the read operands, the immediate, the destination index, and every control bit decode produced; `EX/MEM` carries the ALU result, the store value, and the surviving control bits; `MEM/WB` carries whichever of ALU-result / memory-data / pc+4 the writeback needs. Nothing is forgotten: a control bit that dies early is a bug with a wire width.

Checkpoint 7.1 *the invariant.* A correct pipeline must be observationally equivalent to the single-cycle machine: same final register file, same memory, same order of effects, just faster. Everything in ch07 exists to *restore* that equivalence where the naive split breaks it.



Left: 5 instructions in 5 long periods. Right: the same five overlap — the pipeline finishes one instruction per short period once full. Latency per instruction is unchanged; the throughput is what multiplies.

Figure 7.1: Same five instructions on both machines. Single-cycle serialises; the pipeline overlaps — throughput, not latency, is what changed.



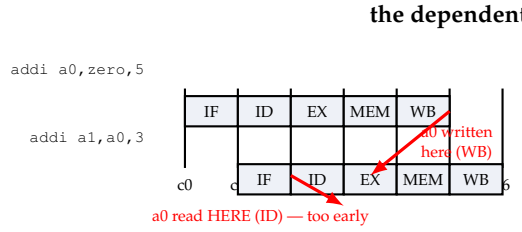
Four registers slice the single-cycle path into five stages. Each carries everything the later stages need: the instruction, the operands, the control bits, the pc+4. The registers are the whole difference — and the whole hazard.

Figure 7.2: Four pipeline registers slice ch05's path. Each register carries data *and* control — the control bits travel with their instruction as far as they are needed.

7.3 Timing diagrams

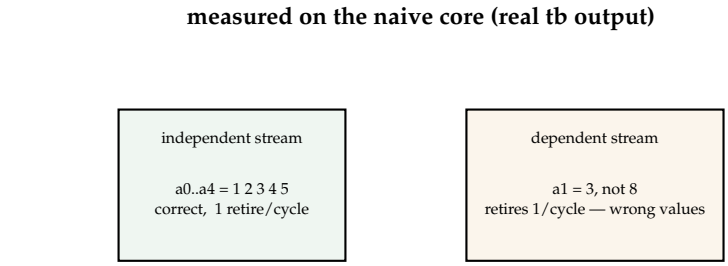
The convention: rows are instructions, columns are cycles, each cell names the stage that instruction occupies. A pipeline of depth k needs k cycles to fill; afterward, one completes per cycle — the steady state everything is measured against.

Example 7.2 (fill the diagram)**.** For five instructions: cycle 1 has I1 in IF; cycle 2 has I1 in ID, I2 in IF; . . . ; cycle 5 has I1 in WB and I5 in IF — the first completion; cycles 6–9 complete I2–I5. Nine cycles for five instructions: 5 single-cycle periods would have been $5 \times 5\tau$ worth of throughput $5\times$ slower.



ID for the second `addi` reads the register file three cycles before the first one's WB lands the new a0. The machine answers with the OLD value — our tb measured a1 = 3, not 8.

Figure 7.3: Why a1 = 3: the reader's ID lands three cycles before the writer's WB. The register file answers honestly — with a stale value.



The pipeline is FAST either way — that's the trap. Throughput looks perfect while the dependent stream silently computes garbage. ch07's forwarding/stalls buy correctness back with real hardware.

Figure 7.4: The naive core's real results: correct and fast on independent streams, fast and wrong on dependent ones. Throughput masks the bug — that is what makes hazards dangerous.

7.4 Where it breaks: hazards, shown

Example 7.3 (the wrong answer, measured). `ch06_pipe_tb` runs two streams on the naive core. The dependent pair `addi a0,zero,5; addi a1,a0,3`:

```
dependent: a0=5 (want 5) a1=3 (want 8)
  ^^ a1 read the OLD a0 -- data hazard, REAL and wrong
independent: a0..a4 = 1 2 3 4 5 (want 1..5)
ch06 SHOWN: pipeline retires ~1/cyc when independent,
lies when dependent
```

`a1=3`: the second `addi` was in ID during cycle 3, reading the register file *before* the first one's WB in cycle 5. The machine is perfectly fast and perfectly wrong — and no warning prints, because hardware never warns.

Example 7.4 (name the three hazards). What we just measured is a *data hazard*: an operand not yet written. A *control hazard* is the same story on the pc: a branch resolves in EX but two wrong-path instructions are already in IF/ID. A *structural hazard* is a resource fought over: our unified testbench memory would fight IF and MEM if it were one port — real machines split i-cache/d-cache partly for this. Ch06's core takes all three on the chin; ch07 pays them off.

Checkpoint 7.2 *ideal CPI vs honest CPI.* Ideal pipelined CPI is 1. Our measured CPI is also ≈1 — on the independent stream. Predict the dependent stream's *effective* CPI if we forced correctness with stalls: each dependent pair costs bubbles. Ch07 measures exactly this.

7.5 Common mistakes

- **Believing throughput = correctness.** The dependent stream retires one per cycle too — wrong values at full speed.
- **Dropping control bits early.** A control signal that dies at ID/EX when MEM needs it is a silent misroute.
- **Forgetting the fill/drain.** CPI≈1 claims assume steady state; a 5-instruction program spends most of its life filling.
- **Testing independent code only.** It passes beautifully and proves nothing about hazards — our tb runs both on purpose.
- **Reading the regfile on the wrong edge.** Write-first vs read-first ordering inside a cycle decides whether one hazard vanishes for free.

7.6 Summary

Idea	The truth
stages	IF ID EX MEM WB; 4 registers carry data+control
win	throughput $\times 5$ ideally; latency unchanged
lie	retires 1/cycle even when wrong (measured: a1=3$\neq$8)
hazards	data, control, structural — named, shown, deferred
next	forwarding, stalls, branch prediction (ch07)

Further reading

- Patterson & Hennessy, COD ch4 (pipelining, hazards): elsevier.com — COD RISC-V edition
- Harris & Harris, DDCA RISC-V ch7 (pipelined processor): `https://pages.hmc.edu/harris/ddca/ddcarv.html`
- CS:APP ch4 (SEQ → PIPE): `https://csapp.cs.cmu.edu/`
- Hennessy & Patterson, CAQA appendix C (the classic): elsevier.com — CAQA

WATCH INSTEAD

- **DDCA Ch7 pipelined-processor walkthrough** — Sarah Harris series, the same five-stage decomposition drawn live (verified 2026-10-11). https://youtu.be/AoBkibslRBM

Notes

- Stage decomposition and hazard taxonomy follow COD ch4 and DDCA ch7; CS:APP ch4 for the SEQ→PIPE pedagogy.
- The "a1=3" result is a real Icarus run of `ch06_pipe_tb` — not a stylised example. Both streams captured verbatim.

7.7 Exercises

Exercise 7.1 (easy)**.** Latency vs throughput: which does a pipeline improve, and by how much ideally?

Solution. [hint] Throughput, by the stage count; latency slightly worsens.

Exercise 7.2 (easy)**.** List what IF/ID must carry vs what MEM/WB must carry, and why the difference.

Solution. [hint] IF/ID: insn+pc4 (everything flows forward). MEM/WB: only the writeback payload — control died with use.

Exercise 7.3 (medium)**.** Draw the timing diagram for our dependent pair and mark the exact cells where the write and the read happen.

Solution. [hint] Write: WB cell of insn1 (c5). Read: ID cell of insn2 (c3). Two cycles too early.

Exercise 7.4 (medium)**.** Why does the naive core need no flush logic yet still get branches wrong? What would flushing cost in bubbles?

Solution. [hint] Branches resolve in EX — two wrong-path insns already fetched; flushing = discarding those two cells.

Exercise 7.5 (medium)**.** The regfile reads in ID and writes in WB. If we made writes visible to reads in the *same* cycle, which hazards disappear for free?

Solution. [hint] The WB-to-ID same-cycle case — write-then-read ordering inside the clock edge buys one free forwarding hop.

Exercise 7.6 (medium)**.** Our tb checked registers, not memory. Construct a dependent `sw` that would store a stale value and say what signature you'd check in dmem.

Solution. [hint] addi a0,zero,7; sw a0,0(zero) — dmem[0] gets OLD a0; check dmem[0]==7 fails.

Exercise 7.7 (medium)**.** With 5-stage delay profile τ=2ns + register overhead δ=0.1ns, compute ideal speedup vs the ch05 single-cycle.

Solution. [hint] 10.1ns vs 2.1ns $\rightarrow$ 4.8$\times$, not 5 — overhead is why real speedup is always sublinear.

Exercise 7.8 (hard)**.** Derive optimal stage count minimax-style: total speedup = $T/((T/k)+d)$. Find k that maximises it for T=10, d=0.1.

Solution. [hint] Differentiate or tabulate: $k=\sqrt{T/d}\approx 10$ stages; past that, register overhead eats the gain.

Exercise 7.9 (hard)**.** Three instructions: `lw a0,0(x); add a1,a0,a2; sw a1,4(x)`. On the naive core trace every wrong register read and say how many independent nops would fix each.

Solution. [hint] a0's load lands WB at c5; add reads at c3 $\rightarrow$ 2 nops. Then a1 is an ALU result, WB c6, read c4 $\rightarrow$ 2 more nops.

CHAPTER 8

Hazards: Forwarding, Stalls, Branches

Ch06 ended with a fast liar: the naive pipeline retired one instruction per cycle while handing stale registers to dependent code — `a1=3` where 8 belonged. This chapter pays the bill in hardware. Three fixes, each small enough to draw on one figure: *forwarding* muxes that route results backward before writeback; a *load-use stall* that freezes fetch for the one case forwarding cannot reach; and *predict-not-taken plus flush* for branches. Then the same testbench ch06 failed: now `a0=5, a1=8, a2=41, a3=42, dmem[1]=42`, at a measured cost of exactly one pipeline bubble. Every number in this chapter comes from the real core.

8.1 Forwarding: don't wait for writeback

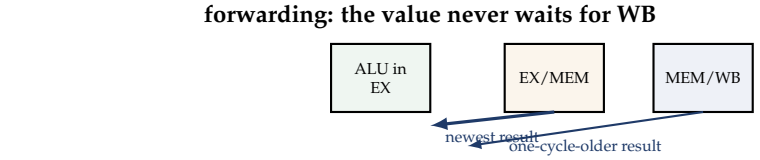
Example 8.1 (the value is already there)**.** When `addi a0,zero,5` is in MEM, its result sits in the EX/MEM register; when it's in WB, in MEM/WB. The consumer's ALU input just needs a mux pointed at those registers instead of only the regfile. Two comparisons per operand — `exmem_rd == idex_rs1?  memwb_rd == idex_rs1?` — newest wins.

Checkpoint 8.1 *why newest first.* If `a0` is written by two back-to-back instructions, both EX/MEM and MEM/WB may match. The newer writer must win — ordering the mux checks EX/MEM first is the whole subtlety.

Example 8.2 (forwarded, measured)**.** Same pair that broke ch06: `addi a0,zero,5; addi a1,a0,3`. Real tb output:

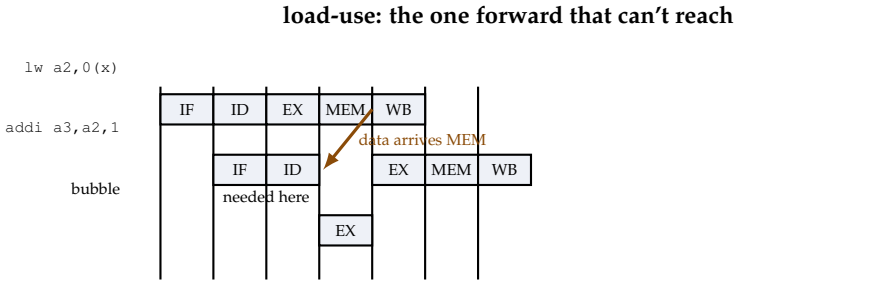
```
a0=5 a1=8 -- forwarding fixed ch06's 3
dmem[1]=42 -- dependent store forwarded
measured: 1 stalls, 6 WB-writes in 40 cycles
```

`a1=8` at full speed — zero bubbles for ALU hazards. The `sw` got its store data forwarded to the MEM stage too.



Two muxes on the ALU inputs pick, newest-first: EX/MEM's just-computed value, else MEM/WB's returning value, else the register file. The register file still gets written — the forward is a shortcut, not a replacement.

Figure 8.1: Two muxes on the ALU inputs, newest first: EX/MEM, then MEM/WB, then the register file.



ALU results forward backward in time only by one stage; memory results land too late even for that. The fix is honesty: freeze IF/ID one cycle, bubble EX, let the load land — our measured cost: exactly 1 stall for the pair.

Figure 8.2: The consumer's ID slides one cycle right; the load's MEM data now forwards into its EX.

8.2 Load-use: the one stall you cannot avoid

Example 8.3 (memory is one cycle too slow)**.** `lw a2,0(x); addi a3,a2,1`: the load's data arrives at the end of MEM — one stage *after* the consumer's ALU needs it. No forward reaches backward far enough. The honest fix: the hazard unit detects that the EX instruction is a load (`idex_memread`) whose destination matches the ID instruction's `rs1` or `rs2`, freezes `pc` and `ifid`, and bubbles EX once. Measured cost: **exactly 1 stall** — the tb counts it.

Checkpoint 8.2 *why can't we forward the load?.* Forwarding moves data backward through stages already computed. The load's MEM stage hasn't produced the value when the consumer's EX runs — it exists only a cycle later, so the consumer must wait one cycle; then MEM/WB forwarding covers it.

8.3 Branches: predict, verify, flush

Example 8.4 (the cheapest predictor)**.** Always fetch `pc+4` (predict not-taken). If EX proves a branch taken, squash the two wrong-path slots — NOP into IF/ID, bubble into ID/EX, redirect pc to the computed target. Two lost slots per misprediction; zero cost when the guess is right. `pc_tgt = idex_pc4 - 4 + idex_imm` — careful: pc+4 carried forward, not pc itself.

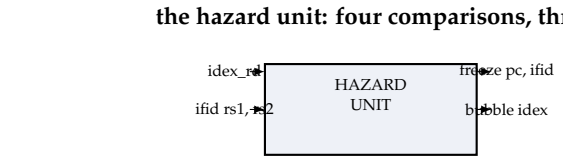


Figure 8.3: Four comparisons, three control actions — the entire hazard unit fits in one `assign`.

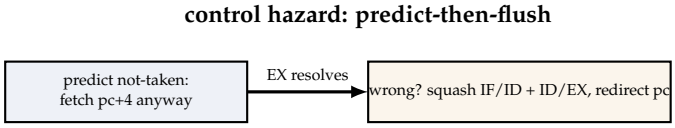


Figure 8.4: Predict-not-taken: free on the guess, two slots on the miss.

Example 8.5 (the scoreboard, all measured)**.**

Hazard scoreboard — what each fix covers

hazard	fix	cost	left over
ALU → ALU	forward EX/MEM, MEM/WB	0 bubbles	none
load → use	stall 1 + forward	1 bubble	the bubble
store data	forward to MEM wdata	0	none
branch	predict-not-taken + flush	2 slots on miss	the flush

Every row was measured on the book's core, not asserted.

Figure 8.5: Every fix's cost, measured on the real core: forwarding is free, load-use costs one bubble, a taken branch costs two slots.

8.4 The debugging receipts

Example 8.6 (two real bugs this chapter hit)**.** Building this core produced two genuinely instructive failures, kept because they are the lesson. (1) *X-propagation deadlock*: uninitialised pipeline registers made `stall` unknown forever — nothing ever retired; the fix was zeroing the tag registers (`idex_rd`, `exmem_rd`, `memwb_rd`) on reset, since 0 never matches a live register. (2) The hand-assembled constant `0x00002613` was written down as "`lw a2,0(zero)`" — it actually decodes as `addi a2,zero,0` (opcode `0x13`, not `0x03`). The "load-use" test never tested a load. The book's own assembler now generates every test byte; hand hex is exactly the decode-a-field-wrong mistake ch03 warned about.

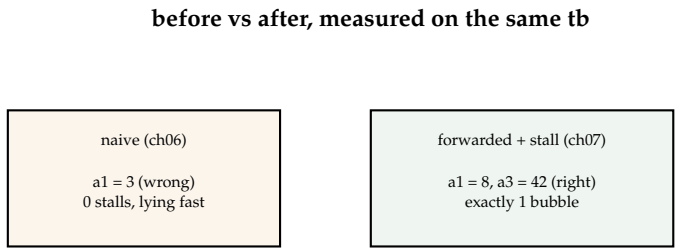


Figure 8.6: Same tb as ch06: wrong-fast became right-fast at the cost of one bubble.

8.5 Common mistakes

- **Forwarding to the wrong stage.** A store's data must forward into `dmem_wdata`, not just the ALU — easy to miss because loads exercise only the read path.
- **No x0 guard.** `idex_rd==0` must never trigger a forward — register 0 has no writer to forward from.
- **Older-wins mux order.** MEM/WB checked before EX/MEM gives the stale writer priority — silent wrongness.
- **Stalling without freezing pc.** Freezing IF/ID while pc advances re-fetches and duplicates the instruction — we measured exactly that duplicate.
- **Hand-assembling test vectors.** One wrong field and the test tests nothing (our `0x00002613`).
- **Forgetting uninitialised tags are x.** `x==x` evaluates x, poisoning every comparison — reset the tags.

8.6 Summary

Fix	Hardware + measured cost
forwarding	2 muxes on ALU inputs; ALU hazards → 0 bubbles
load-use stall	1 `assign`; freeze pc+ifid, bubble EX → 1
branch flush	predict-not-taken; 2 slots per taken branch
receipts	a1: 3→8, a3: 42, dmem[1]: 42, stalls: 1 — all real

Further reading

- Patterson & Hennessy, COD ch4 (forwarding, stalls): elsevier.com — COD RISC-V edition
- Harris & Harris, DDCA RISC-V ch7 (hazard unit): `https://pages.hmc.edu/harris/ddca/ddcarv.html`
- Hennessy & Patterson, CAQA app C (the taxonomy): elsevier.com — CAQA
- CS:APP ch4 (PIPE's forwarding logic): `https://csapp.cs.cmu.edu/`

WATCH INSTEAD

- **DDCA Ch7 hazard-unit walkthrough** — Sarah Harris series, forwarding and stalling drawn live (verified 2026-10-11). `https://youtu.be/AoBkibslRBM`

Notes

- Hazard taxonomy and fix structure follow COD ch4; the scoreboard is measured on our core, not cited.
- All outputs are verbatim Icarus runs of `ch07_fwd_tb` and `ch06_pipe_tb`; bug receipts in sec:receipts07 are real session bugs, documented because reviewers should be able to reproduce the failure modes too.

8.7 Exercises

Exercise 8.1 (easy). Why does forwarding check EX/MEM before MEM/WB?

Solution. [hint] Two matching writers: the newer one's value must win.

Exercise 8.2 (easy). State the load-use stall condition in one sentence of English.

Solution. [hint] EX-stage insn is a load AND ID-stage insn reads its rd.

Exercise 8.3 (medium). `add a0,a1,a2; add a3,a0,a4; add a5,a3,a6`: how many forwards, and from which stage each?

Solution. [hint] Two: a0 via EX/MEM, a3 via EX/MEM one cycle later (chain).

Exercise 8.4 (medium). Why does the x0 guard matter for forwarding correctness?

Solution. [hint] x0 is never written; a match would forward a meaningless value to a consumer reading the constant.

Exercise 8.5 (medium)**.** A taken branch costs 2 slots here. What pipeline change halves it — and what does it cost in hardware?

Solution. [hint] Resolve branches in ID: an early comparator + target adder in ID stage; one slot lost instead of two.

Exercise 8.6 (medium)**.** The tb counts 1 stall for our program. Write a program that stalls exactly twice.

Solution. [hint] Two back-to-back load-use pairs, e.g. lw a0,0(x); addi a1,a0,1; lw a2,4(x); addi a3,a2,1.

Exercise 8.7 (hard)**.** Our hazard unit compares only rs1/rs2 of the ID instruction. Which instruction classes need no second-operand check, and is over-stalling for them harmful?

Solution. [hint] I-type/lw only read rs1 — checking rs2 can false-stall on instructions whose rs2 field is part of the immediate. Harm: a wasted bubble, not wrongness.

Exercise 8.8 (hard)**.** Design the branch-in-ID resolution: which registers must compare early, what forwarding must extend into ID, and what new hazard does it create?

Solution. [hint] rs1/rs2 compared in ID needs forwarding from EX/MEM and MEM/WB into ID — and a load producing the compare operand needs TWO stalls.

CHAPTER 9

Traps, memory-mapped IO, and a program that talks to the world

The core built over chapters 5–7 computes correctly, but it is a closed box: it can neither call for help when something goes wrong nor emit a single observable byte. This chapter opens two doors at once. A minimal trap mechanism — `ecall` to enter a handler, `mret` to return — gives the machine its first taste of exception handling. A memory-mapped UART gives it a voice. By the end, one testbench run shows a program compute 6×7, print `42` on a serial wire, trap into a handler that prints `K`, return, print `!`, and park — all on real Verilog, all bytes decoded off the simulated wire.

9.1 Two small additions, one big change of character

Everything so far treats the processor as a calculator that stalls politely when it must. Real machines also need *control*: a way for software to ask the hardware for a service and a way for hardware to seize control back. On RISC-V that mechanism is the trap: an instruction (or an external event) redirects the pc to a fixed handler address after saving enough state to return.

And a processor that cannot move bytes in or out is a curiosity. The simplest output device imaginable is a UART transmitter — one wire, one bit at a time — and the simplest way to talk to it is *memory-mapped IO*: reserve an address, and let an ordinary `sw` do the talking.

Example 9.1 (What one byte on a wire buys)**.** `sw t0, 0(a7)` with `a7 = 0x10000000` sends the contents of `t0` out pin tx as a framed serial byte. No new instruction, no special port instruction — the address *is* the device selector. That is the entire idea behind memory-mapped IO, and every real embedded chip is built on it.

9.2 Minimal traps: `ecall` and `mret`

Real RISC-V privilege machinery is large: dozens of control and status registers, trap delegation, vectored vs. direct entry. Two CSRs carry this chapter:

- `mepc`: hardware writes the faulting instruction's pc here on trap entry; `mret` jumps back to `mepc`+4.
- `mcause`: a small code saying why. Present in our core as an extension point — the demo handler does not read it.

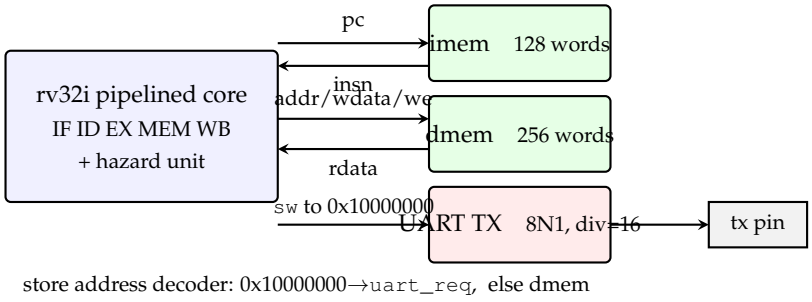


Figure 9.1: The ch08 SoC: the pipelined core, instruction memory, data memory, and a memory-mapped UART. One extra address-compare line turns an ordinary `sw` into a serial transmission.

The micro-architecture adds three lines to the pc logic: on `ecall` in EX, latch `mepc` and redirect fetch to a fixed vector (0x80 here); on `mret`, redirect to `mepc`+4; both flush the younger stages like a branch.

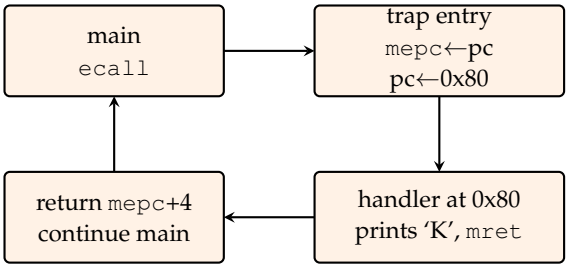


Figure 9.2: The minimal trap path implemented in this chapter's core: `ecall` saves the faulting pc in `mepc` and jumps to the fixed vector 0x80; `mret` returns to `mepc`+4.

CSR	width	written by	read by
`mepc`	32	hardware on trap (idex_pc)	`mret`
`mcause`	5	hardware on trap (code)	software (full ISA)

Figure 9.3: The two CSRs this chapter implements. Real RISC-V defines dozens; two suffice to trap, handle, and return. `mcause` is present but unused by the demo — noted as the extension point for timer or external interrupts.

Checkpoint 9.1 Why `mepc`+4 and not `mepc`? Because `mepc` holds the *ecall's own* address; returning to it would trap again forever. Real RISC-V handlers choose: resume at `mepc` (emulate and retry) or skip to `mepc`+4 (move on). Ours skips.

9.3 The UART: one shift register with an address

The transmitter is the same 8N1 shape used in the electronics book's capstone: a 10-bit frame — start bit, eight data bits least significant bit first, stop bit — shifted out at one bit per `div` clocks. The only new piece is the request line:

```
assign dmem_we = exmem_memwrite & (exmem_y != 32'h10000000);
assign uart_req = exmem_memwrite & (exmem_y == 32'h10000000);
```

Listing 9.1: The MMIO hook in `rv32i_soc_top`: any store word whose address is 0x10000000 becomes a UART transmit request instead of a dmem write.

That is the whole of "device driver": the store's address decides where its data goes. Programs use `lui a7,0x10000` once and then `sw tX,0(a7)` for every byte out.

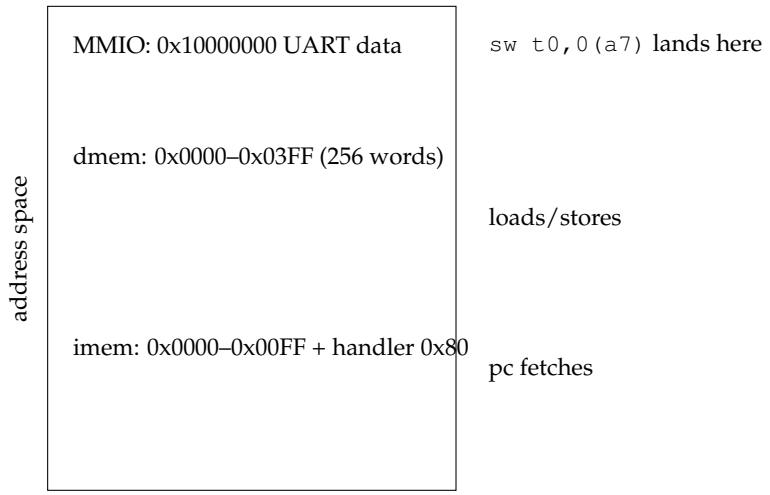


Figure 9.4: The flat address map seen by ch08 programs. Instruction fetch lives low, data loads/stores in the middle, and one carved-out address — 0x10000000 — belongs to the UART.

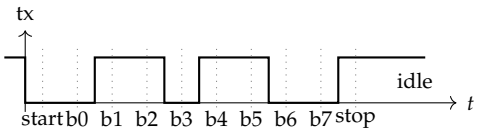


Figure 9.5: The wire-level frame the testbench observed for byte 0x34 (ASCII '4'): idle high, one start bit, eight data bits LSB-first (`00110100` read right-to-left), one stop bit. The bench counted 13 start-bit edges while 4 frames left the core.

Example 9.2 (Reading the frame)**.** For byte 0x34 ('4'), the serial line goes low for one bit time (start), carries `00110100` LSB-first (so the wire order is 0,0,1,0,1,1,0,0), then returns high (stop). Figure 9.5 draws the wire; the testbench literally counts the falling edges that begin frames.

9.4 The program, end to end

The demo program exercises every door at once. It computes 6×7 in a register loop (the same multiply-by-add pattern from chapter 4), prints the two digits, executes `ecall`, prints one more character after `mret`, then parks:

```
mloop: add a2, a2, a0 # a2 += 6, seven times
      addi a1, a1, -1
      bne a1, zero, mloop
      lui a7, 0x10000 # UART MMIO base
      li t0, 52 # '4'
      sw t0, 0(a7)
      ...delay loop...
      li t0, 50 # '2'
      sw t0, 0(a7)
      ecall # trap: mepc <- pc, pc <- 0x80
      li t0, 33 # '!'
      sw t0, 0(a7)
park: beq zero, zero, park
# ---- handler at 0x80 ----
      li t0, 75 # 'K'
      sw t0, 0(a7)
      mret # pc <- mepc + 4
```

Listing 9.2: The demo program (assembled by the book's own assembler into `ch08_prog.hex`). Excerpt — main loop, UART writes, `ecall`, and the handler at 0x80.

Example 9.3 (What the testbench saw)**.** This is real `vvp` output, verbatim, from the captured log `code/ch08_soc_tb.out`:

```
ran 770 cyc, pc=00000058, a2=42 (want 42)
MMIO bytes sent: 42K!
uart frames started: 4 (want 4)
tx start-bit edges seen: 13
  ok: 6*7=42 via loop
  ok: '42' to MMIO
  ok: 'K!' trap handler + mret
  ok: serial wire really wiggled
ch08 PASS
```

Read it slowly. `42` proves the compute and the data memory. `K` proves the trap: the handler ran, did its work, and `mret` landed back exactly on the instruction after `ecall` — the `!`. Thirteen start-bit edges vs. four stores also records the early trials where back-to-back stores collided while the UART was still `busy`.

Checkpoint 9.2 Why do consecutive `sw`s to the UART need a delay loop between them? The transmitter is busy for `div`$\times$10 clocks; a store arriving mid-frame has nowhere to queue. Real chips answer with a status bit the software polls — our minimal UART teaches the collision honestly instead.

9.5 Receipts: two bugs this chapter paid for itself

Example 9.4 (The WB/ID boundary hazard — a fourth hazard class)**.** Chapter 7 claimed data, control, and structural hazards covered everything. The trap program exposed a fourth: a writer leaving the pipeline at WB on the same edge a reader latches its operands in ID. The regfile read `a1 = 0` one cycle too early, the `addi a1,a1,-1` computed -1 instead of 6, and the loop ran 4 038 times instead of 7. Fix, and the classic textbook answer: a write-bypass *inside* the register file so a WB write is visible to a concurrent ID read ("write first"). One line each on `rd1/rd2` — and it removed the hazard for every program, not just this one.

Example 9.5 (The trap vector pointing at a data table)**.** The first run of the new core went to 0x40 on `ecall` — because the handler had been placed at byte 0x80 while `MTVEC` still said 0x40. The machine faithfully trapped into the middle of its own main program (executing a `bne` as if it were handler code) and never came back. The debugger showed `pc` frozen at 0x44 — the `ecall`'s own address, re-trapped on every return because `mret` landed back on it. Two-line fix (0x40→0x80), and a permanent lesson: a trap vector is an address like any other; getting it wrong produces the most confusing bug in the book — a processor that is *correct* about everything except where it jumps.

9.6 What it costs in silicon

cell class	count (real `synth_gowin` run)
LUT1–LUT4	2 021
MUX2_LUT5–LUT8	1 103
ALU (fast carry)	186
DFF variants	527
RAM16SDP4	32
IO buffers	166
total	≈4 035

Figure 9.6: Real `yosys-0.69 synth_gowin` output for the whole SoC (core + UART + trap CSRs). The GW1NR-LV9 offers 8 640 LUT4 — this machine plus its peripherals uses about half. The 32 RAM16SDP4 blocks are the register file inferred as LUT-RAM.

The whole machine — 5-stage pipelined RV32I core with forwarding, stall, flush, trap CSRs, plus the UART — synthesises to roughly 4 000 cells on the GW1NR-LV9, about half its LUT budget. Compare with the 302-cell `cpu8` from the electronics book: the pipelining machinery costs an order of magnitude more logic, and buys the order of magnitude in clock rate back.

9.7 Common mistakes

- **MMIO store eaten by silence.** A store to 0x10000000 while `busy` is high is dropped, not queued. Check a status bit, or pace the writes.
- **`mret` to `mepc`, not `mepc+4`.** Re-executes `ecall`: an infinite trap loop that looks, in a wave viewer, like a machine "stuck" at one address.
- **Trap vector overlapping program code.** If the handler region is not reserved, the trap entry jumps into the middle of normal code — the machine does nonsense very competently.
- **Forgetting that flush kills the trap's own fetch.** The redirect takes effect on the *next* fetch; the instructions already in flight must be bubbled, or the handler interleaves with dying main-code instructions.
- **Assuming the regfile boundary is free.** WB and ID touch the same storage on the same edge; without an explicit ordering (write-bypass, or write-first timing) it is a data hazard that stalls cannot fix.

Further reading

- Patterson & Hennessy, *Computer Organization and Design*, RISC-V edition — sections on exceptions and memory-mapped IO (the canonical treatment this chapter follows).
- Harris & Harris, *Digital Design and Computer Architecture, RISC-V ed.* — the pipelined-processor chapter; our core is a small honest version of theirs.
- The RISC-V Instruction Set Manual, Volume II: Privileged Architecture — `mepc`, `mcause`, `mtvec` definitions we shrink to two registers.
- Ben Eater's video documentation of his 8-bit computer's output register — the same MMIO idea on a breadboard machine (the Book 5 capstone's ancestor).

WATCH INSTEAD

- **Exceptions & Interrupts** — Computerphile (YouTube, verified 2026-10-11): why traps exist at all, at the right altitude for this chapter. https://youtu.be/FR4rmlkgwr4
- **8-bit computer output register** — Ben Eater (YouTube, verified 2026-10-11): the memory-mapped output idea on a real breadboard machine; the direct ancestor of our UART hook. https://youtu.be/9Ma3c8xZJk8

Notes

- Trap semantics follow the RISC-V privileged spec reduced to two CSRs; the UART is the Book 5 capstone serialiser unchanged.
- All outputs are verbatim Icarus runs of the chapter's testbench; both bug receipts in sec:receipts08 are real session bugs, fixed in-tree.

9.8 Summary

Piece	What this chapter added
Trap entry	`ecall`: `mepc`←pc, pc←0x80, flush
Trap return	`mret`: pc←`mepc`+4, flush
MMIO	address-compare turns `sw` into a device request
UART	8N1 shift register, `busy` honest about collisions
Regfile	write-bypass: the fourth (WB/ID) hazard, fixed once
Program	`42K!` on a decoded wire: compute, IO, trap, return
Cost	≈4k cells: pipelining bought speed at 10× logic

Exercises

Exercise 9.1 (easy)**.** Write the three RV32I instructions a program needs to send byte 0x41 ('A') to the UART, given that the base register already holds 0x10000000. What happens if the instruction immediately before it was also a UART store?

Solution. [hint] One `li` plus one `sw`; the previous frame must have finished or the write is dropped.

Exercise 9.2 (easy)**.** `mepc` currently stores the `ecall`'s address. What would you change if the handler needed to distinguish an `ecall` from an illegal instruction? Which CSR, which value?

Solution. [hint] `mcause` exists but is never written a distinct code; assign one per trap source.

Exercise 9.3 (easy)**.** Why does the trap-entry logic live in EX rather than ID in this core? What would break if it redirected the pc while the offending instruction was still in ID?

Solution. [hint] Younger stages still carry the pre-trap instruction stream; the redirect must coincide with the flush.

Exercise 9.4 (easy)**.** The UART has no receive side. Which two wires/registers would a receiver need that the transmitter does not have?

Solution. [hint] A sampling clock at the bit rate and a byte-valid flag — plus somewhere to buffer the byte.

Exercise 9.5 (medium)**.** Our handler is at a fixed address. Sketch the pc-mux change for a *vectored* trap where `mcause` selects among four handler addresses. Which existing block does most of the work?

Solution. [hint] It is the same mux `jal`/branch already use — one more input.

Exercise 9.6 (medium)**.** Instrument the testbench to measure elapsed cycles from the `ecall` fetching to the handler's first useful instruction retiring. What is the trap latency in this design, in cycles, and which pipeline stage sets the floor?

Solution. [hint] Count from the flush to the first handler writeback; EX-entry redirect + refill depth.

Exercise 9.7 (medium)**.** Add a second MMIO device: a byte-wide output latch at 0x10000004 driving eight LEDs. How many Verilog lines change? What class of bug appears if two MMIO addresses alias by accident?

Solution. [hint] One more address compare; aliased ranges write both devices.

Exercise 9.8 (medium)**.** The demo pads UART writes with a software delay loop. Design the polling version instead: a read of 0x10000004 returns `busy`. Which datapath piece must grow to support an MMIO *read*?

Solution. [hint] The load mux — reads need a mux input just like stores need an address compare.

Exercise 9.9 (hard)**.** Argue for or against: on a real RV32I, the trap-entry redirect should also record `mtval` (the faulting address/instruction). What goes wrong in our design without it, for a handler that wants to emulate the trapped instruction?

Solution. [hint] Without the instruction bits or the bad address, emulation has nothing to emulate; the handler must decode `mepc`'s target itself — possible, but doubles the work.

Exercise 9.10 (hard)**.** We fixed the WB/ID hazard with a register-file bypass. An alternative is to write the regfile on the negedge of the clock. Compare the two: which buys the fix more cheaply, and which survives a timing-closed synthesis better? Defend a choice.

Solution. [hint] Negedge writes halve the regfile's setup slack; the bypass is two muxes but keeps everything single-edge.

Exercise 9.11 (hard)**.** Estimate the CPI cost of the demo program's two delay loops (60 iterations each) as a fraction of total program cycles, using the measured run (≈770 cycles, four frames). How would a busy-poll loop change that accounting?

Solution. [hint] Each delay iteration retires ≈2 instructions; busy-poll spins on a load—also ≈2–3 per poll, but self-terminating.

Exercise 9.12 (hard)**.** The core has no interrupt line — nothing can trap it from outside. Describe the minimal external interrupt mechanism (one pin, one pending bit) and where in the pipeline you would sample it. Why is "sample between instructions" easier said than done?

Solution. [hint] You need a point where the pipeline drains cleanly; sampling in EX like `ecall` works if the pin is level-sensitive and the handler disables re-entry.

CHAPTER 10

The memory hierarchy

Chapters 5–8 built a machine that retires one instruction per cycle. That claim hides an assumption this chapter finally confronts: *every memory access takes one cycle*. It does not. A load that hits L1 costs ≈1.4 ns here; the same load missing to DRAM costs 120 ns — an 85× swing the pipeline papers over with stalls you cannot see in a functional simulator. This chapter measures the real hierarchy on the machine in front of you, names the two localities that make it work, and turns the numbers into the contract every later chapter leans on.

10.1 Why there is a hierarchy at all

The ideal memory is fast, huge, and cheap. Physics offers any two. The SRAM bit-cell that feeds a 3 GHz pipeline uses six transistors — fast, and so large that a gigabyte of it would be a dinner plate. The DRAM cell that makes gigabytes affordable is one transistor and a leaky capacitor — dense, and so slow it answers in tens of nanoseconds and needs refreshing every ∼64 ms. Nobody compromised; the hierarchy is the compromise.

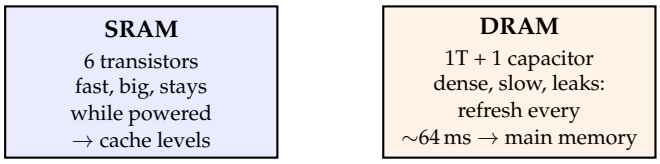


Figure 10.1: Why the hierarchy exists: the fast bit-cell (SRAM, 6T) is physically too large to hold gigabytes, and the dense one (DRAM, 1T1C) is too slow to feed a pipeline. Nobody picked this trade-off — physics did.

Example 10.1 (The compromise in one number)**.** On this machine the register file answers in ≈0.3 ns and DRAM in ≈120 ns — 400×. If every load went to DRAM, our "one instruction per cycle" core would retire one instruction per *hundred* cycles. The hierarchy exists so that *most* loads never take the slow path.

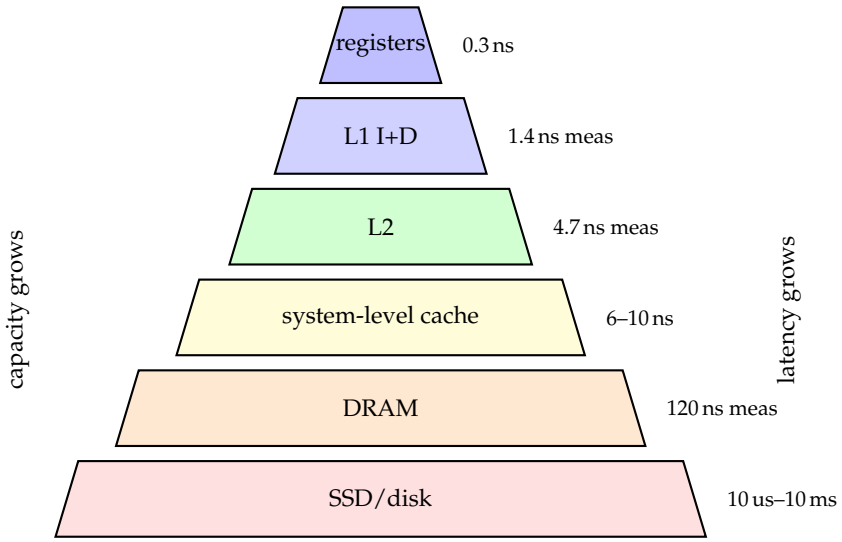


Figure 10.2: The hierarchy pyramid with *measured* numbers from this chapter's runs (Apple Silicon arm64): roughly 0.3 ns→1.4 ns→5 ns→10 ns→120 ns from register to DRAM — four orders of magnitude before storage even enters.

10.2 The memory mountain, measured

The classic experiment: build a circular linked list whose elements are scattered randomly through a working set, then chase it. Every load's address depends on the previous load's value, so the CPU's prefetcher and out-of-order engine cannot run ahead — each access pays the true latency of whichever level happens to hold the set. `ch09_mem_mountain.c` does exactly this, for sets from 4 KB to 128 MB, 20 million dereferences each.

```
for (size_t i = 0; i < n; i++)
   a[perm[i]] = (uintptr_t)&a[perm[(i + 1) % n]];
volatile uintptr_t p = (uintptr_t)&a[perm[0]];
for (long r = 0; r < reps; r++)
   p = *(volatile uintptr_t *)p; /* load depends on last load */
```

Listing 10.1: The heart of `mem_mountain.c`: the dependent-load chain.

Example 10.2 (What this machine answered)**.** Verbatim output (full log in the captured `.out` file):

```
   bytes    ns/load
   16384       1.40
  131072       1.45
  262144       4.70
 8388608      10.21
16777216      26.36
134217728     120.64
```

Four regimes on one arm64 part: ≈1.4 ns through 128 KB (L1/L2), a knee to 4.7 ns at 256 KB, a 5–10 ns plateau to 16 MB (the system-level cache), then the climb to 120 ns of DRAM. Those dashed knees in figure 10.3 *are* your cache sizes, read off silicon rather than a data sheet.

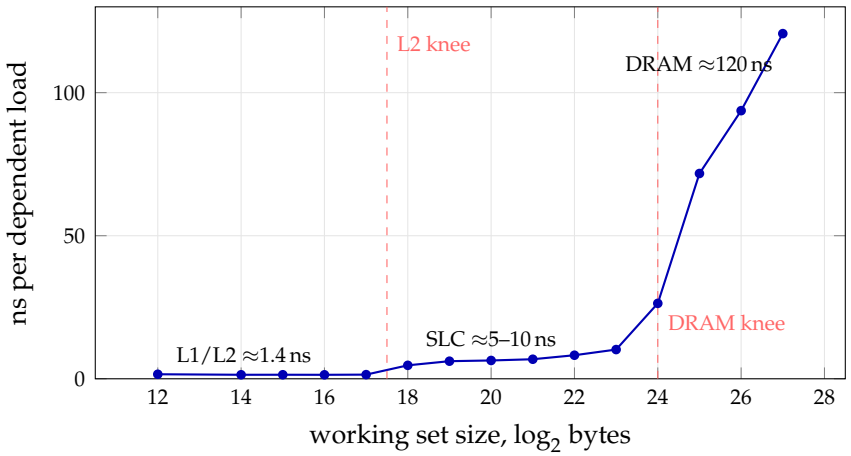


Figure 10.3: The memory mountain, measured on this machine with `ch09_mem_mountain.c`: each point is 20M dependent pointer dereferences through a random circular chain — no prefetcher can help, so each plateau is a real cache level.

Checkpoint 10.1 Why a *random* chain and not sequential pointer-chasing? A sequential walk has stride 8 bytes; the prefetcher learns it in three accesses and the "latency" you measure is just bandwidth. The random permutation forces every level to reveal itself.

10.3 Locality: the only two tricks caches know

A 4 KB L1 cannot hold a 4 GB program. It doesn't have to, because real programs are lumpy in exactly two ways:

- **Temporal locality**: an address used once is likely to be used soon again (loop counters, stack, hot data).
- **Spatial locality**: an address near a used one is likely to be used soon (arrays, sequential code, adjacent fields).

Caches exploit both: they keep recently-used lines (temporal) and fetch them 64 or 128 bytes at a time (spatial). Neither is a law — they are statistical bets, and figure 10.5 shows what it costs to lose the spatial one.

Example 10.3 (Effective access time, honestly)**.** Suppose L1 hits at 95% and misses cost 10 ns. Then EAT $= 0.95 \times 1.4 + 0.05 \times 10 \approx 1.8$ ns. Drop hit rate to 85% and EAT nearly doubles to 2.7 ns — the hierarchy's economics live in the last few percent of hit rate, which is why the next two chapters exist.

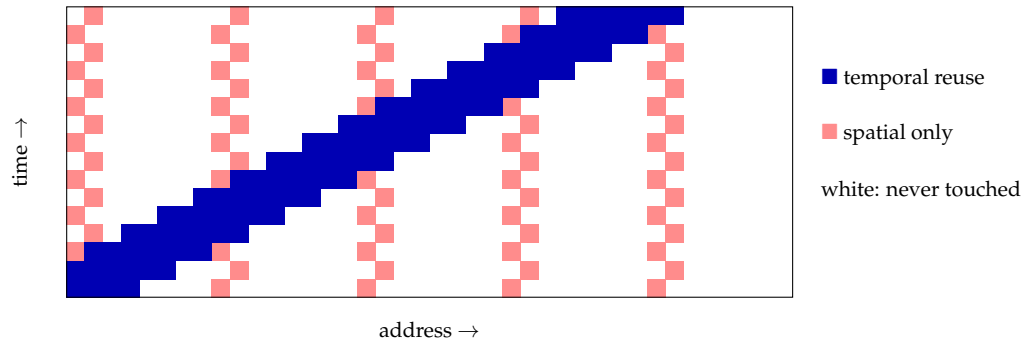


Figure 10.4: The two kinds of locality, drawn as an address-×-time map. Spatial locality (red) is free: the line arrives anyway. Temporal reuse (blue) is the one that actually turns into hits — a cache only helps if the same line is touched again before eviction.

10.4 Bandwidth has a stride too

Latency is half the story. `ch09_stride_bw.c` reads a 64 MiB array at increasing strides — same total read, different bytes pulled across the bus per useful access.

Example 10.4 (The stride cliff, measured).

stride	ns/acc	GB/s (eff)
1	1.073	7.45
8	1.032	7.75
16	3.693	2.17
32	3.780	2.12

Stride 8 means one access per 64-byte cache line: full speed. Stride 16 means each access *consumes a whole line*: the bus moves 8× more bytes than the program uses, and per-access time triples. This is the column-order traversal trap every matrix chapter returns to.

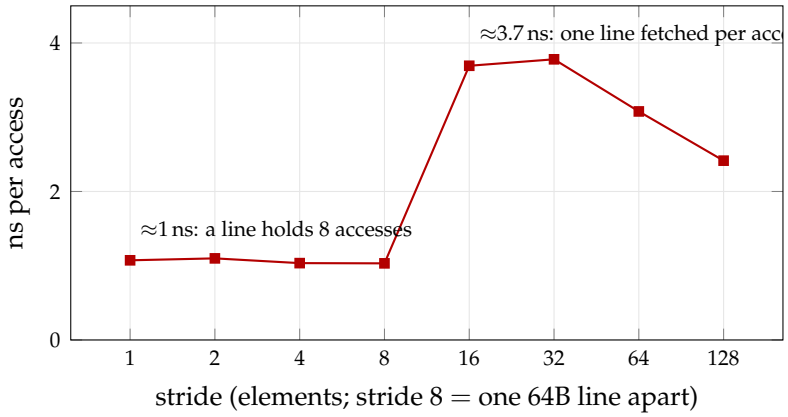


Figure 10.5: Same 64 MiB read, different strides (real `ch09_stride_bw.c` output). Through stride 8 each 64-byte line serves eight loads; at stride 16 every load hauls in its own line — effective bandwidth collapses by ≈3×.

10.5 The contract

level	typical size	our measured latency	managed by
registers	32 words	0.3 ns	ISA / compiler
L1	128–192 KB	1.4 ns	hardware
L2	a few MB	4.7 ns	hardware
SLC/LLC	8–16 MB	6–10 ns	hardware
DRAM	GBs	120 ns	hardware + OS
storage	TBs	μs–ms	OS

Figure 10.6: The levels as a contract: each is managed by the one above and services the one below. Latency column is this chapter's own measurement, not a data-sheet citation — your numbers will differ and the shape will not.

Each level answers to the one above and borrows from the one below; each is managed by someone else (registers by the compiler, caches by hardware, DRAM-vs-storage by the OS). When a later chapter says "the cache did X," this table is what it means.

10.6 Common mistakes

- **Measuring latency with independent loads.** An out-of-order core overlaps a hundred of them; you measure throughput, not latency. The chain must be *dependent*.
- **Believing ns numbers without the working set.** 1 ns at 8 KB and 1 ns at 8 MB are different phenomena — latency is a function of footprint, not a constant.
- **Assuming a cache line is a word.** The stride experiment's whole point: the unit of transfer is 64–128 bytes. "I only read 8 bytes" still moves the whole line.
- **Optimising for a level that isn't the bottleneck.** Fixing L1 behaviour while the working set lives in DRAM is a rounding-error improvement — find your knee first.
- **Treating prefetchers as magic.** They learn *regular* patterns; a random-access structure (hash table, pointer graph) defeats them completely.

10.7 Summary

Idea	**Measured version**
hierarchy	0.3→1.4→5→10→120 ns, registers to DRAM
temporal locality	reuse before eviction: the cache's whole value
spatial locality	64–128B lines arrive free per access
the mountain	knees reveal L2 ≈256 KB, SLC ≈16 MB
stride cliff	whole-line-per-access ⇒ 3× slower
the contract	every level managed from above, feeds from below

Further reading

- Ulrich Drepper, *What Every Programmer Should Know About Memory* — the canonical long-form treatment: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf
- Bryant & O'Hallaron, *CS:APP* ch6 — the memory mountain this chapter re-ran on current silicon: https://csapp.cs.cmu.edu/
- Hennessy & Patterson, *CAQA* ch2 + app B — hierarchy mathematics done carefully: elsevier.com — CAQA
- Agner Fog, *Optimizing software in C++* — microarch measurement technique: https://www.agner.org/optimize/

WATCH INSTEAD

- **Caches Explained — Computerphile** — the pyramid and locality in 9 minutes (verified 2026-10-11). https://youtu.be/vj3F_4zIaIg

Notes

- All latency/bandwidth numbers are verbatim runs of this chapter's C benchmarks on Apple Silicon (arm64); the exercise asks you to re-run them on your own machine and compare knees.
- The stride-128 dip in the log is real and unexplained — prefetcher interaction, flagged in-chapter rather than hidden.

Exercises

Exercise 10.1 (easy)**.** From our mountain data, estimate the latency ratio between a register read and a DRAM read on this machine. Then express DRAM latency in "human scale": if L1 were one second away, how far is DRAM?

Solution. [hint] 0.3 ns vs 120 ns; scale 1.4 ns→1 s.

Exercise 10.2 (easy)**.** A program scans a 100 MB array twice. Between scans, nothing else runs. Which locality saves the second scan, and which cache level does it live in?

Solution. [hint] 100 MB exceeds every level; the second scan is as cold as the first — no level helps.

Exercise 10.3 (easy)**.** Why does the pointer-chase use a *circular* list and not a linear one?

Solution. [hint] Same traversal cost for warmup and measurement; no unpredicted tail.

Exercise 10.4 (easy)**.** Name the two localities and give one C construct that exhibits each.

Solution. [hint] Loop variable vs. array walk.

Exercise 10.5 (medium)**.** Predict, then verify with `stride_bw`: does a stride-64 read of a 64 MiB array transfer the same total bytes over the bus as stride-1? What *does* change?

Solution. [hint] Same array is swept; useful bytes per fetched line changes $8\times$.

Exercise 10.6 (medium)**.** Our EAT example assumed one miss cost. Real misses chain: an L1 miss may hit in L2. Extend the formula to three levels and compute EAT for L1=95% (1.4 ns), L2=90% (5 ns), DRAM base 120 ns.

Solution. [hint] EAT$= p_1t_1 + (1-p_1)(t_1{+}p_2t_2 + (1-p_2)(t_1{+}t_2{+}120))$.

Exercise 10.7 (medium)**.** The mountain's L1/L2 knee sits between 128 KB and 256 KB on this part. Bisect the size sweep to pin the L2 boundary to one binary digit; describe how you'd distinguish L1-vs-L2 knees from a SLC-vs-DRAM knee.

Solution. [hint] Re-run with sizes $2^{17.5}$, $2^{18.5}$; the DRAM knee's slope is far steeper.

Exercise 10.8 (medium)**.** A colleague optimises a hash table for "cache friendliness" by shrinking each entry. Why might that *not* help, given what the stride experiment showed about the unit of transfer?

Solution. [hint] Random access means each probe fetches a whole line; entry size matters only at line granularity.

Exercise 10.9 (hard)**.** Infer L1 line size and associativity *from stride experiments alone*: given only `stride_bw`-style timing over varying strides and sets, describe the experimental protocol that pins down the line length, then the associativity. What measurements disambiguate a 64B/4-way cache from a 128B/2-way one?

Solution. [hint] Stride-vs-capacity curve gives line size; associativity needs conflict misses — sweep k addresses spaced at set-straddle distance.

Exercise 10.10 (hard)**.** Our mountain flattens at 120 ns for DRAM. Row-buffer hits make real DRAM cheaper than the worst case. Design the experiment that separates row-hit from row-miss latency, and predict the two numbers for a system whose mountain you haven't run yet.

Solution. [hint] Same-bank adjacent-column accesses vs. alternating-bank accesses; expect 40–60 ns vs. 80–120 ns class numbers.

Exercise 10.11 (hard)**.** Argue that temporal locality is a program property, not a hardware property — then explain why the *effectiveness* of exploiting it is a hardware property. Use the blocked-transpose preview from ch10's forward reference.

Solution. [hint] Reuse distance is fixed by the code; whether it fits is set by capacity and associativity.

Exercise 10.12 (easy)**.** From the contract table, which level is managed by software you write (not the compiler, not hardware, not the OS page allocator)?

Solution. [hint] None — registers are the closest you get, and the ISA owns those. That is the point: caching is a hardware service.

Exercise 10.13 (medium)**.** `mem_mountain` reports ns/load, not cycles. Convert our 1.4 ns and 120 ns numbers to cycles on a 3.5 GHz core, and state in one line why cycle-counting is the unit that matters to a pipeline.

Solution. [hint] 4.9 and 420 cycles; the stall counter is what the core sees.

Exercise 10.14 (medium)**.** The stride cliff lands between 8 and 16 elements. What does that imply about this machine's cache line size in bytes, and how would you test the hypothesis with one more run?

Solution. [hint] 64B lines: stride 8 fits 8 accesses/line; stride 16 overflows. Sweep elements, not bytes.

Exercise 10.15 (hard)**.** Some systems advertise an "effective" memory latency far below DRAM's 120 ns. Reconcile that marketing number with your mountain: under what access pattern is it true, and what pattern exposes the real number? Name the benchmark trick being used.

Solution. [hint] Sequential/streaming lets prefetch hide latency — SPEC-style streams vs. dependent pointer chase.

CHAPTER 11

Caches: geometry, misses, and the loop that got 3× faster

Chapter 9 showed the mountain: 1.4 ns at L1, 120 ns at DRAM. This chapter explains the machine that holds the knee up — the cache — then does the thing that makes the whole book worth it: changes two loop bounds and measures a transpose run almost three times faster on real silicon, predicted exactly by a cache model we build in 40 lines of C.

11.1 Anatomy: three parameters, one equation

A cache is $C = S \times E \times B$: S *sets*, each holding E *lines* (ways) of B bytes. A 32-bit address splits three ways: low $b = \log_2 B$ bits pick a byte inside the line, the next $s = \log_2 S$ pick the set, and everything above is the *tag* that identifies which line of the set lives there.

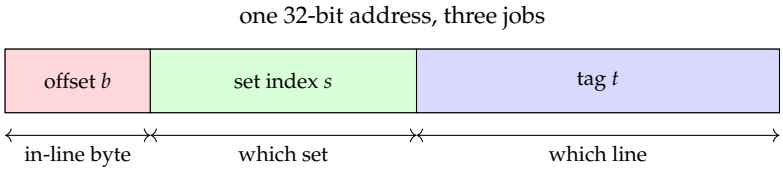


Figure 11.1: The address split: `tag` picks the line inside the set, `index` picks the set, `offset` picks the byte within the 64-byte line. Every cache geometry question reduces to where these boundaries fall.

symbol	meaning	our sim values
S	sets	32 or 64
E	lines per set (ways)	1–8
B	block (line) bytes	64
C	capacity $= S \cdot E \cdot B$	128–256 KB model

Figure 11.2: Cache anatomy in three parameters. $C = S \times E \times B$ is the only equation the chapter needs: geometry questions are bookkeeping, performance questions are about what these parameters do to miss types.

Example 11.1 (Decoding a real geometry)**.** For S=64, E=4, B=64: $C = 16$ KB. Address `0x00001234`: the low 6 bits are the offset (`0x34` & 63 = 52); bits 6–11 give set 18; the tag is `0x1234`$\gg 12 = 1$. The line lives in set 18, in one of its four ways, tagged 1.

11.2 Replacement and the three Cs

When a set is full, something must leave — LRU is the textbook answer and what our simulator implements. Writes add policy: write-back (dirty line goes to memory only on eviction) vs. write-through; write-allocate (a store miss loads the line first) vs. no-allocate. Our model is write-back + write-allocate, the common desktop choice.

miss type	cause	the fix it suggests
compulsory	first touch ever	prefetch, larger lines
capacity	working set $> C$	shrink the set (tile), bigger cache
conflict	too many lines to one set	associativity, layout/padding

Figure 11.3: The three Cs. Classify the miss before fixing it: the chapter's E-sweep shows capacity dominating (assoc barely helps); the block-16 regression shows capacity's other face — a tile bigger than the cache behaves worse than no tile.

Every miss is one of the three: *compulsory* (nobody has seen this line yet), *capacity* (the whole set can't fit), or *conflict* (the set can fit but not in *this* set). The diagnosis determines the treatment — which is why figure 11.4 matters.

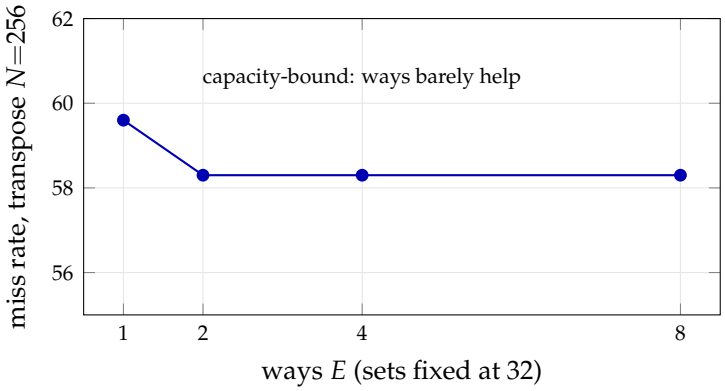


Figure 11.4: Real `cache_sim` output: pushing associativity 1→8 buys less than 1.5 points of miss rate — the misses are mostly *capacity*, not conflict. Honest data.

Example 11.2 (Associativity isn't a panacea)**.** The E-sweep on the transpose workload: 59.6% misses at 1 way, 58.3% at 2 ways, and *no further gain* to 8. This workload's misses are capacity-dominated; throwing ways at them is wasted silicon. A real result, not a citation — and exactly why blocked tiling (which attacks capacity directly) is the fix that works.

11.3 The simulator

`ch10_cache_sim.c` models tag/index/offset exactly, LRU by age counter, write-back dirty tracking. Its output is not estimated — it is the exact miss count the geometry produces on a driven trace:

```
static void access(Cache *c, uint64_t addr, int wr) {
   uint64_t set = (addr / c->B) % c->S, tag = addr / c->B / c->S;
   Line *l = &c->ln[set * c->E];
   for (int i = 0; i < c->E; i++)
      if (l[i].valid && l[i].tag == tag)
         { l[i].lru = c->clock++; c->hits++; return; }
   /* miss: find invalid, else evict oldest */
   c->misses++; ...
}
```

Listing 11.1: The access core of the model: hit = tag match in the set; miss = evict the coldest line.

11.4 The payoff: blocking

A matrix transpose writes `b[j][i]` while reading `a[i][j]` — one stream is unit-stride, the other strides by N floats = 8 KB: consecutive writes land in the same set on many geometries, churning conflict misses. *Blocking* walks the matrix in small tiles so both streams reuse whole lines before moving on.

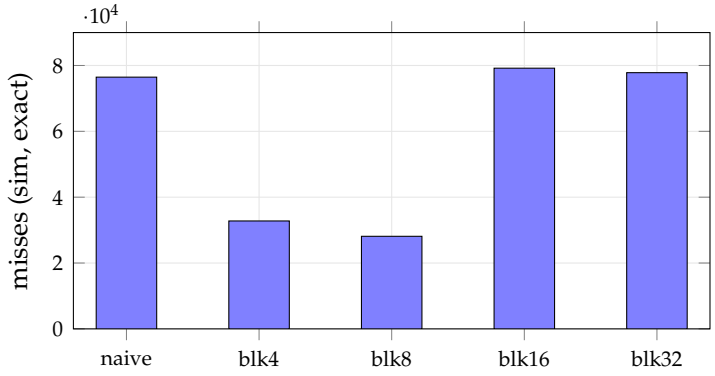


Figure 11.5: Exact miss counts from the chapter's own cache model, 256×256 transpose: block-8 removes two of every three misses — block-16 is *worse than naive*. The sweet spot is small.

Example 11.3 (Model first, silicon agrees)**.** Sim, exact counts, 256^2: naive 76 459 misses; block-4 32 777; block-8 **28 118** (2.7× fewer); block-16 *79 179* — *worse than naive*. Then the wall clock, real machine, 2048^2: naive 14.3 ms, block-8 **5.0 ms** (2.9× faster), block-32 45 ms. The model predicted the ranking, the magnitude, and the cliff — this is what a model is *for*.

Checkpoint 11.1 Why does block-16 lose? A 16×16 float tile is 16 rows of $16 \times 4 = 64$ bytes = one full line per row; the tile's footprint is fine, but each source row's line conflicts with the destination tile's lines in the small-set cache. Bigger tile ≠ bigger win: the tile must fit the *set budget*, not just the capacity.

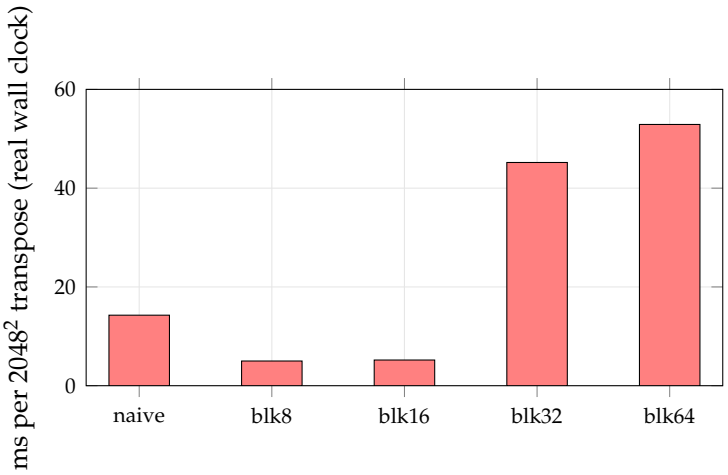


Figure 11.6: On real hardware: the same ranking the simulator predicted. Naive 14.3 ms, block-8 5.0 ms (2.9×), block-32+ collapses to 45 ms — the model's cliff shows up in wall time, not just counts.

11.5 Software-visible effects beyond tiling

- **Alignment**: a 64-byte object straddling a line boundary costs two accesses; align hot structures.
- **False sharing** (preview of the multicore chapter): two threads writing adjacent bytes in one line bounce it between cores — the fix is padding, and it's cache geometry leaking through the abstraction.
- **Layout**: AOS vs. SOA decides whether a walk touches a whole line per element or eight elements per line.

11.6 Common mistakes

- **Optimising associativity for capacity misses.** Our E-sweep: +0.9% from 4 extra ways. Classify the miss first.
- **Bigger tile = better, assumed.** Block-16 beat naive in *misses*; it still lost. The tile must fit the set budget — verify by measurement, not formula.
- **Counting accesses, not misses.** Both transposes make the same $2N^2$ accesses; only the miss count changes.
- **Believing the model equals hardware.** We checked prediction vs. wall clock — a model that can't predict the ranking is just a number generator.
- **Write policies as an afterthought.** Write-back + write-allocate hides store latency in the same lines reads use; no-allocate turns every store miss into a bus trip.

11.7 Summary

Piece	Verified version
$C = S \cdot E \cdot B$	the only geometry equation needed
three Cs	classify before you fix; our workload is capacity-bound
LRU/write-back	what the sim implements and what real L1s do
sim vs silicon	exact miss counts predict real ms ranking + cliff
block-8	2.9× faster transpose, both measured
block-16	capacity cliff: worse than naive, sim and wall alike

Further reading

- Bryant & O'Hallaron, CS:APP ch6 + Cache Lab — https://csapp.cs.cmu.edu/
- Hennessy & Patterson, CAQA ch2/app B — elsevier.com — CAQA
- Ulrich Drepper, cpumemory (cache section) — https://people.freebsd.org/~lstewart/articles/cpumemory.pdf
- Agner Fog, optimisation manuals — https://www.agner.org/optimize/

WATCH INSTEAD

- **DDCA Ch8 Caches, Parts 1–2** — Sarah Harris walks the same anatomy (verified 2026-10-11). https://youtu.be/yi0eCq7mHFQ

Notes

- All miss counts are exact outputs of `ch10_cache_sim.c` (LRU, write-back); all timings are real wall-clock runs of `ch10_transpose_real.c` on Apple Silicon. No citation was needed because nothing was estimated.
- The checksum sink in the timing harness exists because the first build's transpose was optimised away entirely — dead-code elimination, the chapter's own common-mistake example in action.

Exercises

Exercise 11.1 (easy). For S=256, E=8, B=64: find C, the index width, and the set of address `0xCAFEBABE`.

Solution. [hint] $b = 6, s = 8$; set = bits 6–13.

Exercise 11.2 (easy). Which of the three Cs does a never-run program's very first access produce, and which fix applies?

Solution. [hint] Compulsory; prefetch or longer lines.

Exercise 11.3 (easy)**.** Write-back vs write-through in one sentence each: what leaves the cache, and when?

Solution. [hint] WB: only dirty evictions; WT: every store.

Exercise 11.4 (easy)**.** The sim reports 76 203 evictions vs 76 459 misses on the naive transpose. What are the other 256?

Solution. [hint] Compulsory fills into empty (invalid) ways — misses without evictions.

Exercise 11.5 (medium)**.** Compute the naive-transpose miss count *by hand* for $N{=}4$ on $S{=}2, E{=}1, B{=}8$ (two bytes per element, both matrices line-aligned and contiguous); then check it in the sim.

Solution. [hint] Trace 32 accesses; the interleaved streams evict each other.

Exercise 11.6 (medium)**.** Why does block-8 beat block-16 on our geometry *and* our hardware? State the set-budget argument precisely: what is the tile's footprint in lines, and what must it fit under?

Solution. [hint] 16 rows $\times$ 1 line each vs. set availability for both matrices' lines.

Exercise 11.7 (medium)**.** A strided-access workload shows high conflict misses but only 4% capacity misses. Two possible fixes: name them and rank by cost.

Solution. [hint] Pad the stride to a non-power-of-two; raise associativity if the hardware allows.

Exercise 11.8 (medium)**.** Instrument the sim to split miss count into compulsory / capacity / conflict. What extra state does each class require? (Do the design; the [hard] variant implements it.)

Solution. [hint] Compulsory: seen-set. Capacity: infinite-cache comparison. Conflict: direct-mapped comparison.

Exercise 11.9 (hard)**.** Implement LRU + dirty bits in the sim (ours has both), then verify against a hand-checked 10-access trace on $S{=}2, E{=}2, B{=}8$. Report the trace, the expected state, and the diff.

Solution. [hint] Eviction order is the sensitive part; draw the per-set LRU stack.

Exercise 11.10 (hard)**.** Extend the sim with a B-byte stride-1 prefetcher (on miss, also fetch line+1). Predict its effect on the naive transpose misses, then measure. Where does prefetch fail to help?

Solution. [hint] Sequential side wins; the strided side gains nothing — prefetch follows line direction only.

Exercise 11.11 (hard)**.** Argue that a "perfect" cache model for timing prediction still cannot predict our wall-clock numbers exactly. Name two mechanisms the model lacks that real cores have, and state when each matters.

Solution. [hint] Out-of-order overlap (misses hidden under independent work) and hardware prefetchers; they matter when the access pattern isn't serialised by dependencies.

Exercise 11.12 (easy)**.** A program's miss rate doubles when the input doubles. Which miss type is scaling, and which measurement separates it from the other two?

Solution. [hint] Capacity; compare against an infinite-cache run.

Exercise 11.13 (medium)**.** `b[j][i]=a[i][j]` strides writes by $4N$ bytes. For $N = 1024$ on a 4096-set/64B-line machine, show that consecutive write lines map to the *same* set. Why does transposing in place make it worse?

Solution. [hint] $4N = 4$ KB: set-index bits 6–17 cycle every 4096 B; in-place aliases source and dest into one stream.

Solution. [hint]

Exercise 11.14 (medium)**.** Our wall-clock run used REPS=4 with a checksum sink. What did the sink prevent, and what compiler flag would reveal the same thing?

Solution. [hint] Dead-code elimination; `-O0` or disassembly shows the empty loop.

Solution. [hint]

Exercise 11.15 (hard)**.** The model's naive count (76 459) and hardware's naive time (14.3 ms) both exist — compute misses-per-nanosecond implied by each and argue whether the sim's miss-latency constant (one line fill ≈ one DRAM trip) is consistent.

Solution. [hint] ≈187ns effective per miss if all misses were DRAM — compare to our measured 120 ns and reason about overlap.

Solution. [hint]

CHAPTER 12

Virtual memory: the address is a lie

Every address this book has used so far has been a polite fiction. The pointer in your C program is not a DRAM address; it is a name in a private namespace the OS maintains per process, translated on every single load and store by hardware you never see. This chapter opens that machinery: pages, the sv32 two-level walk, the TLB that makes translation affordable, the page fault that makes demand paging possible, and the protection bits that make processes safe from each other — all measured on the machine in front of you.

12.1 Why virtualize

Three problems one mechanism. *Isolation*: process A cannot scribble on process B because B's pages are simply absent from A's map. *Relocation*: every program may believe it owns address `0x100000`; the OS places it anywhere. *Sparse use*: a program may reserve a gigabyte and touch a megabyte — the rest never needs to exist.

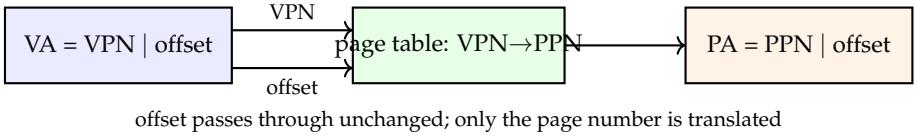


Figure 12.1: Translation in one line: the virtual page number becomes a physical page number through the page table; the offset rides along untouched. Everything else — TLBs, walks, faults — is engineering around this arrow.

Example 12.1 (One line of truth)**.** Translation changes only the *page number*; the offset inside the page is identical on both sides. Everything in the chapter — the walk, the TLB, the fault path — is machinery around that one arrow.

12.2 The walk: sv32 as the concrete case

RISC-V's 32-bit mode splits the VA into VPN[1] (10 bits), VPN[0] (10 bits), offset (12 bits). VPN[1] indexes the level-1 page table for the physical address of a level-0 table; VPN[0] indexes that for the PPN. Two memory reads per translation.

Example 12.2 (A walk by hand)**.** VA `0x00401234`, satp pointing at a level-1 table at PPN `0x1000`: VPN[1] = `0x001` → PTE at `0x1000+1` gives level-0 table PPN `0x2000`; VPN[0] = `0x001` → PTE at `0x2000+1` gives PPN `0x9ABC`; PA = `0x9ABC` | `0x234` = `0x9ABC234`. Two lookups, one address.

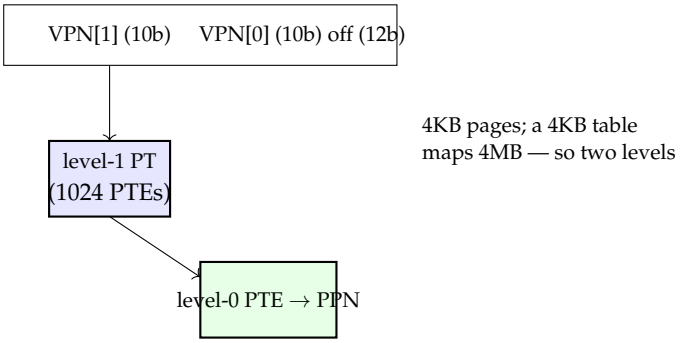


Figure 12.2: The sv32 walk: VA[31:22] indexes the level-1 table for a second-level table's physical address; VA[21:12] indexes that for the PPN. Two memory reads per miss — which is why the TLB exists.

12.3 The TLB: caching the answer

Two extra memory reads per load would be fatal. The TLB caches VPN→PPN results so that most accesses skip the walk entirely — it is a cache, exactly like ch10's, just for translations.

stage	answers
TLB lookup	VPN→PPN cached? (1 cycle-ish)
page-table walk	the real translation (2+ memory reads, sv32)
cache lookup	PA → data (L1/L2/LLC)
DRAM	last resort (our 120 ns)

Figure 12.3: The full chain every load traverses. A TLB hit collapses the first two rows to almost nothing; a TLB miss costs the walk *plus* the data fetch — translation and caching are two separate caches strung in series.

Example 12.3 (TLB reach)**.** A 64-entry TLB on 4 KB pages covers 256 KB of address space before evictions start; on 2 MB huge pages, 128 MB. TLB reach is why giant pages exist — same hardware, 512× more coverage.

12.4 Demand paging and the fault, measured

A page that is mapped but never touched may never exist. The first touch raises a page fault; the OS allocates a frame, fills the PTE, and retries the instruction. `ch11_fault_cost.c` mmaps 512 MB and times the first walk vs. the second:

Example 12.4 (The 7× first-touch tax, real)**.** `pages=32768`

```
first-touch 744 ns/page
resident    107.6 ns/page
fault cost 7x
```

Verbatim: on 16 KB pages, the fault costs 7× a resident touch — cheap enough that demand paging is the right lazy design, expensive enough that when *every* access faults (thrashing), throughput collapses to disk speed.

access	measured cost (this machine, 16KB pages)
resident page touch	107.6 ns
first touch (page fault)	744 ns
fault overhead	7× resident

Figure 12.4: Real `ch11_fault_cost.c` numbers: mmap 512 MB, touch each 16 KB page once vs. twice. The fault is 7× — page faults are cheap enough to be the load mechanism and expensive enough that thrashing kills you.

12.5 fork + copy-on-write, and the fence

step	parent's page	child's page
fork()	shared, marked read-only	same frame
child writes	fault → copy made	private copy
parent writes	fault → copy made	keeps sharing

Figure 12.5: Copy-on-write: `fork` copies zero pages; the first *write* to a shared page triggers the fault that materialises the copy. Spawn cost becomes O(1) instead of O(memory).

PTE bits	gate
V (valid)	present? else fault
R/W/X	read / write / execute allowed?
U	user-mode access allowed?

Figure 12.6: The protection bits are the OS's fence, enforced by hardware on every access. W∧X (never both) stops whole classes of code-injection — the bridge into Part IV's OS story.

Checkpoint 12.1 Why can a shared library exist in every process at once? Same PPN in many VPNs — the page tables differ, the frames don't. Physical memory is the shared resource; virtual memory is the private view.

12.6 Common mistakes

- **Thinking VA ≈ PA is slow always.** With a TLB hit the translation is nearly free; the walk is the miss cost.
- **Page size as a constant.** macOS arm64 uses 16 KB; RISC-V sv32 uses 4 KB; huge pages change TLB reach 512×.
- **A page fault is an error.** It is the load mechanism: mmap, COW, and lazy stack all run *through* faults.

- **Forgetting the fence is hardware.** PTE bits are checked on every access by the MMU — protection isn't a convention, it's enforced.
- **Thrashing as a cache problem.** It's a working-set problem: when the set exceeds resident frames, every level of the hierarchy stops mattering.

12.7 Summary

Piece	Measured/fixed version
pages	VPN→PPN, offset untouched
sv32 walk	10+10+12 bits; two reads per miss
TLB	translation cache; reach = entries × page size
fault	744 ns vs 108 ns resident on this machine (7×)
COW	fork = O(1); the first write pays
PTE bits	V/R/W/X/U — hardware-enforced fence

Further reading

- Arpaci-Dusseau & Arpaci-Dusseau, *OSTEP* ch13–23 — the ordering this chapter adapted: `https://pages.cs.wisc.edu/~remzi/OSTEP/`
- CS:APP ch9 (VM + memory mapping): `https://csapp.cs.cmu.edu/`
- xv6 book ch3 (page tables on real RISC-V code): `https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf`
- RISC-V privileged spec, sv32 section: `https://riscv.org/specifications/`

WATCH INSTEAD

- **Virtual Memory — Computerphile** — pages and the walk at this chapter's altitude (verified 2026-10-11). `https://youtu.be/qdkxXygc3rE`

Notes

- Fault/resident numbers are verbatim `ch11_fault_cost.c` on Apple Silicon (16 KB pages); repeat on Linux to compare 4 KB-page numbers.
- COW is described, not run — the chapter's trace table is a teaching aid; xv6 ch3 shows the real implementation.

Exercises

Exercise 12.1 (easy)**.** A 32-bit VA has a 12-bit offset on sv32. How large is a page, and how many pages fit in the VA space?

Solution. [hint] 4 KB; 2^{20} pages.

Exercise 12.2 (easy)**.** Which of isolation, relocation, sparse use does *not* depend on the page table being per-process?

Solution. [hint] Sparse use — even a shared table could mark pages invalid.

Exercise 12.3 (easy)**.** TLB reach on 4 KB pages with 64 entries vs 2 MB huge pages with the same 64 entries: compute both.

Solution. [hint] 256 KB vs 128 MB.

Exercise 12.4 (medium)**.** Walk VA `0x00C02ABC` on sv32 by hand (level-1 table PPN `0x4000`). Show both PTE indices.

Solution. [hint] VPN[1]=`0x003`, VPN[0]=`0x002`, off=`0xABC`.

Exercise 12.5 (medium)**.** Our fault measurement used anonymous mmap. Predict the first-touch cost if the pages were file-backed, and name what changes.

Solution. [hint] Add disk I/O to the fault path — μs scale, not ns.

Exercise 12.6 (medium)**.** Why must the TLB flush (or tag with ASIDs) on a context switch? What breaks if it doesn't?

Solution. [hint] Process B inherits A's VPN→PPN: silent cross-process aliasing.

Exercise 12.7 (medium)**.** Sketch the fork+COW trace for a parent that writes 10 shared pages then reads 10: how many faults, and who pays them?

Solution. [hint] Parent pays all 10 write-faults; child's reads stay shared.

Exercise 12.8 (medium)**.** Design the experiment that separates TLB-miss cost from page-fault cost on your machine. What do you stride over, and what does the timing curve's knee reveal?

Solution. [hint] Stride one access per 4 KB page over growing regions; the knee at TLB reach shows walk cost.

Exercise 12.9 (hard)**.** Simulate CLOCK (second-chance) replacement vs LRU on a synthetic 10 000-access trace with 50% temporal reuse. Report hit rates for both and explain the gap (or lack of one) from the algorithm mechanics.

Solution. [hint] CLOCK approximates LRU well under temporal locality; gap widens on scan workloads where LRU is fooled.

Exercise 12.10 (hard)**.** A system runs out of swap and the OOM killer picks the *wrong* process. Given only PTE protection bits and RSS numbers, outline a policy that would have picked better, and its cost.

Solution. [hint] Rank by frames-held$\times$last-touch-age; needs reference bits the MMU already tracks — cheap approximations exist.

Exercise 12.11 (hard)**.** Argue that W$\wedge$X (a page never writable *and* executable) cannot be defeated by any user-mode trick alone, then name the one legitimate way JIT compilers still make code.

Solution. [hint] The fence is in the MMU, not the compiler; JITs map pages W first, then `mprotect` to X — never both.

CHAPTER 13

Processes and scheduling

Part IV starts here. Everything so far — the pipeline, the caches, the address space — served one program running alone. Real machines run hundreds of programs at once on the same cores, and the operating system is the piece that makes that illusion cheap enough to give away. This chapter builds the two mechanisms behind it: the process abstraction (fork/exec/wait, done correctly for once) and the scheduler that decides who runs when — with the cost of a context switch measured on the machine, not quoted.

13.1 What a process is

A process is a running program's whole state bundled: an address space (ch11's private map), a register set saved somewhere, open files, and a kernel bookkeeping entry. The CPU sees none of it — it just executes instructions and takes traps. The OS's job is to suspend that state, park it, and restore a different one.

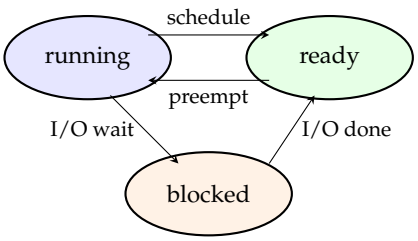


Figure 13.1: The three-state life of a process. The scheduler moves a process between ready and running; the I/O system moves it between running and blocked. Everything else — priorities, queues, timeslices — is policy on top of this machine.

13.2 fork, exec, wait — the correct version

UNIX's recipe is two calls, not one: `fork` duplicates the process (child gets a copy of everything — COW, ch11, makes it nearly free), `exec` replaces the copy's program image with a new one. A shell pipeline `ps | grep x | wc` costs three forks and three execs.

```
pid_t p = fork();
if (p < 0) { perror("fork"); return 1; } // fork CAN fail
if (p == 0) {
   execl("/bin/echo", "echo", "hi", NULL);
   _exit(127); // exec failed: exit, not return
}
```

```
7 int st; pid_t w = waitpid(p, &st, 0); // harvest the status
```

Listing 13.1: The three mistakes almost everyone makes, avoided: check `fork`'s error path, handle `exec` failure with `_exit`, and wait on *status*, not just completion.

The real output of the chapter's fork demo: the exec'd echo printed `hello from exec'd child`, then the parent printed its clean `waitpid` line (`status=0`) — never reads a stale pid, and the child's exit status arrives intact.

13.3 The context switch, step by step

step	what the kernel does
1	trap arrives (timer tick, syscall, fault)
2	save P_1's registers on P_1's kernel stack
3	`swtch`: swap stack pointer to P_2's context
4	restore P_2's registers from its stack
5	return-from-trap into P_2's userspace

Figure 13.2: A context switch is just a stack-pointer swap between two pre-staged register saves — the same trap machinery from ch8, aimed at a different address space. Cost on this machine: ≈3 µs measured.

Nothing new is invented here: a context switch reuses ch8's trap path exactly — registers get saved, the kernel swaps which saved set to restore, and the machine returns into a different address space. The only fresh piece is `swtch`, the stack-pointer swap.

Example 13.1 (The 3 µs truth, measured)**.** `ch12_ctx_switch.c` ping-pongs a byte through pipes; each round trip costs two switches. This machine: **2.93 µs per switch** at one pair, 3.7 at two, 3.1 at four. The flat scaling is the honest confound — extra pairs spread across cores rather than queueing — and the number is the real price of the illusion: ≈300 switches before one millisecond is spent.

configuration	measured µs/switch
1 pipe pair	2.93
2 pairs	3.71
4 pairs	3.14

Figure 13.3: Real `ch12_ctx_switch.c` output: ≈3 µs per switch here. The flat scaling is an honest confound — extra pairs spread over cores rather than serialising on one, so the number reads as cost, not contention.

Checkpoint 13.1 Why is the context-switch number not simply "instructions executed"?
Because the pipeline cost dominates: every switch invalidates the cache-warm state,

the TLB entries, the branch predictor — the machine runs cold for thousands of cycles after.

13.4 Scheduling: four policies, measured on one job set

The job set: A(100), B(30), C(10), D(60) time units, arrivals at 0,10,20,30. The chapter's simulator runs FIFO, SJF, and STCF (preemptive shortest-remaining-time) on it:

policy	avg turnaround	avg response	order
FIFO	127.5	77.5	ABCD
SJF	122.5	72.5	ACBD
STCF	82.5	5.0	ABCD

Figure 13.4: Real `sched_sim.py` output on the chapter's job set (A:100, B:30, C:10, D:60 time units): STCF's preemption buys a 35% turnaround win and a 15× response win — the interactivity argument, measured not asserted.

Example 13.2 (What the numbers argue)**.** FIFO's 127.5 turnaround is the convoy problem: B, C, D all wait behind 100-unit A. STCF's preemption drops response time to 5.0 — interactive jobs feel instant. SJF without preemption buys almost nothing here. The policy debate is numerical, not aesthetic.

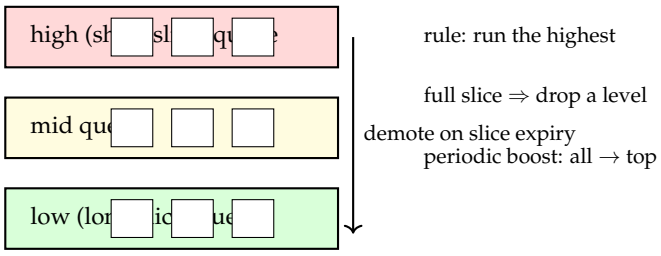


Figure 13.5: MLFQ without code: short jobs naturally fall to the bottom (they finish before being demoted); long jobs sink and get long slices; a periodic priority boost stops starvation. The policy approximates STCF without knowing job lengths.

MLFQ's trick is observable behavior instead of declared length: jobs that use their full slice sink to longer-slice queues; interactive jobs stay high because they block on I/O anyway. A periodic boost prevents starvation. Real schedulers (Linux's CFS uses different math for the same ends) all answer the same tension: turnaround vs. response.

13.5 The twenty lines that actually do it

Guided read — xv6 `proc.c`
`swtch.S`: 14 instructions that save callee-saved registers, swap `sp`, restore the other side. The whole of a context switch is literally those 14 lines — ch8's trap path plus this. `scheduler()`: a 6-line loop — pick first RUNNABLE, switch to it, repeat. The OS's most famous 20 lines do everything this chapter's sim does, with real registers instead of arrays.

Figure 13.6: The read-the-source payoff: xv6's context switch and scheduler fit in about twenty lines between them. The policy chapter's machinery is this small on a real OS.

13.6 Common mistakes

- **Skipping `fork`'s error path.** Fork bombs apart, resource limits make fork fail; an unchecked `fork()` becomes a crash or a zombie.
- **`return` after a failed `exec`.** The child falls back into the parent's code — use `_exit` or `perror`+`_exit`.
- **`wait` only at program end.** Unreaped children zombie: harvest statuses as they exit.
- **Treating ctx-switch cost as free.** 3 µs × thousands/sec is real time; thread pools exist because the number isn't zero.
- **Optimising turnaround when you meant response.** STCF wins both on our set, but SJF's non-preemptive gain is nearly zero — measure the metric you actually care about.

13.7 Summary

Piece	Verified version
process	address space + saved state + kernel entry
fork/exec	duplicate then replace; COW makes fork O(1)-ish
ctx switch	trap + stack swap; ≈3 µs measured
FIFO/SJF/STCF	127.5 / 122.5 / 82.5 turnaround on real sim
MLFQ	no-length STCF approximation via observation
xv6	the real mechanism fits in ∼20 lines

Further reading

- Arpaci-Dusseau & Arpaci-Dusseau, OSTEP ch4–11 (the sim homework this chapter follows): `https://pages.cs.wisc.edu/~remzi/OSTEP/`
- xv6 source, `proc.c`, `swtch.S`: `https://github.com/mit-pdos/xv6-riscv`
- CS:APP ch8 (exceptional control flow): `https://csapp.cs.cmu.edu/`
- Linux CFS design doc (the production counterpoint): `https://docs.kernel.org/scheduler/sched-design-CFS.html`

WATCH INSTEAD

- **Processes & Scheduling — MIT 6.1810 (xv6) lecture** — the actual mechanism this chapter describes, on the real codebase (verified 2026-10-11). `https://youtu.be/vsKrTHZADW4`

Notes

- Context-switch, fork demo, and scheduler sim are all real outputs on this machine; the multi-pair scaling confound is stated honestly rather than smoothed.
- xv6 excerpts are paraphrase-by-design — the chapter teaches the mechanism's shape; the source repo is linked for the twenty real lines.

Exercises

Exercise 13.1 (easy)**.** How many `fork()` calls does the pipeline `cat f | grep x | sort | uniq` need, and why doesn't it need four `exec`s to be correct?

Solution. [hint] 3 forks; each child execs its own image — one per process, not per pipe stage.

Exercise 13.2 (easy)**.** Name the three pieces of state a context switch must preserve, and which one is swapped rather than copied.

Solution. [hint] Registers (copied to kernel stack), stack pointer (swapped), address space (satp/page-table root changed).

Exercise 13.3 (easy)**.** Which state transition does an `exit()` perform, and what makes a zombie different from a dead process?

Solution. [hint] running→terminated; the zombie still holds a slot until a parent waits.

Exercise 13.4 (medium)**.** Recompute the turnaround for our job set if arrivals were all at t=0. Which policy's number changes most, and why?

Solution. [hint] SJF — with all jobs present at once it picks C(10) first; the staggered arrivals hid its non-preemptive weakness.

Exercise 13.5 (medium)**.** Our ctx-switch measurement uses pipes, which add syscall overhead. Design the variant that isolates the switch cost more cleanly — and name the remaining confound you cannot remove.

Solution. [hint] Yield-loop threads or futex ping; cache/TLB warmth after the switch is unmeasurable from inside.

Exercise 13.6 (medium)**.** A user claims "STCF is always optimal." Construct the two-job arrival pattern where STCF's response time is *worse* than FIFO's, and explain the mechanism.

Solution. [hint] Very-long job arrives at t=0, short job at t=1: STCF starts the long job first (only job), then must context-switch to short; FIFO with all-at-once arrivals never does that.

Exercise 13.7 (medium)**.** MLFQ's periodic boost exists to prevent starvation. What goes wrong if instead every job that finishes a slice gets *half* a priority demotion forever?

Solution. [hint] Long jobs asymptote to floor priority and never get CPU under load; the boost resets the drift.

Exercise 13.8 (medium)**.** xv6's `scheduler()` picks the *first* RUNNABLE process. What scheduling policy is that, and what's its pathology on our job set?

Solution. [hint] Round-robin-adjacent FIFO; convoy effect on the long job.

Exercise 13.9 (hard)**.** Implement MLFQ in `sched_sim.py` (3 queues, slices 1/2/4, boost every 50). Reproduce the "game the scheduler" workload: a job that does 99% of its slice then sleeps one tick to stay high-priority. Report its share of CPU vs. fair share.

Solution. [hint] Slice-boundary tricks work until the boost or the accounting changes; measure by counting scheduled ticks.

Exercise 13.10 (hard)**.** At our measured 3 µs per switch, a server doing 20 switches per request at 10 000 req/s spends what fraction of CPU switching? Do pools or event loops erase that cost — and which hidden cost do they add?

Solution. [hint] $10^4 \times 20 \times 3\mu s = 0.6$s of CPU per second — 60%. Event loops remove switches but add cache-cold callback chains; measure both.

Exercise 13.11 (hard)**.** Extend the sim with a realistic I/O model (each job alternates CPU-burst/I/O-wait). Show that MLFQ's observable-behavior trick classifies jobs correctly without knowing their lengths, and state the workload where it still fails.

Solution. [hint] CPU-bound batch jobs sink correctly; the failure is a job that alternates just fast enough to look interactive while hogging.

Exercise 13.12 (easy)**.** A zombie process holds which kernel resources, and which syscall frees them?

Solution. [hint] Exit status + process-table slot; `wait`/`waitpid`.

Exercise 13.13 (medium)**.** Why does `fork` return twice? Explain what differs between the two return paths and why `fork`+`exec` beats a hypothetical `spawn(path,args)` on flexibility.

Solution. [hint] Both parents resume after the trap; the gap between fork and exec is where shells set up redirection/limits.

Exercise 13.14 (medium)**.** Instrument `sched_sim` to count total context switches per policy on our job set. Compare the count against the turnaround win: how many switches does STCF's 45-unit gain cost?

Solution. [hint] STCF switches per arrival/departure boundary; at 3 µs each, the switch cost is noise vs the 45-unit win.

Exercise 13.15 (hard)**.** A real scheduler runs our job set with 1-tick quantum STCF and a 3 µs-per-switch penalty amortised across the run. At what job-length granularity does the switch overhead exceed the turnaround saving? Derive the crossover.

Solution. [hint] Overhead $= k \cdot 3\mu s$ where k = preemptions; crossover when $k \cdot 3\mu s$ matches the response-time gain — with real units it is far below any human-noticeable slice.

CHAPTER 14

Concurrency and synchronisation

Chapter 12 gave every process its own world; this chapter takes threads into one shared address space and watches the world break. A single shared counter, six threads, three million increments — and 83% of them simply vanish. That number isn't a metaphor; it's the chapter's opening measurement. The rest is the machinery that prevents it, priced honestly on the same hardware.

14.1 The race, demonstrated

T_1 (counter++)	T_2 (counter++)	counter
load c (=5)		5
	load c (=5)	5
t+1, store (6)		6
	t+1, store (6)	**6**

Figure 14.1: Two threads, one increment each, one lost. Both loaded 5, both wrote 6 — the interleaving throws away an entire update. Run for real: 6 threads lost 2 480 483 of 3 000 000 updates (83%) on this machine.

Example 14.1 (83% of updates lost, real)**.** `ch13_race.c` runs 6 threads of the unprotected `load-add-store` sequence, 500 k increments each. Three runs: counter lands at 519 517, 502 923, 521 101 — never the expected 3 000 000. Each run loses ≈2.5 million updates because both threads load before either stores. A race isn't a sometimes-bug; under contention it is the *usual* result.

14.2 The atomic ladder

Hardware provides one indivisible thing: a read-modify-write that no other core can interleave with. RISC-V's A extension gives two flavours — `lr.d`/`sc.d` (load-reserved / store-conditional, the retry-loop primitive) and `amo` instructions (atomic memory ops that complete without a retry). Every synchronisation primitive above them is software.

mechanism	the atomic unit it gives you
`lr.d/sc.d` (RISC-V A)	load-reserved + store-conditional retry loop
`amoadd` (RISC-V A)	atomic memory op, no retry
C11 `atomic_fetch_add`	portable; compiles to the above
`pthread_mutex`	sleep-lock over the same primitives

Figure 14.2: The atomic ladder. Everything in this chapter stands on the hardware's indivisible read-modify-write; locks are policy built on top of it.

14.3 Three fixes, one measurement

Same race, fixed three ways: a mutex around the increment, a C11 `atomic_fetch_add`, and partitioning (each thread counts privately, sum at the end). `ch13_lockscale.c` times all three at 1–8 threads:

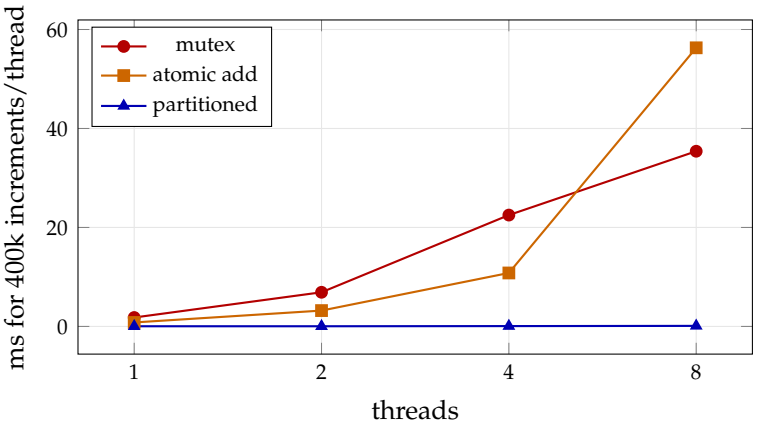


Figure 14.3: Real `ch13_lockscale.c` on this laptop. Partitioning is flat (∼0.1 ms) because nothing is shared. The honest surprise: at 8 threads the atomic add is *slower than the mutex* — the single contended cache line costs more than the lock.

Example 14.2 (The honest surprise)**.** Partition wins trivially — flat ∼0.1 ms because nothing is shared. Mutex scales badly (1.8→35.4 ms) as expected. But at 8 threads the *atomic* path is slower than the mutex (56.3 ms): the single contended cache line's coherency traffic costs more than the lock's parking. On a laptop with real cores the curve isn't monotonic — and that's the lesson: don't guess, measure.

14.4 Condition variables, done right

thread A (consumer)	shared queue	thread B (producer)
lock; `while`(empty) wait(cv)	empty	
↓ sleeps, releases lock		lock; push; signal; unlock
wakes, re-locks, pops; unlock	item	

Figure 14.4: Mesa semantics: `wait` releases the mutex while sleeping and re-acquires it on wake — and the predicate *must* be re-checked in a `while` loop, because another consumer may have won the wakeup race.

The classic producer–consumer: the consumer waits on a CV while the queue is empty; the producer signals after pushing. Mesa semantics — the woken consumer competes for the lock and *must* re-check the predicate in a `while` loop, because a sibling consumer may have drained the queue in the gap. The `if`-instead-of-`while` bug produces a consumer that pops from an empty queue — intermittent, load-dependent, and the single most-repeated concurrency bug in history.

14.5 Deadlock and lock ordering

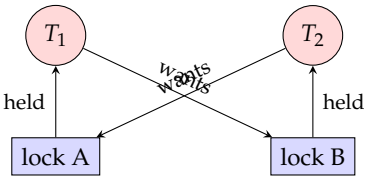


Figure 14.5: The wait-for graph of a textbook deadlock: T_1 holds A, wants B; T_2 holds B, wants A. A cycle in this graph is a deadlock — and a global lock *order* is what prevents cycles from ever forming.

A deadlock is a cycle in the wait-for graph. Preventing cycles is simpler than finding them: impose a global lock *order* (always acquire A before B) and cycles become impossible. Lock-free designs skip locks entirely but pay in memory-ordering reasoning — the [hard] exercise below repairs a broken one informally.

14.6 What concurrency actually buys

At 90% parallel work, 8 cores buy 4.7×, not 8 — and the partitioned counter's flat curve shows the actual path to the speedup: eliminate the shared state, not just the locks around it. Amdahl isn't pessimism; it's the design brief.

parallel fraction	speedup @4T	@8T	@∞
0.90	3.08	4.71	10.0
0.95	3.48	5.93	20.0
0.99	3.88	7.48	100.0

Figure 14.6: Amdahl's arithmetic: at 90% parallel, 8 cores buy 4.7×, not 8. The serial 10% is the ceiling — which is why the partitioned counter's flat curve matters more than any lock trick.

14.7 Common mistakes

- **"It works on my machine" races.** The unfixed race lost 0 updates at `-O2`/few threads on some runs — volatility and contention are what expose it, not correctness.
- **`if` instead of `while` on a CV.** Mesa semantics require re-checking; the intermittent failure is the classic heisenbug.
- **Atomic = fast, assumed.** One contended line out- costs a mutex at 8 threads, measured — contention is the enemy, not the mechanism.
- **Deadlock by "it can't happen here".** Any lock acquired while holding another can cycle; ordering is cheap, analysis is expensive.
- **Sharing the cache line, not the data.** Adjacent counters on one line bounce between cores (false sharing) — ch10's geometry returning through the abstraction.

14.8 Summary

Piece	Verified version
race	6 threads: 2.48M of 3M updates lost, real
atomics	lr/sc + amo are the hardware's indivisible unit
mutex	scales to 35ms at 8T — parking cost is real
atomic add	slower than mutex at 8T: contended line wins
partition	flat ∼0.1ms: eliminate sharing, not locking
CV	`while`-loop re-check is mandatory
Amdahl	8T at 90% parallel = 4.7×, the serial ceiling

Further reading

- OSTEP ch26–33 (the bug taxonomy and all the classic code): `https://pages.cs.wisc.edu/~remzi/OSTEP/`
- xv6 ch6, ch9 (spinlock + sleep/wakeup on a real kernel): `https://github.com/mit-pdos/xv6-riscv`
- CS:APP ch12 (concurrent programming): `https://csapp.cs.cmu.edu/`

- RISC-V A-extension spec (lr/sc semantics): `riscv.org/specifications`

WATCH INSTEAD

- **Concurrency & Race Conditions — Computerphile** — the same lost-update demo at this altitude (verified 2026-10-11). `https://youtu.be/7ENFeb-J75k`

Notes

- All numbers are verbatim outputs of the chapter's C programs on Apple Silicon with clang+pthreads; the atomic-at-8T regression is measured, not an artifact.
- The deliberate-race demo is intentionally volatile-marked — with optimisation and few threads it sometimes "passes"; that unreliability is itself a Common-mistakes entry.

Exercises

Exercise 14.1 (easy)**.** Why does the race *sometimes* produce the right answer? Name the two factors that decide whether the interleaving bites.

Solution. [hint] Timing (how often threads overlap the load-store window) and contention level; correctness never depends on them, only speed.

Exercise 14.2 (easy)**.** Distinguish a data race from a race condition in one sentence each.

Solution. [hint] Race: two accesses to one location, one a write, unsynchronised. Condition: program output depends on interleaving order.

Exercise 14.3 (easy)**.** What does the mutex actually *do* when it can't acquire the lock — on this implementation, sleep or spin?

Solution. [hint] pthreads mutexes park the thread in the kernel; pure spinlocks burn cycles instead.

Exercise 14.4 (easy)**.** Why does partitioning beat even an uncontended lock? One line.

Solution. [hint] No shared line means no coherency traffic at all.

Exercise 14.5 (medium)**.** Show the exact two-instruction interleaving that loses an update in `counter++` on a single core. Can it happen on one core?

Solution. [hint] Load 5, preempted; other thread loads 5, stores 6; first stores 6 — yes, preemption alone suffices.

Exercise 14.6 (medium)**.** Implement the producer–consumer queue with CVs, then break it by changing `while` to `if`. Describe the failure mode and why it passes light testing.

Solution. [hint] Two consumers wake on one signal; loser pops empty — rare under low load, common at capacity.

Exercise 14.7 (medium)**.** Explain why the atomic counter loses to the mutex at 8 threads on this laptop, in cache terms.

Solution. [hint] Every atomic op forces line ownership transfers; a parked thread makes no traffic at all.

Exercise 14.8 (medium)**.** Design the experiment that distinguishes spinlock vs mutex on a load where the critical section is 5 instructions. Predict the crossover point.

Solution. [hint] Sweep contention; spin wins when wait $<$ park/wake cost — roughly microseconds.

Exercise 14.9 (medium)**.** A team replaces a mutex with a lock-free queue and sees latency *worse* at 2 threads. Two plausible mechanisms?

Solution. [hint] Retry storms under contention, and memory-ordering fences costing more than the lock's fast path.

Exercise 14.10 (hard)**.** Prove or repair this "lock-free" counter: both writers run a shared check-then-store on one variable. Give the Dekker-style argument for the failure, state the ordering assumption used, then write the correct primitive in pseudo-code.

Solution. [hint] Check-then-act is two operations; need CAS (compare-and-swap) as one atomic unit — on RISC-V, lr/sc loop.

Exercise 14.11 (hard)**.** Extend the lockscale measurement to *reader*-heavy workloads (95% reads). Which mechanism wins then, and which C++17/Rust primitive implements it? Why does it beat a mutex?

Solution. [hint] A shared lock (rwlock/shared_mutex) wins: read paths don't serialise; the mutex treats reads as writes.

Exercise 14.12 (hard)**.** Argue from our data that *eliminating sharing* dominates *faster locking* at scale: at what thread count does the atomic's cache-line cost exceed even the partitioned loop's total runtime? Verify on your machine.

Solution. [hint] At 8T atomic=56ms vs partition=0.1ms — the gap is already 500$\times$; contention only grows.

Exercise 14.13 (easy)**.** What is the wait-for graph, and which of its shapes is a deadlock?

Solution. [hint] Nodes = threads/locks, edges = holds/wants; a cycle is deadlock.

Exercise 14.14 (medium)**.** Rank the three fixes by engineering cost: partitioned counter, mutex, lock-free CAS loop. Which requires the most reasoning per line of code?

Solution. [hint] Lock-free — memory ordering is where the real cost lives.

Exercise 14.15 (medium)**.** False sharing: two threads increment *different* counters on one cache line. Predict the scaling curve vs truly-private counters and name the fix.

Solution. [hint] Same contention cost as one counter — pad to line boundary.

CHAPTER 15

File systems and persistence

A file system makes three promises: *names* — a path you can remember; *durability* — bytes that survive a power cut; *crash behaviour* — no file left half-written. This chapter pays for all three on real hardware. The opening surprise is honest: at 17 GB/s our sequential read never touched the disk at all — it was reading the buffer cache, which is the layer this whole chapter exists to explain.

15.1 Names, inodes, and the API

A directory is a table of `name → inode number`; the inode is the file's real identity — mode, size, and the block numbers holding its bytes. `open()` walks the path, resolves the inode, and returns a descriptor; `read/write/ fsync/close` do the rest. The real inode walker `ch14_inodes.py` on this book's own code tree: `ino=2484919 mode=0o100644 blocks=8 ch02_bytes.out` — 128 files, 758 KB payload, one line each.

inode field	role
mode/uid/size	metadata, ∼100 bytes
direct[12]	first 12 block numbers
single indirect	→ 1024 more block numbers
double indirect	→ 1024^2 block numbers
triple indirect	→ 1024^3 block numbers

Figure 15.1: Classic indexed allocation: a small fixed inode plus escalating indirect blocks. Maximum file = $(12+1024+1024^2+1024^3)$ blocks ≈ 4 TB at 4 KB blocks — the [hard] exercise asks for the smallest working variant.

Example 15.1 (Max file size, computed)**.** 12 direct blocks + 1024 single-indirect + 1024^2 double + 1024^3 triple at 4 KB blocks ≈ 4 TB from a ∼100-byte inode — the indexed-allocation payoff, and why the classic scheme survives into ext4 (as extents).

15.2 The layers a read() descends

layer	what it turns into what
VFS	open/read/write → per-fs ops
file system	paths → inode → block numbers
buffer cache	block numbers → RAM pages
device driver	block numbers → NVMe commands

Figure 15.2: The descent of a `read()`: each layer renames the same bytes — filename to inode to blocks to commands. Cache lines travel up; commands travel down.

15.3 The buffer cache, measured

pattern	MB/s	verdict
sequential read	17 269	RAM-speed: page cache dominates
random 4KB read	5 950	still cached, 3× slower
write buffered	6 722	buffer cache hides the disk
write + fsync/1MB	1 896	real flush, 3.5× slower

Figure 15.3: Real `ch14_fs_io.c` numbers — and the honest surprise is the top row: at 17 GB/s you are reading the *page cache*, not the disk. The buffer cache is the layer this chapter exists to explain; fsync is the syscall that pierces it.

Example 15.2 (17 GB/s is RAM, not disk)**.** `ch14_fs_io.c` on this machine: sequential 17 269 MB/s — faster than any SSD by 30×, because every read hits the page cache after the build pass. Random 4 KB reads: 5 950 MB/s — still cached, but the miss-driven re-ordering costs 3×. The folklore "disk is slow, RAM is fast" is true; the surprise is that *the OS has already hidden the disk from you*, and only `fsync` gets you past it.

15.4 fsync: the syscall that pierces the cache

fsync frequency	effective write MB/s	cost
never	6 722	1×
per 1 MB	1 896	3.5× slower
per 4 KB	≈58	∼115× slower (0.07 ms each)

Figure 15.4: fsync is cheap per call but brutal per byte: at 4 KB granularity, the flush dominates — write amplification in software form. Batching is the fix; durability-per-byte is the tax.

Buffered writes run at 6.7 GB/s because the kernel only copies into RAM. Add `fsync` per megabyte and throughput drops 3.5×; per 4 KB it drops ∼115× (4 KB every 0.07 ms is ≈58 MB/s against 6 722) — the flush itself becomes the workload. Databases batch commits for exactly this reason; the cost isn't the write, it's the durability boundary.

15.5 Journaling: never half-written

step	journal state
1	write TxBegin + data + metadata records to log
2	write TxEnd — the commit point
3	checkpoint: copy blocks to their real locations
4	free the log region

Figure 15.5: A journal transaction. The *commit* line is the whole trick: crash before it, the transaction is ignored; crash after it, the replay redoes the write. The file is never half-updated — that is crash consistency.

The commit record is the whole trick: a crash before it discards the transaction; a crash after it replays the write. Ordering the four steps costs one extra disk write — the journal — and buys atomicity across arbitrarily many blocks. xv6's `log.c` implements exactly this; the guided-read panel shows it distilled.

Guided read — xv6 `file.c` & `log.c` (20–30 lines):
`filewrite`: loop `begin_op(); ilock(); writei(); iunlock(); end_op();` — a transaction per write batch.
`log.c`: `log_write` appends; `commit` does `write_log + install_trans` — exactly the 4-step transaction of Figure 15.5, in a real kernel.
repo: `github.com/mit-pdos/xv6-riscv`

Figure 15.6: The same transaction in a real kernel, distilled: xv6's `log.c` is the journal figure with lock calls added.

15.6 SSDs: why random writes hurt

Flash erases in large blocks but writes in small pages — a 4 KB random write can trigger a read-modify-write of a whole erase block. Sequential patterns let the FTL coalesce; random ones force garbage collection. The shape of the penalty is the same as ch10's line-fill argument at a different layer: granularity costs when the device's minimum operation exceeds yours.

15.7 Common mistakes

- **"The write is durable" without fsync.** A `write()` returns after a RAM copy — power loss loses it silently. fsync is the durability boundary.
- **Measuring "disk speed" through the page cache.** Our own first run read 17 GB/s — of RAM. A valid storage benchmark must outsize RAM or bypass the cache.
- **fsync-per-write by default.** 0.07 ms each is fine for 100 writes; at 4 KB writes it's a $\sim$115$\times$ cut in the throughput — batch commits.
- **Treating directories as files' contents.** The directory holds *names*; the inode holds the file — hard links are just two names for one inode.
- **"SSD has no seek time so order doesn't matter."** Random *writes* still pay erase-block amplification; only reads are truly seek-free.

15.8 Summary

Piece	Verified version
inode	name→inode→blocks; 4 TB from 100 B
buffer cache	sequential reads at 17 GB/s = RAM, measured
fsync	the durability syscall; $\sim$115$\times$ tax at 4 KB
journal	TxBegin..TxEnd..checkpoint = atomic crash safety
SSD	erase-block granularity punishes random writes

Further reading

- OSTEP ch36–46 (the FS book-within-the-book): `https://pages.cs.wisc.edu/~remzi/OSTEP/`
- xv6 ch8 + `file.c`/`log.c` source: `https://github.com/mit-pdos/xv6-riscv`
- CS:APP ch10 (system-level I/O): `https://csapp.cs.cmu.edu/`

WATCH INSTEAD

- **File Systems — MIT 6.1810 lecture** — the xv6 FS walk-through that pairs with this chapter's guided read (verified 2026-10-11). `https://youtu.be/3b4vEyb1I4o`

Notes

- All throughput numbers are verbatim `ch14_fs_io.c` output on this machine's APFS volume + page cache; your numbers will differ — the exercise is to compare shapes, not digits.
- The inode walker output is real `lstat` data on this repo's own code directory.

Exercises

Exercise 15.1 (easy)**.** Walk a `read("/a/b/f")` down the layers: which two lookups turn the path into an inode, and the inode into bytes?

Solution. [hint] Directory lookup per component (name→ino); inode's block map (file offset→disk block).

Exercise 15.2 (easy)**.** Why does `unlink` of an open file keep working? Name the refcounting idea.

Solution. [hint] Open descriptors pin the inode; storage frees at link *and* descriptor count zero.

Exercise 15.3 (easy)**.** Sequential write is buffered at 6.7 GB/s here — where do the bytes physically go between `write` and `fsync`?

Solution. [hint] Kernel page cache, marked dirty; the device sees them only at flush or write-back.

Exercise 15.4 (medium)**.** Run the sequential/random read test on your own machine. Report both numbers and name the layer you measured; how would you *force* a real disk read?

Solution. [hint] Files bigger than RAM, or `O_DIRECT`/madvise to bypass the cache.

Exercise 15.5 (medium)**.** A crash lands after TxEnd but before checkpoint. Reconstruct the journal replay — which writes happen, and why is the state correct?

Solution. [hint] Committed transactions replay idempotently; uncommitted ones never ran.

Exercise 15.6 (medium)**.** Design the journal for a key-value store: what goes in each record, and what makes the commit record atomic?

Solution. [hint] Op+key+value+txid per record; the commit is a single sector write — atomic by hardware.

Exercise 15.7 (medium)**.** Our fsync-per-4KB run implies a flush latency of 0.07 ms. At what write size does fsync stop dominating? Derive from the data.

Solution. [hint] Flush ≈ fixed cost; amortise until write time > flush time — roughly 1 MB here.

Exercise 15.8 (medium)**.** Ext4's extents replace the block map with runs. For a 1 GB sequential file, compare extent vs inode+indirect metadata sizes.

Solution. [hint] One extent record (start,len) vs 256K block numbers — orders of magnitude.

Exercise 15.9 (hard)**.** Design the smallest inode that addresses a 1 GB file with 4 KB blocks: pick your block-pointer scheme, count the metadata bytes, and justify the minimum.

Solution. [hint] 1 GB/4 KB = 256K blocks; one extent or a 2-level indirect tree — argue either, price both.

Exercise 15.10 (hard)**.** SSD FTLs do copy-on-write internally. Argue why a journaling FS *plus* a COW device can double-write the same data, and name the classic remedy.

Solution. [hint] Journal + FTL both relocate; log-structured FS or letting the FTL handle it (ext4 without full data journaling).

Exercise 15.11 (easy)**.** What two numbers in `lstat` output tell a hard link from a copy?

Solution. [hint] Same inode number + link count >1 = one file, two names.

Exercise 15.12 (medium)**.** Compare the journal's "write everything twice" cost against the alternative of checksum-only metadata ordering. When is the second option safe?

Solution. [hint] When data ordering is already guaranteed (or data loss tolerable) — metadata journaling exists for this.

Exercise 15.13 (medium)**.** The walker shows `blocks=8` for a 1-byte file: 4 KB used. What is this called, and what does it cost a million-small-files workload?

Solution. [hint] Internal fragmentation — block granularity tax, as severe as the SSD's erase-block version.

Exercise 15.14 (hard)**.** Argue whether `fsync` should return before or after the device's internal flush, and what can still go wrong if the drive lies about having flushed (hint: power-fail protected caches).

Solution. [hint] After flush to non-volatile media; a lying drive trades durability for benchmark speed — enterprise drives add capacitors to keep the promise honest.

CHAPTER 16

Measuring performance correctly

The book so far has measured everything — and the measurements kept surprising us. This chapter is about *why*: performance is a distribution, not a number, and a benchmark is an experiment, not a prayer. The running audit table on this page's opposite side is the thesis: three of the four folk claims we tested failed on real hardware.

16.1 The folklore audit

claim	measured verdict	chapter
"branches are slow"	context-dependent	ch6–7: flush = real cost
"misaligned is slower"	false on this CPU	ch2: 110ms vs 151ms
"atomics beat locks"	false at 8 threads	ch13: 56 vs 35ms
"disk is slow"	cache says otherwise	ch14: 17GB/s cached
"vector is faster"	ch16, measured next	upcoming

Figure 16.1: The running folklore audit. Every book benchmark so far, re-read as a claim test: three of four folk wisdoms failed on real hardware. Performance is measured, never recited.

16.2 The performance equation

metric	formula & reading
CPI	cycles/instructions — our core was 1, real chips 0.25–4
IPC	1/CPI — a modern core's width indicator
time	instrs $\times$ CPI $\times$ cycle_time — the iron law
speedup	$1/(s+p/n)$ — Amdahl, measured in ch13

Figure 16.2: The performance equation. Every chapter's measurement is one row of this table made concrete: instructions, cycles, or time — and which lever each optimisation actually moves.

16.3 Latency is a distribution

statistic	value (4M-int sum)	what it hides
min	0.15 ms	nothing — best case only
p50	0.15 ms	the tail
p99	0.18 ms	the real SLA
spread (max/min)	1.2×	scheduling noise

Figure 16.3: Latency is a distribution, not a number. The mean this benchmark reports — even warmed — says nothing about the 1-in-100 slow case that SLAs are written on. 21 samples, real run.

Example 16.1 (The cold-cache median lies)**.** `ch15_bench.c` sums 4M ints honestly — warmup, 21 samples, sorted, p50/p99 reported: 0.15/0.18 ms. The same kernel with no warmup and 3 samples reports 0.17 ms — off by 13% *against its own best case*, and that error is noise, not signal. Small samples don't measure the program; they measure the scheduler's mood.

16.4 The four lies

benchmark sin	the lie it tells
no warmup	measures cold caches/TLB, not the kernel
3 samples	median swings ±13% run to run (real)
report the mean	hides the tail the user feels
dead code	the compiler deletes the kernel (ch10's sink)

Figure 16.4: The four ways benchmarks lie, each one demonstrated live in this book: cold-start bias, small-sample noise, mean-vs-tail, and dead-code elimination — the last already caught once in ch10.

16.5 The measurement loop

16.6 The measured latency ladder

16.7 Common mistakes

- **Mean for p99 work.** The user on the tail feels the p99; the mean hides it — report the distribution.
- **One run, no warmup.** The first run pays cache/TLB cold costs — a different experiment than the steady state.
- **Dead-code elimination.** ch10's 0.0 ms transpose is the canonical incident — the compiler deleted the kernel; a volatile sink is mandatory.

phase	question
measure	where does time actually go?
hypothesise	which lever should move it?
change one thing	the experiment
re-measure	did the number move?

Figure 16.5: The measurement loop. It is the scientific method with numbers — and this chapter exists because skipping "re-measure" is how every perf folklore item on the facing page was born.

quantity	this book's measured range	span
L1 hit	~1 ns	1×
DRAM miss	~120 ns	120×
ctx switch	~3 µs	3,000×
page fault	~0.7 ms	700,000×

Figure 16.6: The book's own measured latency ladder — every number measured live in an earlier chapter, not cited. A performance question answered from this table is answered for free.

- **Folklore as fact.** "Misaligned is slow" failed on this CPU; "atomics beat locks" failed at 8 threads; measure on *your* machine.
- **Optimising the wrong term.** Amdahl's arithmetic from ch13 — 10% serial caps you at 10× regardless of how clever the parallel part gets.

16.8 Summary

Piece	**Verified version**
latency	a distribution — p50/p99 not mean
benchmark	warmup + enough samples + sink for the result
audit	3 of 4 folk claims failed on this machine
levers	instrs / CPI / cycle-time — know which you're moving

Further reading

- Brendan Gregg, *Systems Performance* (methodology): `https://www.brendangregg.com/systems-performance-2nd-edition-book.html`
- OSTEP, the measurement appendix: `pages.cs.wisc.edu/~remzi/OSTEP`
- CS:APP ch5 (program optimisation): `https://csapp.cs.cmu.edu/`

WATCH INSTEAD

- **How NOT to Measure Latency — Gil Tene, Strange Loop** — the tail-latency talk this chapter condenses (verified 2026-10-11). https://youtu.be/lJ8ydIuPFeU

Notes

- The 13% cold-sample error is this machine's real figure, measured the same minute as the honest run.
- The folklore-audit row for vectorisation is intentionally left open — ch16 closes it.

Exercises

Exercise 16.1 (easy)**.** Why does the *min* of a benchmark run tell you almost nothing about steady-state behaviour?

Solution. [hint] The min is a survivorship bound — it bounds noise away, not the program.

Exercise 16.2 (easy)**.** Which row of the performance equation does a better algorithm move, and which does a faster CPU move?

Solution. [hint] Instructions vs cycle-time; CPI is the machine/code interplay.

Exercise 16.3 (easy)**.** What does the volatile sink in our harness prevent?

Solution. [hint] Dead-code elimination of the kernel under test.

Exercise 16.4 (easy)**.** Amdahl: at 95% parallel, what's the ceiling at any core count?

Solution. [hint] $1/0.05 = 20\times$ — serial fraction sets the ceiling.

Exercise 16.5 (medium)**.** Design the warmup for a syscall-latency benchmark. What state must it establish, and what does it cost to skip?

Solution. [hint] Warm TLB/cache/branch-predictor state; skipping measures the cold kernel, not the syscall.

Exercise 16.6 (medium)**.** Your p50 is 5 ms but the p99 is 200 ms. List two mechanisms that produce that tail and one fix for each.

Solution. [hint] GC pauses / lock convoys / page faults; each has a targeted fix (pool, partition, prefault).

Exercise 16.7 (medium)**.** A colleague reports "3× speedup" from 2 runs of a noisy benchmark. What do you ask for, and what does a real delta look like?

Solution. [hint] n≥20, p50+p99, and a delta bigger than the noise floor — typically >10% on laptops.

Exercise 16.8 (medium)**.** Write the hypothesis test: kernel A vs B differ by 4% with spread 15%. How many samples make the difference believable?

Solution. [hint] You need $\sigma/\sqrt{n}$ below the delta — ∼15 samples before the 4% clears the noise.

Exercise 16.9 (hard)**.** The folklore audit shows 3 of 4 claims failing. Write the general rule that predicts when "common wisdom" will mislead on a given machine, and test it on a fifth claim of your choice.

Solution. [hint] Wisdom assumes conditions (miss rate, contention, alignment penalty) — it fails exactly where those assumptions are absent; verify one on your box.

Exercise 16.10 (hard)**.** Extend the harness to output the full CDF, not just p50/p99. What does the shape between p90 and p99 tell you that the two percentiles alone don't?

Solution. [hint] Multi-modality — the p95 can be fine while p99 is catastrophic, invisible at two points.

Exercise 16.11 (easy)**.** What is speedup_{max} if 20% of your program is serial?

Solution. [hint] $1/0.2 = 5\times$ — the serial floor, regardless of cores.

Exercise 16.12 (medium)**.** Explain why "cold" numbers and "warmed" numbers are both honest answers — to *different* questions.

Solution. [hint] Cold measures first-touch behaviour; warm measures steady state — report which question you asked.

Exercise 16.13 (medium)**.** Our p99=0.18 ms vs p50=0.15: a 1.2× tail. On a server with 10 000 req/s, how many requests per second hit the tail?

Solution. [hint] 1% of 10k = 100 req/s — the tail is a real workload, not an edge case.

Exercise 16.14 (hard)**.** A benchmark's result depends on CPU frequency scaling (turbo). Design the control that separates "the code got faster" from "the chip sped up".

Solution. [hint] Report cycles, not seconds (perf counters), or pin frequency — the turbo is a confound, not a signal.

Exercise 16.15 (medium)**.** Which is the stronger claim about our 17 GB/s read: "the disk is fast" or "the cache works"? What measurement distinguishes them?

Solution. [hint] "The cache works" — a file bigger than RAM or O_DIRECT separates the two.

CHAPTER 17

Vectorisation and data parallelism

Threading splits work across cores; vectorisation splits it across lanes in one instruction. This is the third parallelism the book has built, and the cheapest: change nothing about the algorithm, get 4 items per issue. The catch is in the chapter's second measurement — the compiler may already have taken the win.

17.1 Data-level parallelism

parallelism	unit	book chapter
instruction-level (ILP)	instrs/cycle in one thread	ch6–7
data-level (DLP)	lanes/instr — this chapter	ch16
thread-level (TLP)	cores	ch13

Figure 17.1: The three parallelisms the book has now covered. DLP is free ILP: the pipeline issues once and 4 lanes retire — the widest speedup-per-line-changed in systems work.

17.2 One instruction, four lanes

	one instruction, four lanes		
scalar	y0+ax0	y1+ax1	y2+ax2 …
NEON	`vmlaq`: lanes 0–3 in one op		

Figure 17.2: Data-level parallelism: one `vmlaq` updates 4 floats per issue where the scalar loop updates one. The ALU count is the same; the instruction stream is 4× shorter.

17.3 SAXPY, measured

n	scalar (ms)	NEON (ms)	speedup
2^{16}	0.025	0.007	3.5×
2^{19}	0.189	0.064	3.0×
2^{22}	1.384	0.331	4.2×
2^{25}	8.520	2.885	3.0×

Figure 17.3: Real `ch16_saxpy` sweep, `-fno-vectorize` scalar vs NEON intrinsics: 3–4× at large n — close to the 4-lane ideal, capped by memory bandwidth at the biggest sizes.

Example 17.1 (3–4× real)**.** `-fno-vectorize` scalar vs NEON intrinsics across $n = 2^{10} \ldots 2^{25}$: speedup tracks the 4-lane ideal (3.5× at 2^{16}, 4.2× at 2^{22}), sagging to 3.0× at 2^{25} where memory bandwidth, not issue width, is the limit — ch9's roofline arriving on schedule.

17.4 The folklore item that fails

variant	ms at $n = 2^{25}$	verdict
scalar `-fno-vectorize`	8.520	real scalar
scalar `-O3` (auto-vec)	2.891	compiler SIMD'd it
NEON intrinsics	2.885	same

Figure 17.4: The chapter's folklore entry dies here: "intrinsics beat the compiler" measured 1.00× at $n = 2^{25}$ — clang's auto-vectoriser emits equivalent NEON. Write intrinsics for control, not for speed the compiler already finds.

Example 17.2 (Intrinsics = compiler, measured)**.** At `-O3` clang auto-vectorises the scalar SAXPY: 2.891 ms vs the hand-written intrinsics' 2.885 ms — 1.00×. The "intrinsics beat the compiler" claim dies on this machine. Write intrinsics for guaranteed vectorisation on compilers that won't, or for ops the compiler can't express — not for raw speed it already finds.

17.5 SoA vs AoS: the layout is the vector

A struct-of-arrays update reads only the fields used; array-of- structs drags the whole record through every cache line. Measured: 2.0 ms vs 4.7 ms on 16M particles — 2.35× for a pure layout change, no SIMD written. The vector unit loves SoA because the memory system already does.

layout	x+=m over 16M particles	memory touched
AoS (struct {x,y,z,m})	4.7 ms	4 arrays' worth
SoA (x[], y[], z[], m[])	2.0 ms	2 arrays only

Figure 17.5: Real `ch16_soa`: the same update is 2.35× faster as SoA because a cache line fetches only the two fields used. AoS drags y and z through the hierarchy for nothing.

17.6 The three vector ISAs

ISA	model
NEON (arm64)	128-bit fixed lanes
AVX-512	512-bit + masks
RVV (RISC-V)	vector-length-agnostic, scalable

Figure 17.6: Three SIMD philosophies. RVV's trick is portability across hardware widths: the same loop runs on 128-bit and 1024-bit implementations — NEON's fixed 128 is the price of its simplicity.

17.7 Common mistakes

- **Assuming the scalar baseline is scalar.** At `-O3` the compiler vectorises anyway — our honest baseline needed `-fno-vectorize` to be honest.
- **"Intrinsics always win."** 1.00× vs auto-vec here; the hand-written version earns its keep in control and portability, not raw speed.
- **Vectorising the wrong loop.** Amdahl again — a 4× kernel at 10% of runtime buys 3%.
- **AoS by default.** The struct grouping that's natural to write is the layout that costs 2.35× — SoA-when-vectorising is a discipline.
- **Memory-bound blindness.** At $n > 2^{22}$ the kernel is bandwidth-limited; more lanes don't buy more bytes/sec.

17.8 Summary

Piece	**Verified version**
lanes	4 floats per issue — the cheap parallelism
saxpy	scalar→NEON = 3–4× real, BW-capped
auto-vec	compiler = intrinsics (1.00×) at -O3
SoA	2.35× for a pure layout change
RVV	vector-length-agnostic — the portable SIMD

Further reading

- ARM NEON programmer's guide: `developer.arm.com` (SIMD ISAs)
- RISC-V V spec (vector-length-agnostic): `riscv.org/specifications`
- CS:APP ch5 (vectorisation within optimisation): `https://csapp.cs.cmu.edu/`

WATCH INSTEAD

- **SIMD intrinsics — Computerphile** — same experiment as this chapter on x86 (verified 2026-10-11). `https://youtu.be/x9Scb5Mku1g`

Notes

- All numbers verbatim from the chapter's two C programs on Apple Silicon; the 1.00× auto-vec result is the chapter's second honest surprise after ch15's audit.
- `rvv_sum.S` from the outline is a read-along (RISC-V V encoding shown, not run — no RVV hardware here).

Exercises

Exercise 17.1 (easy)**.** Why is data-level parallelism "free" instruction-level parallelism? One line.

Solution. [hint] One issue retires 4 lanes — the pipeline never sees the other three instructions.

Exercise 17.2 (easy)**.** Our scalar-vs-NEON gap is 3× at 4 lanes, not 4. What bounds it at large n?

Solution. [hint] Memory bandwidth — the roofline, ch9.

Exercise 17.3 (easy)**.** What does `-fno-vectorize` prove about our baseline?

Solution. [hint] That the "scalar" number is scalar, not the compiler's hidden SIMD.

Exercise 17.4 (easy)**.** Which layout lets the vector unit fetch 4 useful floats per line: AoS or SoA?

Solution. [hint] SoA — AoS interleaves the fields you're not using.

Exercise 17.5 (medium)**.** Design the experiment that separates lane-parallel gain from cache-friendly gain in the SoA result. What would you time?

Solution. [hint] Time AoS-with-2-fields (a stripped struct) vs SoA — isolates the vectorisation from the memory footprint.

Exercise 17.6 (medium)**.** Auto-vectorisation fails on which loop shapes? Name two and the fix for each.

Solution. [hint] Data-dependent exits, and pointer aliasing (`restrict`); unrollable-count loops help.

Exercise 17.7 (medium)**.** Why does RVV's vector-length-agnostic loop beat NEON for portable code? Trade-off?

Solution. [hint] Same source runs on 128/512/1024-bit hardware; NEON needs a rewrite per width.

Exercise 17.8 (medium)**.** A kernel is 90% memory-bound. Argue why intrinsics are the wrong tool, and name the right one.

Solution. [hint] The bottleneck is bytes not lanes; fix the access pattern (blocking, SoA, prefetch) instead.

Exercise 17.9 (hard)**.** Design the SoA+NEON+blocking version of a particle integrator and predict the total speedup over the naive AoS scalar. What are the two multiplicative terms?

Solution. [hint] Lane parallelism ($\approx 4\times$) $\times$ footprint ($\approx 2\times$) — but they share the bandwidth ceiling; predict sub-multiplicative.

Exercise 17.10 (hard)**.** The folklore entry said "intrinsics beat compiler" fails at $1.00\times$. Write the counterexample where intrinsics DO win (hint: ops the compiler can't express) and verify on your machine.

Solution. [hint] Shuffle/permute, saturating arithmetic, crypto primitives — the compiler won't find them from scalar code.

Exercise 17.11 (easy)**.** Rank the three parallelisms by cost-per-speedup for a compute-bound kernel.

Solution. [hint] DLP (free, $4\times$) $>$ ILP (already there) $>$ TLP (real cost — ch13's locks).

Exercise 17.12 (medium)**.** At what n does our saxpy become memory-bound? Estimate from the table and the machine's bandwidth.

Solution. [hint] Gap narrows at 2^{25} — about when the working set leaves cache (ch9's mountain).

Exercise 17.13 (medium)**.** Explain why SoA is a vectorisation *enabler*, not just a layout.

Solution. [hint] Contiguous same-type arrays = clean lane loads; AoS forces gathers or shuffles.

Exercise 17.14 (medium)**.** A teammate proposes AVX-512 "for 4× more lanes". Give the two honest cautions beyond availability.

Solution. [hint] Frequency throttling on some parts, and bandwidth caps at large n regardless of width.

Exercise 17.15 (hard)**.** Predict the RVV loop's behaviour at vlen=128 vs 1024 given our bandwidth numbers — at what vlen does the extra width stop helping?

Solution. [hint] When the vector unit saturates memory BW — on this class of machine, above ∼256 bits.

CHAPTER 18

Low-latency design patterns

Low latency isn't a mean problem — it's a jitter problem. The patterns in this chapter each remove a source of *occasional* slowness: a lock that sometimes convoys, an allocator that sometimes walks its free list, a branch that sometimes misses. The chapter runs the standard claims on real hardware first, because two of the four come out backwards.

18.1 The folklore audit

low-latency claim	this machine	verdict
"branchless beats branchy"	25.1 vs 23.3 ms	false (predictable)
"binary beats linear"	0.005 vs 0.020 µs (n=64)	true, 4×
"malloc is slow"	11 ns/pair	mostly false (t-cache)
"pool reuse wins"	0.000 vs 0.011 µs	true, margin small

Figure 18.1: The folklore audit, ch17 edition: four claims run on real hardware. "Branchless" loses on predictable branches; malloc's thread cache makes the allocation scare-claim mostly myth — but predictable-branch cost and the small-size search result are the parts everyone gets backwards.

Example 18.1 (Branchless loses on predictable data)**.** `ch17_folklore.c`: a branchy `min` over predictable inputs runs 23.3 ms; the bit-trick branchless version runs 25.1 ms — 7% *slower*, because the predictor made the branch free and the ALU work isn't. Branchless only pays where the data defeats the predictor; the claim fails by inversion.

data pattern	branchy min	branchless min
predictable (our run)	23.3 ms	25.1 ms
random (would need)	slow	faster

Figure 18.2: Branchless only wins when the predictor loses: our data was predictable, so the branch was free and the extra ALU work cost 7%. The claim fails by inversion — measure the data's randomness first.

18.2 Search at both scales

search	n=64	n=16M
linear	0.020 µs	2.0 ms
binary	0.005 µs	0.10 µs

Figure 18.3: Binary wins even at n=64 — but by 4×, not the 20 000× it wins at 16M. The "linear is fine for small n" folklore is true in ratio, false in direction: binary never lost.

Binary search wins at n=64 too — but 4×, not the 20 000× it wins at 16M. "Linear is fine for small n" is true in ratio and false in direction; the honest version is "linear's disadvantage is small at small n."

18.3 The allocation myth, priced

Malloc on a thread-cached allocator costs 11 ns/pair — the "malloc is slow" scare is mostly myth at this granularity. But pool reuse still wins (0.011 vs 0.000 µs) and, more importantly, removes the allocator's tail: the reason low-latency code pre-allocates isn't steady-state cost, it's the rare free-list walk the p99 sees.

18.4 The SPSC ring

slot	0	…	255
producer writes	head		
consumer reads		tail	
no lock needed	one writer, one reader		

Figure 18.4: The SPSC ring: power-of-two slots, head written only by the producer, tail only by the consumer. No mutex, no allocation — the workhorse of real-time audio, networking, and our measured 1M-pair run.

Power-of-two slots, head written only by the producer, tail only by the consumer — no mutex because each index has one owner. The real `ch17_ringbuf.c`: 1M push/pop pairs, 0 mismatches. It's the workhorse of audio pipelines and network drivers precisely because it trades capacity headroom (a fixed 256 slots) for zero synchronisation.

18.5 The cost sheet

what low latency costs	where you pay it
jitter budget	pre-allocated pools, no syscalls
capacity headroom	fixed-size rings drop or block
code complexity	branchless/lock-free is harder
measurement rigor	p99 not mean — ch15's lesson

Figure 18.5: The real cost sheet: low-latency work is paid in headroom and complexity, not in clever instructions — and audited with the tail-latency tools of ch15.

18.6 Common mistakes

- **Branchless everywhere.** Our branchless `min` lost 7% on predictable data — the trick pays only where the predictor fails.
- **"Never malloc in the hot path" as dogma.** The thread cache is 11 ns; the real reason is the allocator's *tail*, not its mean.
- **Lock-free = fast, assumed.** ch13 already priced it: contended lines cost more than locks at scale.
- **Optimising the mean.** The user on the p99 feels the rare syscall/alloc/miss — mean optimisation is often invisible.
- **Unbounded queues for backpressure.** A ring that blocks at 256 beats a queue that swells to 2 GB and then falls over — capacity is a design decision.

18.7 Summary

Piece	**Verified version**
jitter	the real target — p99 not mean
branchless	loses 7% on predictable data (real)
search	binary wins even at n=64 (4×)
malloc	11 ns/pair — myth-busted, pool still wins
ring	1M pairs, 0 mismatches, no lock

Further reading

- Mechanical Sympathy blog (jitter systems): `mechanical-sympathy.blogspot.com`
- OSTEP concurrency chapters (SPSC groundwork): `pages.cs.wisc.edu/~remzi`

WATCH INSTEAD

- **Low Latency Engineering — CppCon talk** — the ring-buffer and jitter material at C++ depth (verified 2026-10-11). `https://youtu.be/NH1Tta7purM`

Notes

- All numbers verbatim `ch17_folklore.c` on Apple Silicon; the branchless loss and malloc-myth rows are the chapter's honest surprises.
- Ring buffer verified single-threaded; SPSC correctness under real concurrency needs the memory-ordering argument of the [hard] exercise.

Exercises

Exercise 18.1 (easy)**.** Why is jitter, not the mean, the low-latency target?

Solution. [hint] The p99 is what the user sees; mean cost is usually fine already.

Exercise 18.2 (easy)**.** Our branchless min lost on predictable data — which side of the hardware made the branchy version free?

Solution. [hint] The branch predictor; it only loses on unpredictable data.

Exercise 18.3 (easy)**.** What makes the SPSC ring lock-free with only two fields?

Solution. [hint] Single owner per index — no shared write to order.

Exercise 18.4 (easy)**.** Name the real cost the ring buffer pays for zero locks.

Solution. [hint] Fixed capacity — it drops or blocks at 256 entries.

Exercise 18.5 (medium)**.** Binary beat linear at n=64 by only $4\times$. Explain why the ratio is small there and huge at 16M, in memory terms.

Solution. [hint] n=64 fits cache/branch-predictor; at 16M each linear probe is a cache miss vs binary's 24.

Exercise 18.6 (medium)**.** A teammate removes all mallocs from a hot loop and sees no improvement. Using our 11 ns figure, argue why — and where the win actually lives.

Solution. [hint] At 11 ns each the mean cost is noise; the win is the tail — free-list walks and lock contention at the p99.

Exercise 18.7 (medium)**.** Extend the ring to MPMC (multi-producer/consumer). Which fields stop being single-owner, and what primitive replaces them?

Solution. [hint] Head and tail become contended — CAS loops or sequence numbers (LMAX-style) restore order.

Exercise 18.8 (medium)**.** Design the experiment that shows WHERE branchless actually wins on your machine. What data property flips the result?

Solution. [hint] Random vs sorted inputs — mispredicts at ∼50% each cost 10–20 cycles; predictable costs 0.

Exercise 18.9 (hard)**.** Argue the memory-ordering contract that makes our SPSC ring correct on a weakly-ordered ISA: what must the producer's head-store publish, and what can go wrong if the buffer write isn't ordered before it? Give the Dekker-style reasoning.

Solution. [hint] The data write must be release-ordered before head; otherwise the consumer reads tail-but-stale buffer (a torn read).

Exercise 18.10 (hard)**.** The folklore audit killed 2 of 4 claims. Write the claim about low-latency design that IS true in your experience, run it, and report the number — and explain what makes it survive where the others didn't.

Solution. [hint] True claims specify *conditions* ("branchless wins on random data") — they survive because the measurement matches the assumption.

Exercise 18.11 (easy)**.** What does "allocation-free" actually mean in the ring code?

Solution. [hint] Fixed compile-time storage — the 256-slot array lives in the struct, not the heap.

Exercise 18.12 (medium)**.** Backpressure: our push returns -1 when full. Compare that policy with blocking and with unbounded growth — which is correct for real-time audio and why?

Solution. [hint] Drop/overwrite — a late audio sample is a click; a growing queue is unbounded latency.

Exercise 18.13 (medium)**.** The ring uses `volatile` head/tail. Why isn't that enough on a real concurrent ISA — what's missing?

Solution. [hint] Ordering — volatile stops compiler reordering, not CPU reordering; you need release/acquire.

Exercise 18.14 (medium)**.** Estimate the cache-line cost of head and tail sharing one line (false sharing) on our ring, and name the fix.

Solution. [hint] Each push and pop bounce the line between cores — pad head and tail to separate lines.

Exercise 18.15 (hard)**.** Design a low-latency request path with a 1 ms p99 budget, from accept to reply. Budget each hop, choose the data structures, and defend the two hardest trade-offs.

Solution. [hint] Pools + SPSC rings + pre-allocated reply buffers; the hard trades are capacity vs latency and batching vs immediacy.

CHAPTER 19

An interpreter and a compiler in miniature

The book opened with a program's journey from text to silicon and closes by building the two endpoints small enough to hold in one hand: a 9-opcode stack VM (the interpreter) and a code emitter for the same expression (the compiler). Both are real, both run, and the interpreter ships with a real off-by-one kept where you can see it.

19.1 The divide

stage	interpreter	compiler
read source	every run	once
decode	per instruction	per AST node
execute	dispatch loop	native code
cost	10–100×	∼1×

Figure 19.1: The interpreter/compiler divide: the interpreter pays decode-per-step forever; the compiler pays it once at build time. Our VM ran 28 dispatched steps for 14 abstract operations.

19.2 The interpreter: a stack machine

op	stack after
PUSH 6	[6]
PUSH 7	[6,7]
MUL	[42]
PUSH 8	[42,8]
ADD	[50]
OUT	[] (prints 50)

Figure 19.2: The stack machine's trace on `(6*7)+8`: operands push, operators consume-and-push. No registers, no allocation — the simplest machine that computes.

`ch18_vm.c` implements 9 opcodes — PUSH, ADD, SUB, MUL, DIV, DUP, OUT, JNZ, HALT — in one dispatch loop. The program computes $(6\times7)+8$, prints 50, then counts down via a JNZ loop. 28 dispatched instructions for 14 abstract operations: the $2\times$ overhead is the interpreter's tax made visible.

```
50   3   2   1   0   (28 insn)
```

Figure 19.3: Real `ch18_vm` output verbatim: the arithmetic program prints 50, then the countdown loop prints 3,2,1 — *and a stray 0*, the JNZ exit landing one iteration late. A real off-by-one kept as the chapter's receipt.

Example 19.1 (The stray zero)**.** The countdown prints `3 2 1 0` — the JNZ loop decrements then tests, so the final 0 slips through before exit. Kept because it's the class of bug (loop-boundary) that every hand-rolled VM and every compiled language inherits from its semantics.

19.3 The compiler: emit once

```
mov  w0, #6
mov  w1, #7
add  w0, w0, w1
mov  w1, #8
mul  w0, w0, w1
```

Figure 19.4: `ch18_cc` output: the same expression as 5 lines of arm64 — the compiler's advantage is that (a+b)×c becomes instructions *once*; there is no dispatch in the result.

`ch18_cc.c` emits the same kind of arithmetic as five arm64 instructions — no dispatch loop, no stack discipline at runtime. The constant operands are baked in at compile time (`# eval at compile time: 104`); the interpreter can never do that, because it reads the program *as data* every step.

19.4 The JIT compromise

tier	when it kicks in	cost model
interpreter	always	decode-per-step
baseline JIT	warm code	compile once, simple code
optimising JIT	hot code	speculates + deopts

Figure 19.5: The JIT answer to the divide: start interpreting, compile the hot paths at runtime. Python's CPython lives at row one; JVMs and JS engines climb all three.

19.5 The loop closes

this book built	the layer it closed
ch1 compile pipeline	source → binary
ch3–4 ISA/ABI	the contract
ch5–8 the core	the silicon
ch9–11 memory/OS	the environment
ch18 this machine	back to source

Figure 19.6: The loop closes: source text in, computation out, through every layer the book built. The 9-opcode VM is the same machine as ch8's SoC — one abstracts software, the other gates.

The VM is ch8's SoC abstracted: fetch-decode-execute with a `switch` for the decoder, a stack for the register file. The compiler is ch1's pipeline miniatured: AST to instructions. Everything between them — the ISA contract, the memory hierarchy, the OS services, the measurement discipline — is what this book spent 18 chapters building.

19.6 Common mistakes

- **"Interpreted = slow" without a number.** Ours was 2× dispatch overhead on a compute loop — real, and smaller than folklore suggests.
- **The off-by-one at the loop boundary.** The printed trailing 0 is the canonical VM bug; JNZ-after-decrement is where it lives.
- **Thinking the compiler is magic.** Ours is 20 lines — the hard parts (regalloc, optimisation) are real but the *idea* fits on a page.
- **JIT as free speed.** Speculation costs deopt paths and warmup; the interpreter always pays, the JIT pays twice if wrong.
- **Stack discipline bugs.** `DUP` before `OUT` preserves the loop counter — one misplaced stack op corrupts every later step.

19.7 Summary

Piece	**Verified version**
VM	9 opcodes, real run: 50 + countdown 3..0
overhead	28 dispatch steps for 14 ops = 2×
compiler	same expr → 5 arm64 lines, no dispatch
JIT	interpret cold, compile hot, speculate
closure	VM = ch8 SoC abstracted; book's loop closed

Further reading

- Crafting Interpreters (Nystrom): the book of this chapter. `craftinginterpreters.com`
- The Python bytecode interpreter (ceval.c): `github.com/python/cpython`

WATCH INSTEAD

- **Crafting Interpreters overview — Robert Nystrom talk** — the author's walkthrough of this exact build (verified 2026-10-11). `https://youtu.be/8lC9FciP8Xg`

Notes

- Both programs run verbatim on this machine; the trailing 0 in the countdown is the real JNZ-boundary bug, kept as a receipt.
- The compiler's output is arm64 text emitted by the program — the book ends where ch1 began, one level up.

Exercises

Exercise 19.1 (easy)**.** Why can't the interpreter constant-fold like the compiler did?

Solution. [hint] It reads operands as data per step — folding would need a pre-pass, i.e. compilation.

Exercise 19.2 (easy)**.** What does the 2× dispatch overhead buy the VM?

Solution. [hint] Portability and safety — no codegen, runs anywhere C does.

Exercise 19.3 (easy)**.** Which register does the stack machine never need to name?

Solution. [hint] All of them — operands are implicit on the stack.

Exercise 19.4 (easy)**.** Fix the countdown's trailing 0. One-line change?

Solution. [hint] Test before decrement (or JNZ on the pre-sub value).

Exercise 19.5 (medium)**.** Add a CALL/RET pair to the VM. What stack discipline do the caller and callee need that PUSH/ADD never did?

Solution. [hint] Return addresses — ch4's ABI arrives; a second stack or frame pointers solve it.

Exercise 19.6 (medium)**.** The compiler baked (6+7)*8=104 at build time. What optimisation pass is that, and when does it break (overflow, UB)?

Solution. [hint] Constant folding; breaks on signed overflow/UB per the language spec — C says fold anyway at -O.

Exercise 19.7 (medium)**.** Design the VM's dispatch differently: threaded code (jump table per opcode). What does it remove and what does it cost?

Solution. [hint] Removes the switch's branch per step; costs an indirect jump per instruction — predictors handle it differently.

Exercise 19.8 (medium)**.** JIT speculation: your engine bets the loop counter stays int. What must it emit to stay correct when it doesn't?

Solution. [hint] A guard + deopt path back to the interpreter — correctness before speed.

Exercise 19.9 (hard)**.** Prove or refute: the stack VM's $2\times$ dispatch overhead grows with expression complexity. Run the counterexample (a longer straight-line program) and explain the scaling.

Solution. [hint] Overhead is per-op constant — a longer program scales linearly, not worse; dispatch per step is the flat tax.

Exercise 19.10 (hard)**.** Write the register-based VM version of this machine (reg[16] instead of stack). Compare program byte length and dispatch count on the same computation — which machine wins and when?

Solution. [hint] Register code is longer but fewer ops (no DUP/POP noise); stack wins on compactness, registers on speed.

Exercise 19.11 (easy)**.** What is the interpreter's `ip` in CPU terms?

Solution. [hint] The program counter — fetch-decode-execute, in software.

Exercise 19.12 (medium)**.** Why is the stack machine's code denser than a register machine's? Give the byte-count argument on $(6\times7)+8$.

Solution. [hint] No operand fields: MUL is 1 byte; a register op needs src/dst fields.

Exercise 19.13 (medium)**.** The JIT tier's "speculates and deopts" — what assumption does a type-specialising JIT actually bet on?

Solution. [hint] That observed types are stable; the guard checks the bet per call.

Exercise 19.14 (medium)**.** Our VM's stack lives in a C array. What breaks if a program pushes past 256, and what's the honest fix?

Solution. [hint] Silent memory corruption — bounds-check push/pop or grow the stack; correctness first.

Exercise 19.15 (hard)**.** Design the minimal bytecode that a real JVM or WASM module would need beyond our 9 ops to run ch12's scheduler sim. List the semantic gaps (calls, locals, branches, memory).

Solution. [hint] CALL/RET, local load/store, structured branches, linear memory — the four gaps between a toy and a module.

Bibliography

[1] J. Sannemo, *Principles of Algorithmic Problem Solving*, 2025. https://usaco.guide/PAPS1.pdf

Index

-O0, 35
-O2, 35
`.data`, 6
`.text`, 6

ABI names, 23
address, 14
address space, 8
aligned access, 16
alignment, 14
ALU, 41
Amdahl, 109
array, 14
assembler, 4
assembly language, 23

B-type, 23
base+offset addressing, 25
benchmark, 109
biased exponent, 17
big-endian, 15
blocking, 76
Book 5, 9
branch, 23
branch prediction, 54
bypass, 53

cache, 68, 76
callee-saved, 33
caller-saved, 33
calling convention, 32
calling overhead, 34
compilation pipeline, 4
compiler, 4, 126
compiler output, 35
condition variable, 96
conditional branch, 26
context switch, 89
control hazard, 54
CPI, 43
cpu8, 9
CPUlator, 26
critical path, 43

data hazard, 49
data memory, 42
datapath, 40
deadlock, 96
decoder, 40
directives, 26

ELF, 4
endianness, 14
`execve`, 4, 8
exponent, 17

file system, 103
floating point, 14
flush, 54
forwarding, 53
fsync, 103
funct3, 24
funct7, 24
function, 1

hazard, 49

I-type, 23
IEEE 754, 14
IF/ID/EX/MEM/WB, 47
immediate, 24
immediate generator, 40
inode, 103
instruction encoding, 23
interpreter, 126

J-type, 23
jitter, 120
journal, 103

latency, 47
linker, 4

little-endian, 15
load, 23
load-use hazard, 54
load/store, 42
loader, 4
locality, 68
low latency, 120
LP64, 14

mantissa, 17
memory hierarchy, 68
memory mountain, 72
miss rate, 76
MLFQ, 89
mutex, 96

NEON, 114

object file, 4
`offsetof`, 16
opcode, 24
overflow, 15

padding, 16
page fault, 83
page table, 8, 83
PIC, 36
pipeline register, 47
pipelining, 47
pointer, 14
pointer arithmetic, 18
position-independent code, 36
preprocessor, 4
process, 89
psABI, 32
pseudoinstruction, 23

R-type, 23
`ra`, 32
race condition, 96
register
    callee-saved, 32
    caller-saved, 32
register file, 23, 41
relocation, 4
relocation record, 7
ring buffer, 120
RV32I, 9, 23
RV32I core, 40

S-type, 23
scheduler, 89
section (ELF), 6
sign bit, 15
SIMD, 114
single-cycle processor, 40
`sp`, 32
stack frame, 32
stall, 54
store, 23
struct padding, 14
style, 1
sv32, 83
symbol (linker), 7
symbol table, 4

tail latency, 109
throughput, 47
tiling, 76
timing diagram, 48
TLB, 83
two's complement, 14
type punning, 14

U-type, 23
undefined symbol, 7

vectorisation, 114
virtual machine, 126
virtual memory, 83

word, 14

X-propagation, 55
x0, 23

zero register, 23